



Ingeniería Civil Informática

Inteligencia artificial

Clase 3: Resolución de Problemas
mediante Búsqueda No Informada en
Contextos de Ingeniería

Dr(c). Francisco Philip Vásquez Iglesias



Resultados de aprendizaje

- 1.- Conocer la historia y el contexto de la inteligencia artificial considerando la multiplicidad de opciones de aplicación en la ingeniería.
- 2.- Resolver problemas mediante técnicas de búsqueda en grafos en contextos de ingeniería.
- 3.- Diseñar soluciones a problemas con incerteza mediante razonamiento Bayesiano documentándola de acuerdo a pautas establecidas.
- 4.- Establecer relaciones y proponer modelos mediante técnicas de aprendizaje automático considerando un trabajo interdisciplinario en contextos de la ingeniería.

Introducción



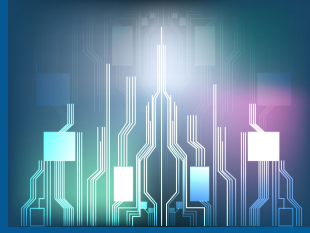
Inteligencia Artificial Clásica vs. Aprendizaje de máquina

1. **Machine Learning** (El enfoque de los Datos)

- a. Le damos a la computadora miles de ejemplos (datos) para que encuentre patrones ocultos.
- b. La salida: Una probabilidad o una predicción. No hay 100% de certeza.
- c. Ejemplo: Reconocer si en una foto hay un gato o un perro.

2. **IA Clásica / Búsqueda** (El enfoque de las Reglas)

- a. Conocemos las reglas del mundo y el objetivo, pero el número de combinaciones es tan grande que un humano no puede procesarlo.
- b. La salida: Una secuencia de acciones matemáticas exactas. 100% de certeza y optimalidad.
- c. Ejemplo: Encontrar la ruta más corta en un laberinto o diseñar una placa de circuitos.



¿Dónde usamos la Búsqueda en Grafos en Ingeniería?

- **Robótica y Manufactura (Planificación de Trayectorias):** ¿Cómo se mueve un brazo robótico desde el punto A al punto B sin chocar con obstáculos? El espacio 3D se discretiza en un grafo y se busca la ruta más corta o segura.
- **Logística y Transporte (Ruteo de Flotas):** El GPS de tu teléfono o el sistema de despacho de camiones mineros modelan las calles o caminos como grafos ponderados (donde el "peso" es el tráfico o la distancia).
- **Electrónica y Telecomunicaciones (Ruteo VLSI / PCB):** Conectar miles de componentes en una placa de circuito impreso sin que las pistas se crucen, minimizando el retardo eléctrico y el uso de cobre.

El desafío de la Explosión Combinatoria

- Para calcular la cantidad de combinaciones en un grafo de $n=5$ puntos, el resultado matemático depende de la estructura exacta que el algoritmo intenta resolver.
- Para problemas de Diseño de Redes (como el MST), donde el objetivo es conectar todos los puntos sin formar ciclos (ideal para fibra óptica o cañerías de agua), la cantidad de topologías posibles en un grafo completo se define por la fórmula del matemático Arthur Cayley: n^{n-2} .
 - Reemplazando el valor: $5^{5-2} = 5^3$
 - El cálculo final es $5 \times 5 \times 5 = 125$ árboles o combinaciones posibles.

El desafío de la Explosión Combinatoria

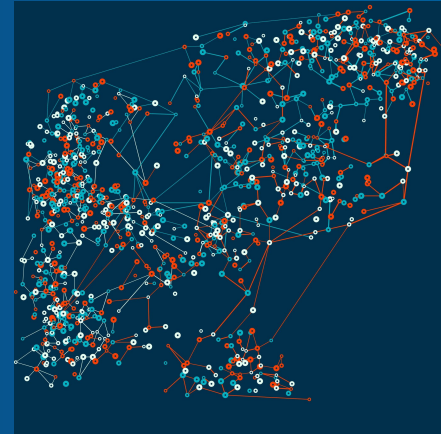
- Para problemas de Rutas y Recorridos (como el TSP), donde el objetivo es visitar cada punto una vez y volver al inicio (ideal para ruteo de correspondencia), el cálculo se basa en permutaciones factoriales.
- Si se fija un punto de partida para evitar contar la misma ruta desde distintos inicios, la fórmula es $(n-1)!$.
 - Reemplazando el valor: $(5-1)! = 4!$
 - El cálculo final es $4 \times 3 \times 2 \times 1 = 24$ rutas únicas (o 120 si contamos todas las permutaciones de secuencia en bruto con $5!$).

¿Cual es el problema?

- La intuición humana no es suficiente para optimizar redes.
- Conectar de forma óptima solo 5 puntos ya genera 125 combinaciones. Al subir a 10 puntos, la cifra se dispara a 100.000.000 de combinaciones.
- Por qué falla el "ojo humano":
 - Nos dejamos engañar por las distancias locales aparentes.
 - Somos incapaces de calcular mentalmente el impacto global que tiene cambiar una sola conexión en el resto de la red.
- La solución: Algoritmos Sistemáticos
 - Permiten la evaluación matemática estricta de las rutas válidas.
 - Garantizan encontrar el óptimo global (o aproximaciones altamente precisas) donde la intuición fracasa.

La Abstracción: De la Realidad al Modelo Matemático

- ¿Qué es abstraer? Es el proceso de eliminar todos los detalles del mundo real que no afectan la resolución de nuestro problema.
- La computadora no entiende de "edificios", "estacionamientos" o "clima".
- Solo entiende de estructuras de datos.
- El Diccionario de Traducción:
 - Intersecciones/Edificios → Nodos (Vértices)
 - Calles/Ductos/Cables → Aristas (Edges)
 - Distancia/Dinero/Tiempo → Costo o Peso de la Arista.

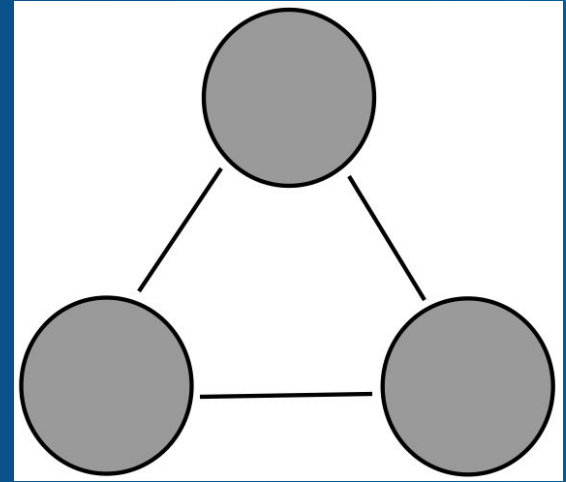


Grafos

EFDS

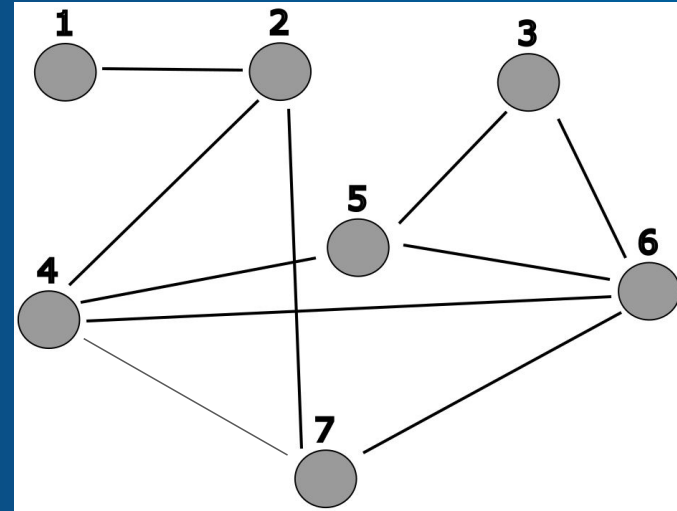
Grafos

- Un grafo es un conjunto de ***vértices o nodos*** unidos por ***aristas o arcos***.
- Cada arista se especifica por un par de nodos.
- Gráficamente, los vértices son puntos y las aristas son líneas que los unen. Un grafo se dice ***trivial*** si tiene un solo vértice.
- Permiten la representación de varios tipos de problemas.



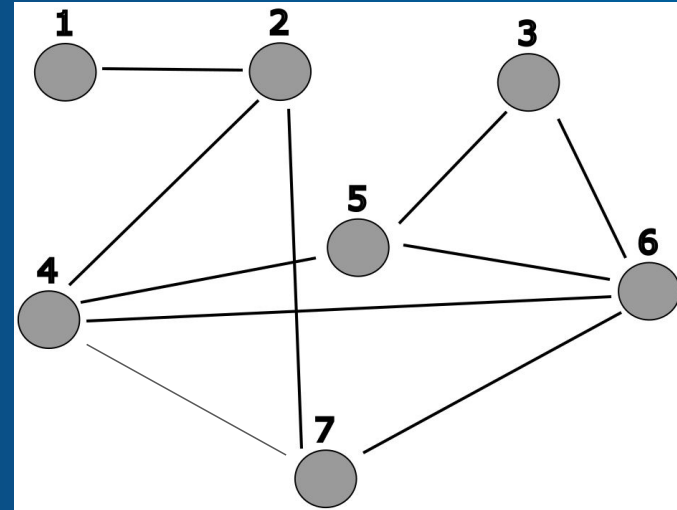
Definición de un grafo

- Un grafo G está formado por un par $(V(G), E(G))$:
 - $V(G)$: es un conjunto finito de vértices de G .
 - $E(G)$: es un conjunto de pares no ordenados de vértices distintos de G (aristas), que se describen como ij o (i,j) .
 - Llamaremos n a la cantidad de nodos de un grafo $|V(G)|$ y m a la cantidad de aristas $|E(G)|$.



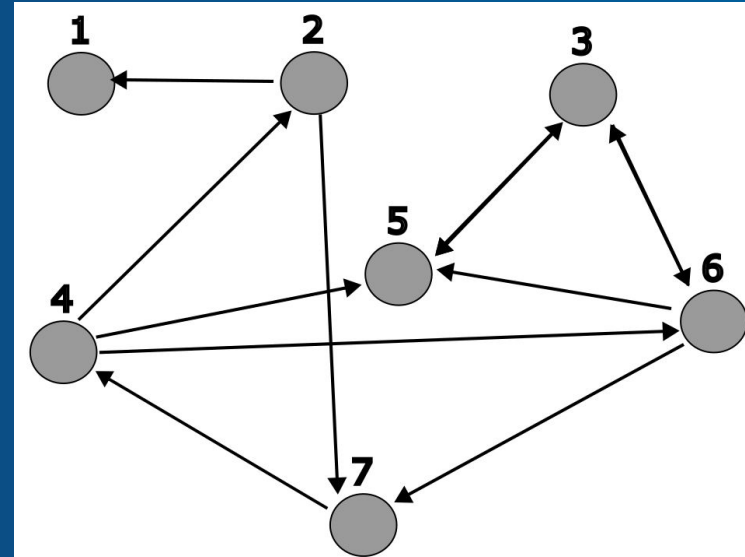
Definición de un grafo

- $V(G) = \{1, 2, 3, 4, 5, 6, 7\}$.
- $E(G) = \{(1, 2), (2, 4), (2, 7), (3, 5), (3, 6), (4, 5), (4, 6), (4, 7), (5, 6), (6, 7)\}$
- $n = 7$.
- $m = 10$.



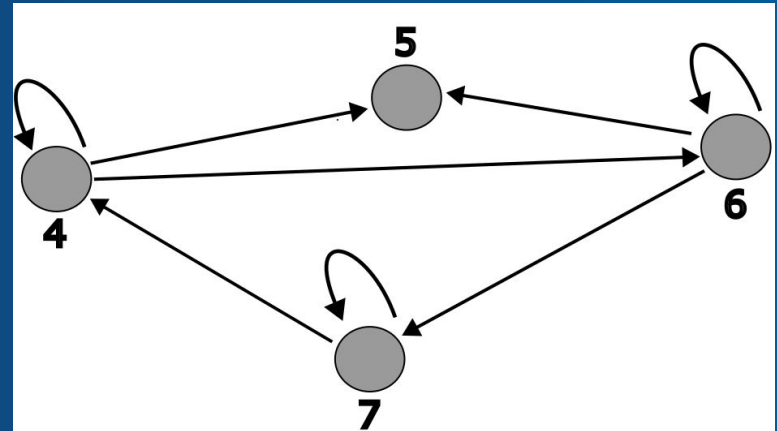
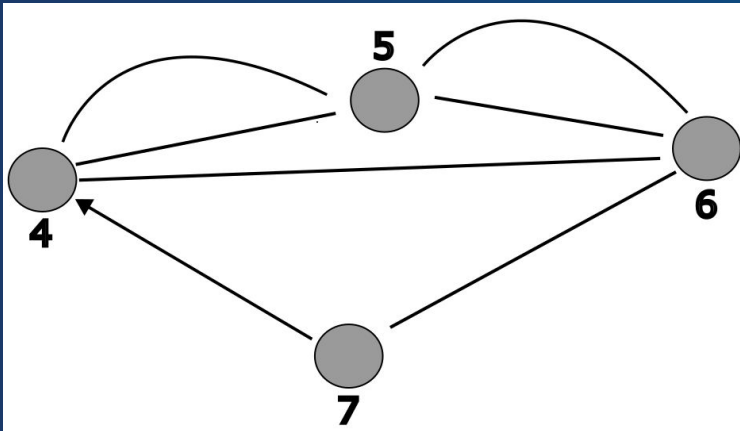
Digrafo o grafo dirigido

- Es un grafo tal que, sus aristas están dadas por un conjunto de pares ordenados de vértices.
- $V(G) = \{1, 2, 3, 4, 5, 6, 7\}$.
- $E(G) = \{(2, 1), (4, 2), (2, 7), (3, 5), (5, 3), (3, 6), (6, 3), (4, 5), (4, 6), (7, 4), (6, 5), (6, 7)\}$
- $n = 7$.
- $m = 12$.



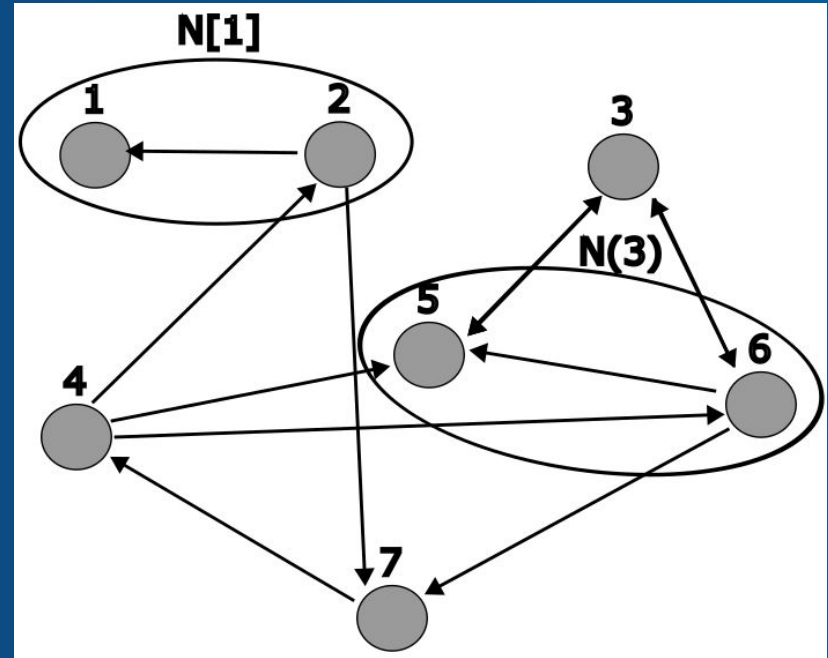
Multigrafo y pseudografo

- **Multigrafos:** son grafos que permiten más de una arista entre los mismos par de vértices.
- **Pseudografos:** son grafos que permiten aristas entre un mismo nodo, generando loops, representados como (v, v) .



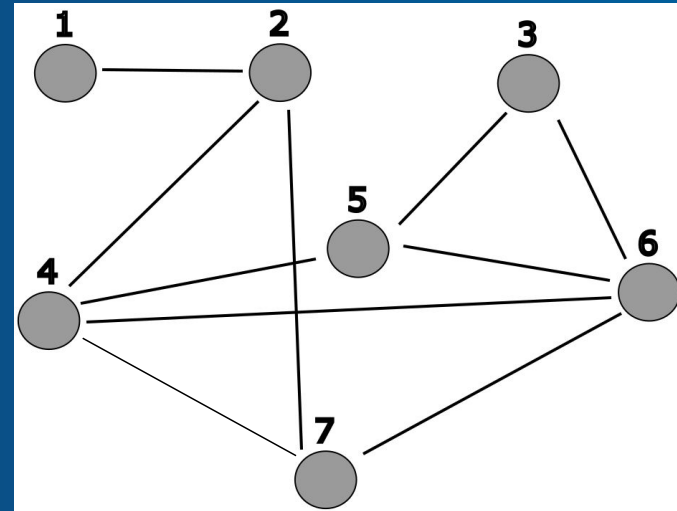
Adyacencia y Vecindarios

- Un vértice v es **adyacente** a otro vértice w en G , si $(v, w) \in E(G)$.
- También se puede decir como “ v y w son los extremos de una arista”.
- El vecindario de un vértice v en un grafo G es el conjunto $N_G(v)$ que consiste de todos los vértices adyacentes a v .
- El vecindario cerrado de v es $N_G[v] = N_G(v) \cup \{v\}$.



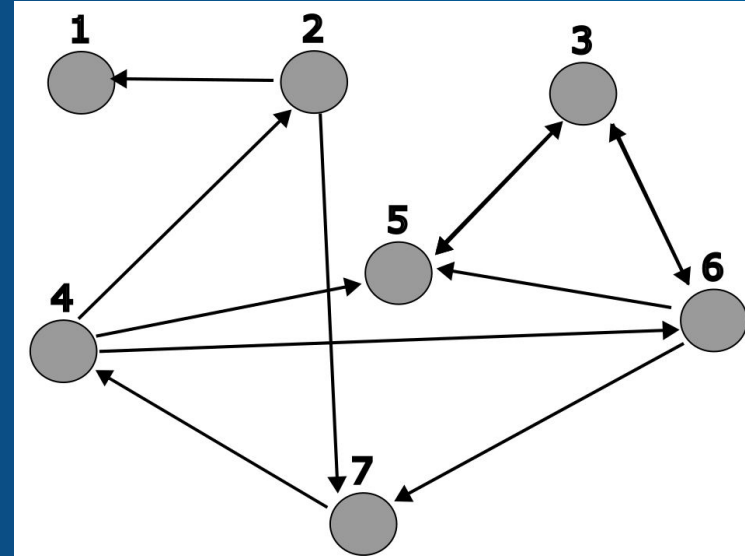
Grado

- El grado de un vértice v en G , es la cardinalidad del conjunto $N_G(v)$ y se define como $d_G(v)$.
- Si no hay ambigüedad, se usa $d(v)$.
- Dado un grafo G , se define como $\delta(G)$ al grado mínimo y $\Delta(G)$ al grado máximo entre los vértices de G .
- En el ejemplo:
 - $d(2) = 3, d(1) = 1, d(6) = 4$.
 - $\delta(G) = 1$ y $\Delta(G) = 4$.



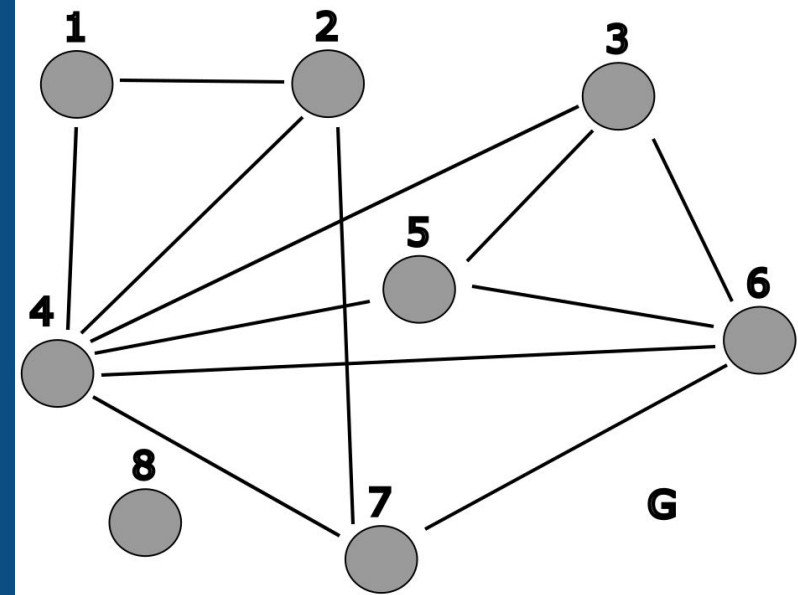
Grado en un grafo dirigido

- En un digrafo, cada nodo tiene dos tipos de grado:
 - Grado de entrada (in-degree): número de aristas que entran al nodo.
 - Grado de salida (out-degree): número de aristas que salen del nodo.



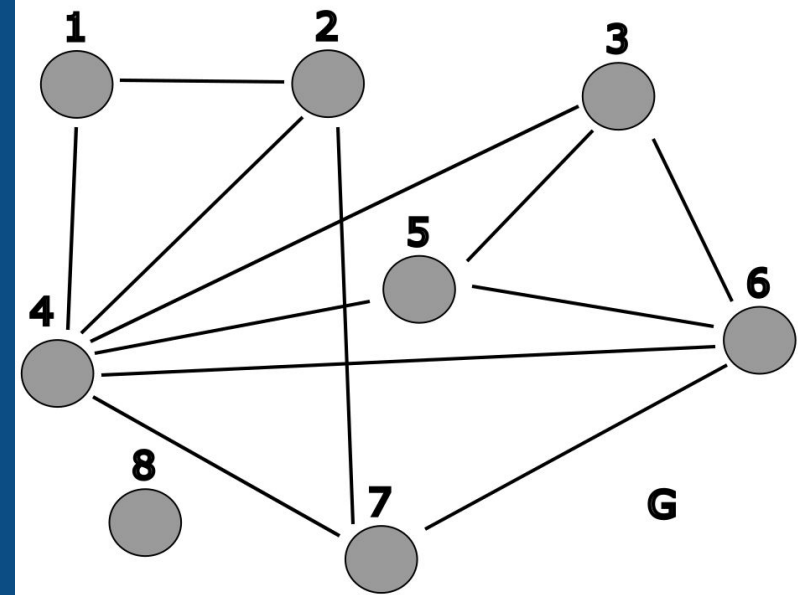
Grado

- Un vértice v es aislado cuando $N(v) = \emptyset$ (vecindad vacía).
- Es equivalente a $d(v) = 0$.
- Un vértice v es universal cuando $N(v) = V(G) - \{v\}$, o equivalentemente $d(v) = n - 1$.
 - El vértice 8 es aislado en G .
 - El vértice 4 es universal en $G - \{8\}$.



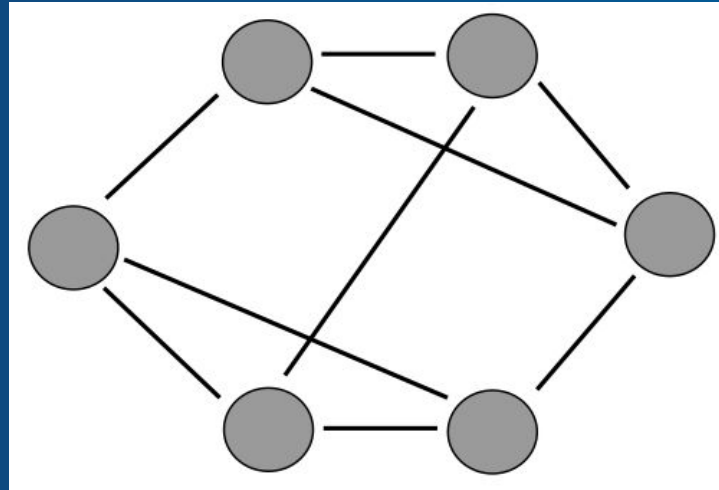
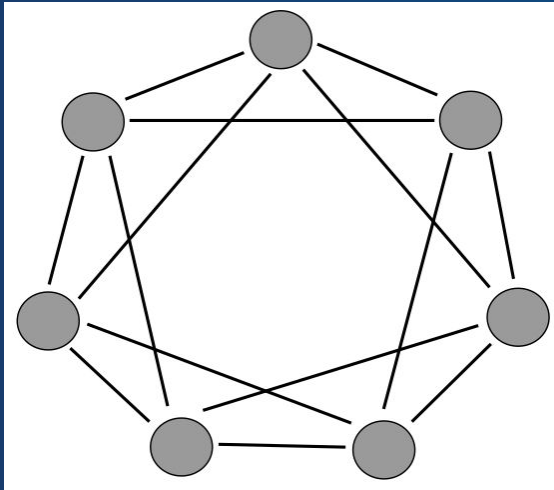
Grado

- Si G es no trivial y tiene un vértice aislado no puede tener también uno universal.



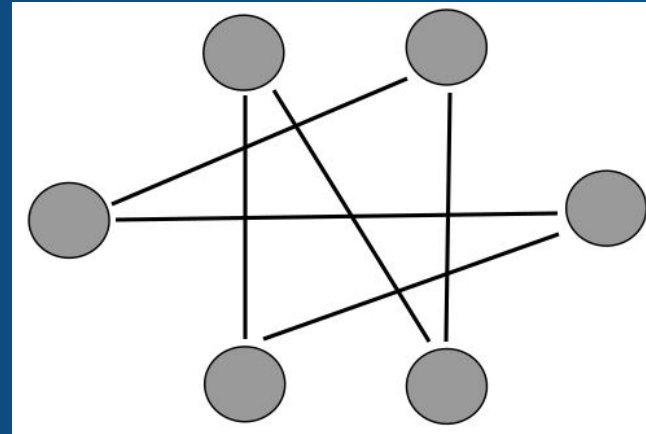
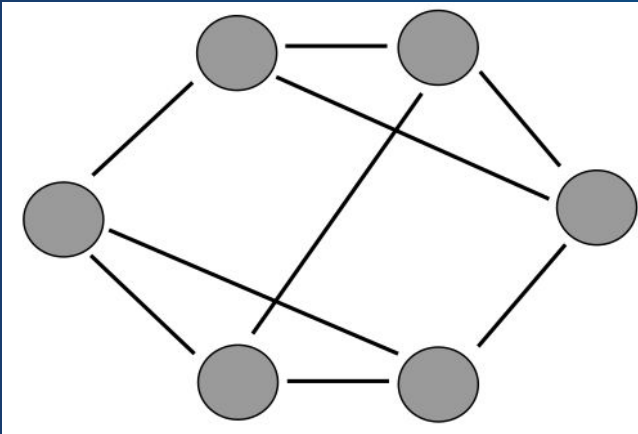
Grafo regular

- Un grafo se dice regular, si todos sus vértices tienen el mismo grado.
- Un grafo se dice cúbico si todos sus vértices tienen grado tres.
 - Todo grafo cúbico tiene un número par de vértices.



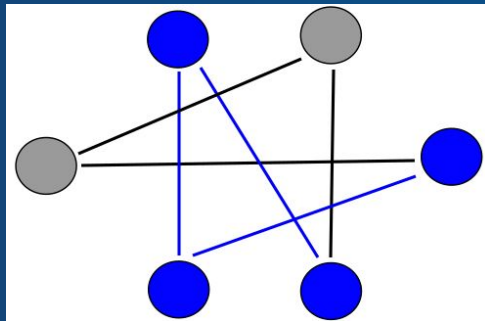
Complemento de un grafo

- El complemento de un grafo G , denotado por $\bar{G}$.
- Es el grafo que tiene el mismo conjunto de vértices de G y tal que: dos vértices distintos son adyacentes en $\bar{G}$ si y sólo si no son adyacentes en G .



Subgrafos

- Un grafo H es un subgrafo de un grafo G si $V(H) \subseteq V(G)$ y $E(H) \subseteq E(G)$.
 - Si $V(H) = V(G)$, decimos que H es un subgrafo generador de G .
- Dado un conjunto de vértices $X \subseteq V(G)$, el subgrafo de G inducido por X es el subgrafo H de G , tal que:
 - $V(H) = X$.
 - $E(H)$ es el conjunto de aristas de G que tiene ambos extremos en X .
- Notación: Si $v \in V(G)$, $G-v$ denota el subgrafo de G inducido por $V(G) - \{v\}$.

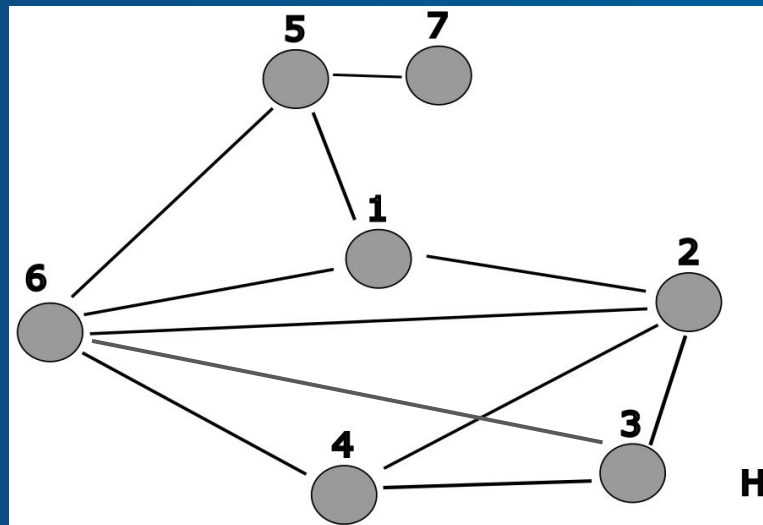
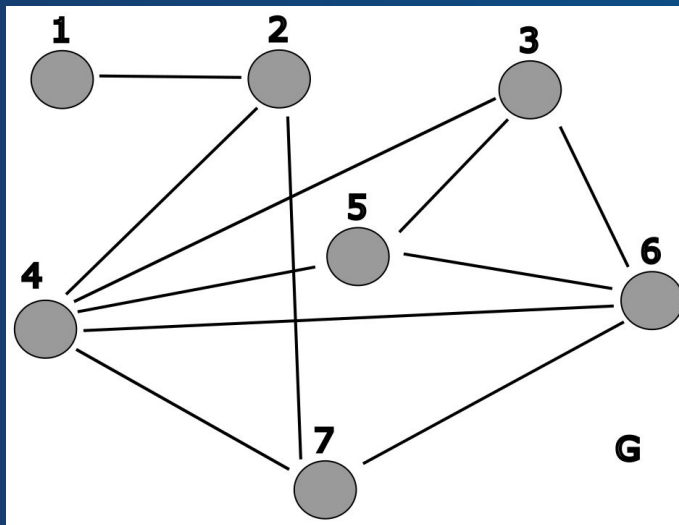


Isomorfismo

- Dos grafos G y H son isomorfos si existe una biyección entre $V(G)$ y $V(H)$ que conserva las adyacencias.
- Permite definir que $G = H$.
- G y H son isomorfos si $f: V(G) \rightarrow V(H)$ biyectiva, tal que $(v, w) \in E(G)$, si y sólo si $(f(v), f(w)) \in E(H)$.
- En resumen, es una relación de equivalencia.

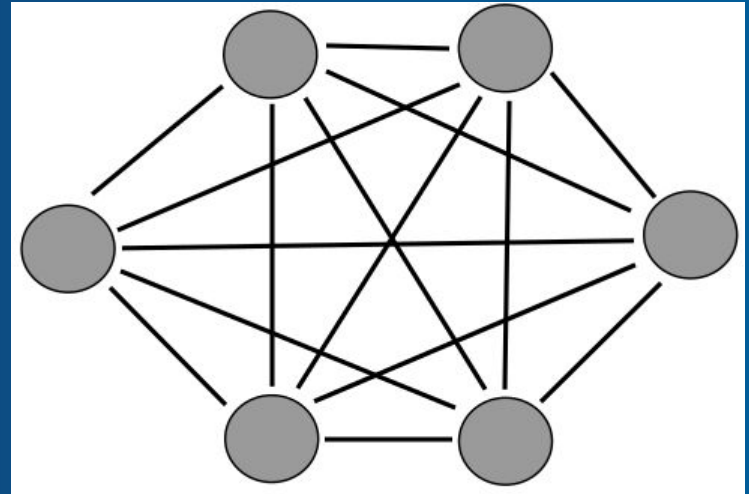
Ejemplo de isomorfismo

- $F(1) = 7.$
- $F(2) = 5.$
- $F(3) = 3.$
- $F(4) = 6.$
- $F(5) = 4.$
- $F(6) = 2.$
- $F(7) = 1.$



Grafos completos

- Un grafo G es completo si cualquier par de vértices distintos de G son adyacentes.
- Se define K_n al grafo completo con n vértices.
- Al K_3 se le llama también Triángulo.
- ¿Cuánto valen $m(K_n)$, $\delta(K_n)$ y $\Delta(K_n)$?

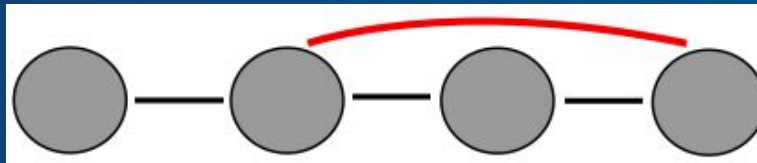


Caminos en grafos



Caminos en grafos

- Es una secuencia de vértices distintos $P = v_1, v_2, \dots, v_k$, donde $(v_i, v_{i+1}) \in E(G)$, $i = 1, 2, \dots, k-1$.
- Una cuerda de P es una arista que une dos vértices no consecutivos de P .
- Un camino inducido es un camino sin cuerdas. Denotamos por P_k al camino inducido de k vértices.

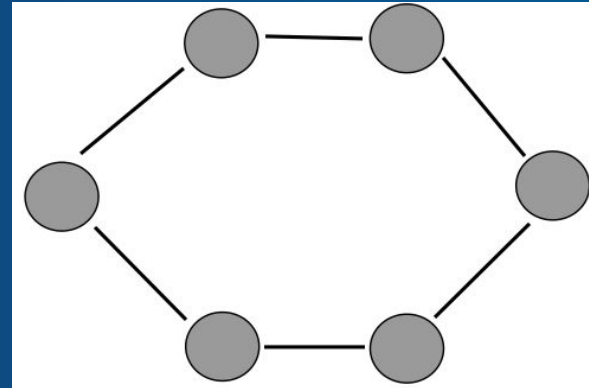
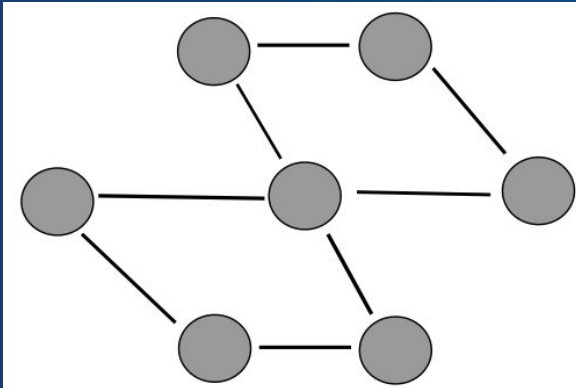


Circuitos en grafos



Circuitos en grafos

- Un circuito en un grafo G , es una secuencia de vértices $C = v_1, v_2, \dots, v_k$, no necesariamente distintos, donde $v_1 = v_k$ y $(v_i, v_{i+1}) \in E(G)$, $i = 1, \dots, k-1$.
- Si $k \geq 3$ y $v_1, v_2, \dots, v_{k-1}$ son distintos, C se llama *ciclo*.

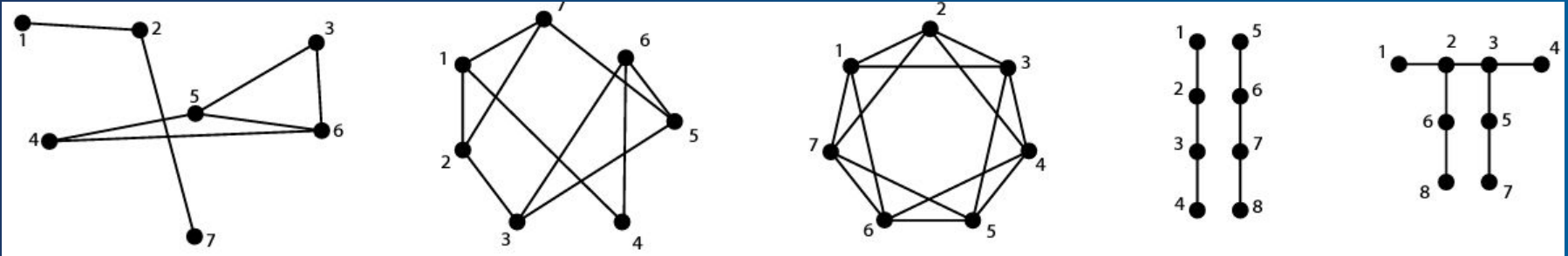


Conexión en grafos



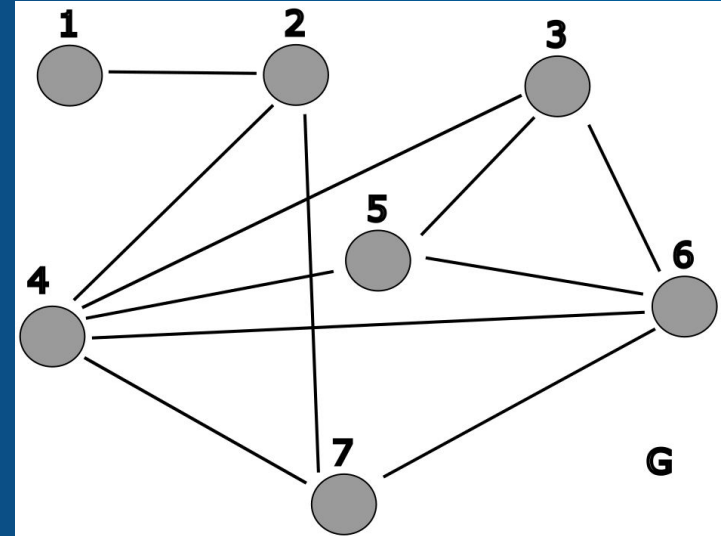
Grafos conexos

- Un grafo G es conexo si para todo par de vértices distintos v y w de G , existe un camino de v a w .
- ¿Cuáles de los siguientes grafos son conexos?



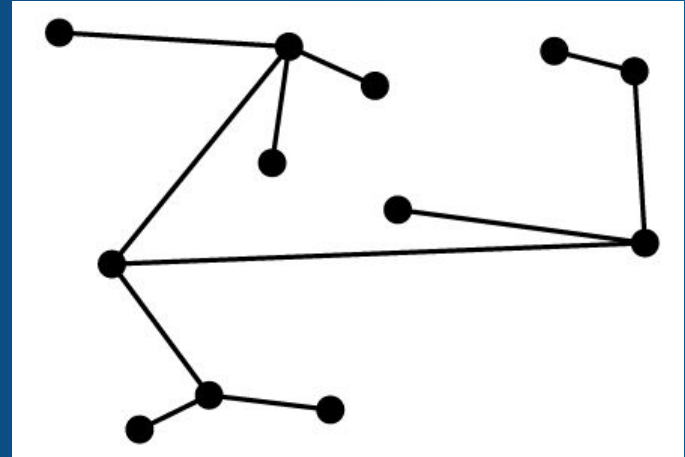
Distancia

- La longitud de un camino se mide por la cantidad de aristas que lo componen.
- La distancia entre dos vértices v y w en G es la longitud del camino más corto entre v y w , se define como $d_G(v, w)$.
- Si el contexto no es ambiguo, se abrevia $d(v, w)$.
- ¿Cuál sería la distancia entre 1 y 5?



Un árbol

- Es un grafo:
 - **Conexo:** todo par de vértices de G está unido por un único camino.
 - **Acíclico:** no tiene ciclos



Grafo pesado



Grafo pesado

- Un grafo pesado es un grafo donde sus aristas tienen asociado un peso.
- En las aplicaciones reales de grafos, las distintas aristas presentes no necesariamente tienen los mismos pesos.
- El peso de un subgrafo es la suma de los pesos de sus aristas.
- Un grafo podría modelar un mapa con la ubicación de vértices de interés, donde dirigirse de un punto A a un punto B puede tener un valor de costo diferente según el camino elegido para recorrer la ruta.
- Estos costes serán asociados a cada arista del grafo y puede representar una distancia, un coste en combustible, una dificultad de recorrido, etc.

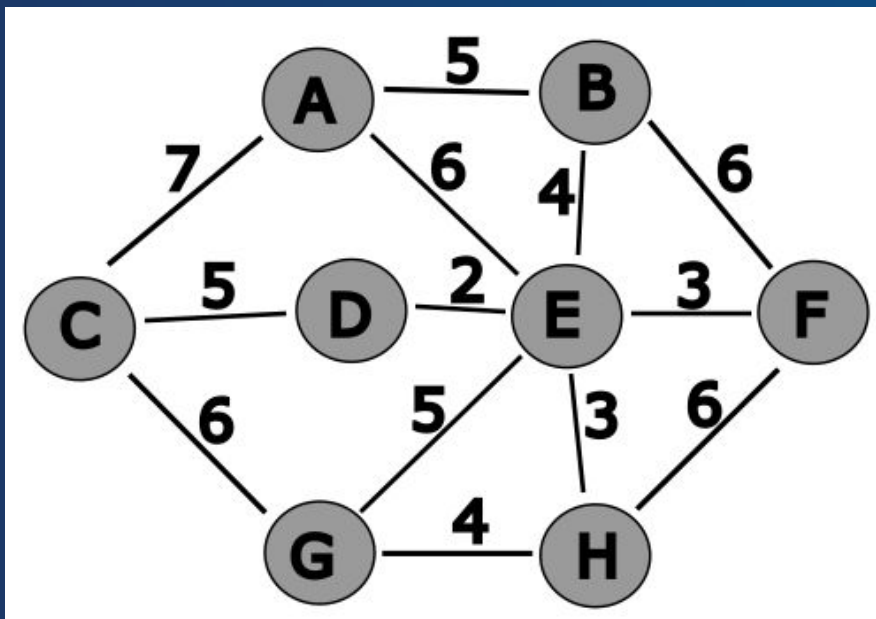
Representación de un grafo como matriz



Representación de un grafo como matriz

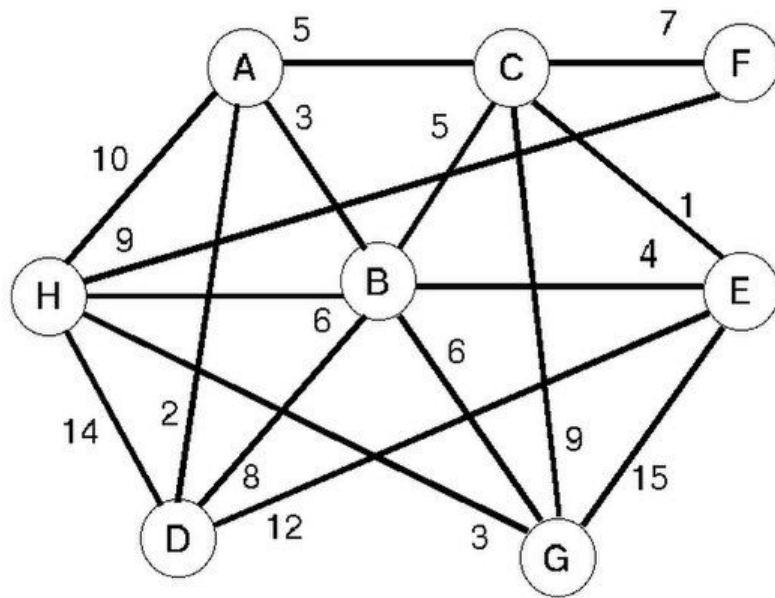
Z	A	B	C	D	E	F	G	H
A	-	5	7	-	6	-	-	-
B	5	-	-	-	4	6	-	-
C	7	-	-	5	-	-	6	-
D	-	-	5	-	2	-	-	-
E	6	4	-	2	-	3	5	3
F	-	6	-	-	3	-	-	6
G	-	-	6	-	5	-	-	4
H	-	-	-	-	3	6	4	-

Representación matricial para el árbol Z

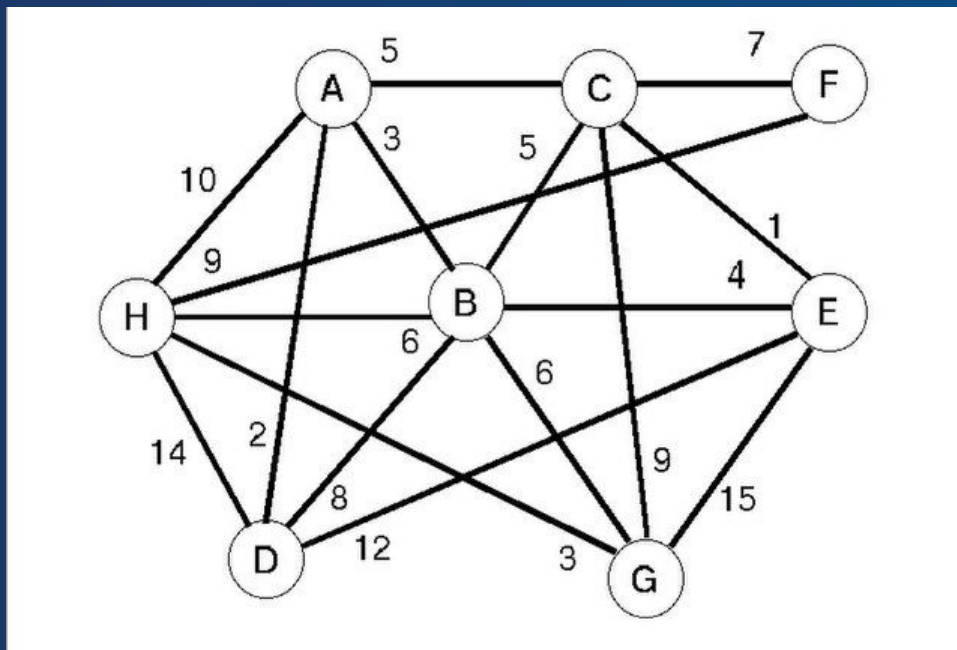


Z	A	B	C	D	E	F	G	H
A	-	5	7	-	6	-	-	-
B	5	-	-	-	4	6	-	-
C	7	-	-	5	-	-	6	-
D	-	-	5	-	2	-	-	-
E	6	4	-	2	-	3	5	3
F	-	6	-	-	3	-	-	6
G	-	-	6	-	5	-	-	4
H	-	-	-	-	3	6	4	-

Llevar a representación matricial



Ejercicio 1: Aplicar Kruskal para generar un árbol de expansión mínima



Z	A	B	C	D	E	F	G	H
A	-	3	5	2	-	-	-	10
B	3	-	5	6	4	-	6	6
C	5	5	-	-	1	7	9	-
D	2	6	-	-	12	-	-	14
E	-	4	1	12	-	-	15	-
F	-	-	7	-	-	-	-	9
G	-	6	9	-	15	-	-	3
H	10	6	-	14	-	9	3	-

Lista de adyacencia

- Es un "Diccionario" o un Arreglo de Listas.
Tenemos una entrada principal para cada Nodo del grafo.
- A cada entrada se le asocia una lista dinámica que contiene únicamente a sus vecinos directos.
- Variante Ponderada: En la lista se guardan tuplas (Vecino, Costo)

Z	A	B	C	D	E	F	G	H
A	-	7	11	-	2	13	-	5
B	7	-	3	-	7	5	13	-
C	11	3	-	11	5	-	-	2
D	-	-	11	-	-	13	7	11
E	2	7	5	-	-	7	7	3
F	13	5	-	13	7	-	3	-
G	-	13	-	7	7	3	-	5
H	5	-	2	11	3	-	5	-



```
A -> (B, 7), (C, 11), (E, 2), (F, 13), (H, 5)
B -> (A, 7), (C, 3), (E, 7), (F, 5), (G, 13)
C -> (A, 11), (B, 3), (D, 11), (E, 5), (H, 2)
D -> (C, 11), (F, 13), (G, 7), (H, 11)
E -> (A, 2), (B, 7), (C, 5), (F, 7), (G, 7), (H, 3)
F -> (A, 13), (B, 5), (D, 13), (E, 7), (G, 3)
G -> (B, 13), (D, 7), (E, 7), (F, 3), (H, 5)
H -> (A, 5), (C, 2), (D, 11), (E, 3), (G, 5)
```

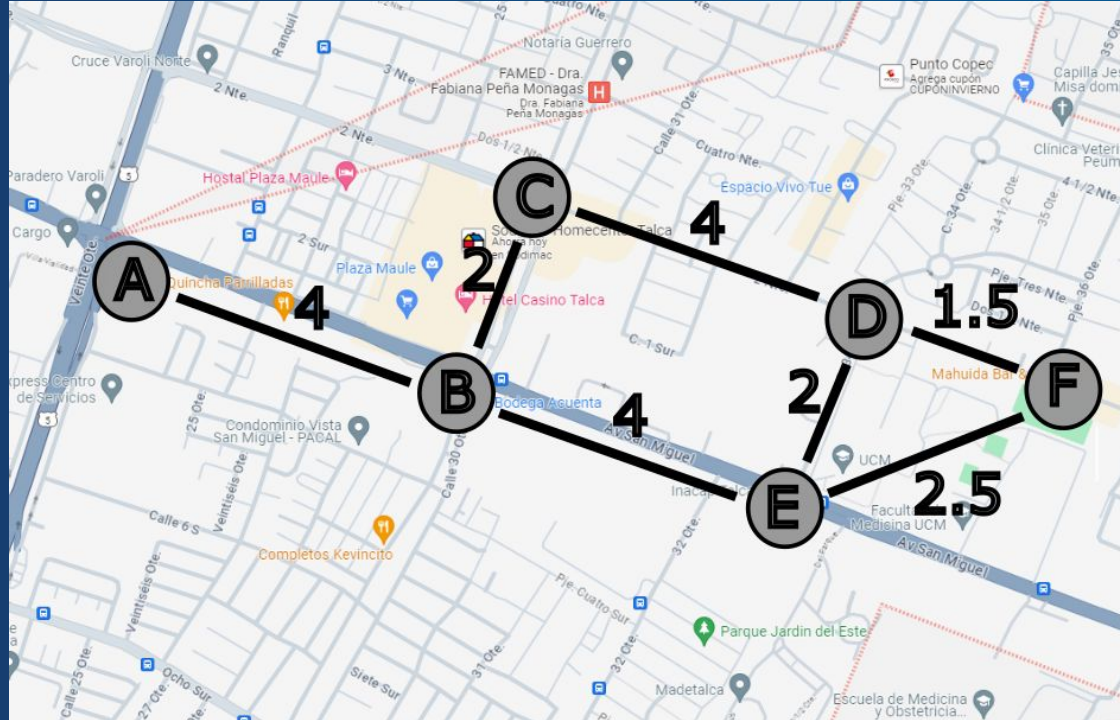
¿Cuál utilizar?

- La Matriz de Adyacencia: Ideal para: Grafos Densos (Casi todos los nodos están conectados con todos).
 - **Pro:** Consultar si A está conectado con D toma un instante ($O(1)$).
 - **Contra:** Gasta memoria $O(V^2)$. En un grafo de 10,000 nodos, creará una tabla de 100 millones de celdas... ¡casi todas llenas de ceros! (Memoria desperdiciada).

¿Cuál utilizar?

- La Lista de Adyacencia: Ideal para Grafos Dispersos (La mayoría de los nodos solo tienen un par de vecinos). ¡El 99% de los problemas de ingeniería!
 - Pro (Memoria): Solo gasta memoria para las conexiones que existen. $O(V + E)$.
 - Pro (Algoritmos): Perfecta para IA, ya que el algoritmo siempre pregunta "dame TODOS tus vecinos", y la lista los entrega de inmediato.

¿Cuál sería el camino más corto para llegar desde Varoli a la Facultad?



*Problema del vendedor
viajero (TSP)*

Definición del problema del vendedor viajero (TSP)

“Dada una lista de ciudades y las distancias entre cada par de ellas, ¿Cuál es la ruta más corta posible que visita cada ciudad exactamente una vez y regresa a la ciudad de origen?”

Problema del vendedor viajero

- Es un problema clásico de la computación.
- Fue formulado por primera vez en 1930 y es uno de los problemas de optimización más estudiados.
- Es un problema computacionalmente complejo, aunque existen muchas formas de resolverlo.
- Una gran cantidad de heurísticas y métodos exactos son conocidos por poder resolver el TSP desde cien hasta miles de ciudades.

Aplicaciones del TSP

- Algunas aplicaciones del TSP son:
 - La planificación.
 - Logística.
 - Fabricación de microchips.
 - Secuencias de ADN.
 - Etc.

TSP respecto a la complejidad computacional

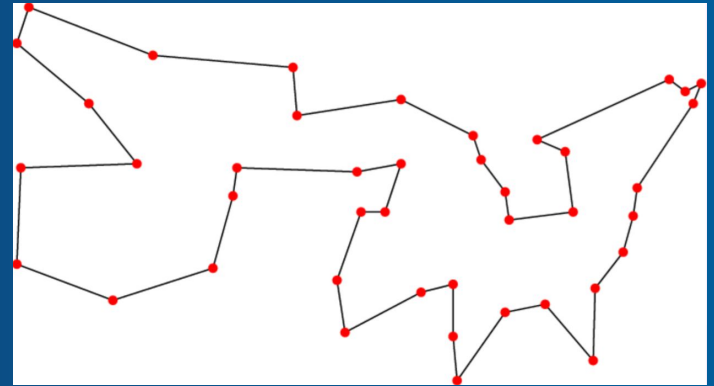
- TSP pertenece a los problemas NP-completo, osea que en el peor de los casos, el tiempo de ejecución para cualquier algoritmo que resuelva el TSP aumente de ***forma exponencial*** con respecto al número de ciudades (Applegate, Bixby, Chvátal, & Cook, 2006).

Algoritmo del vecino más próximo



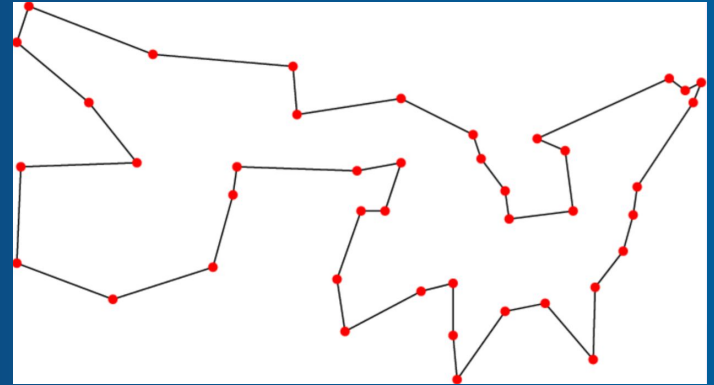
Algoritmo del vecino más próximo (KNN)

- Es una heurística común para obtener soluciones subóptimas rápidamente en el TSP.
- Se llama vecino más cercano debido a que en cada ciudad visitada se continúa con la ciudad disponible más cercana que haya en el mapa.
- Pertenece a la categoría de los algoritmos Greedy o Voraces.



Algoritmo del vecino más próximo (KNN)

- Es una solución de fácil implementación, pero en el TSP que suele tener problemas en los últimos nodos, ya que el agotamiento de posibilidades provoca malas elecciones de caminos.
- ***El resultado obtenido depende de la ciudad de inicio.***



Ejemplos aplicados de TSP



***Ejemplo 1: recorrido de
131 ciudades***

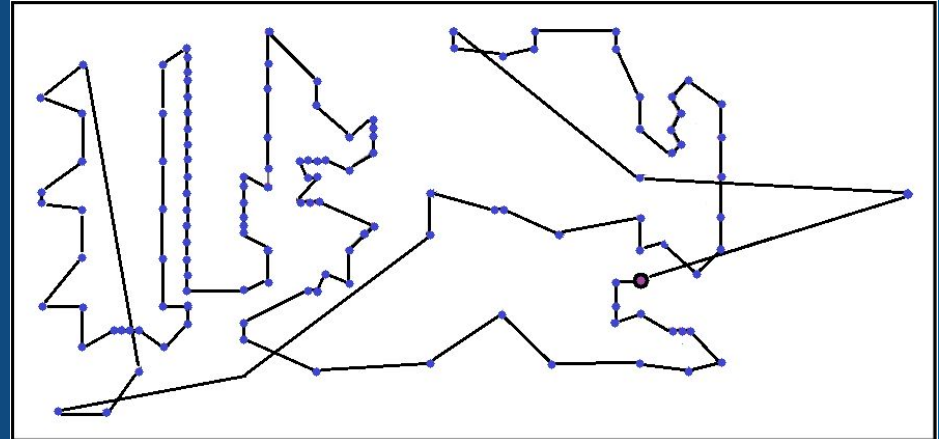
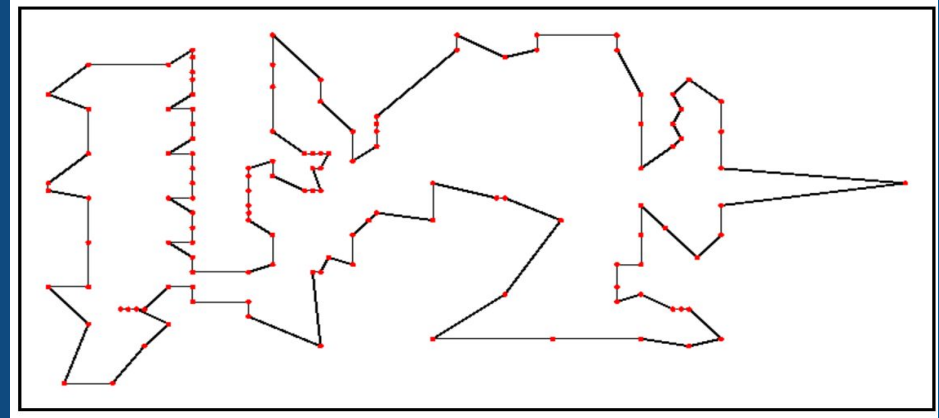


Problema resuelto en años anteriores

- Se solicita encontrar una ruta óptima en un problema de vendedor viajero que posee un recorrido de 131 ciudades, las ciudades no se pueden repetir y el recorrido debe terminar en la misma ciudad en la que empieza.
- Un recorrido aleatorio posee costos de distancia recorrida que se encuentran entre los 4.000 km y 5.200 km, por lo que se aplicará el método Greedy para hallar una mejor ruta.

Mejor ruta contra la encontrada por el KNN

- **Mejor resultado entre todas las combinaciones:** 613.034.
- **Peor resultado KNN:** inicio en la ciudad 79, con un coste de 800.6606.
- **Mejor resultado KNN:** inicio en la ciudad 106, con un coste de 684.4484.



Métricas de distancia



1. Distancias Geométricas Clásicas (Métricas Espaciales)

- Basadas en coordenadas en un plano (espacio Euclidiano).
- Son las más comunes en problemas teóricos y de robótica.
 - **Distancia Euclidiana:** La "línea recta" entre dos puntos. Es la distancia más corta posible en un plano.
 - **Distancia Manhattan o Taxicab:** Mide la distancia asumiendo que solo te puedes mover en ángulos rectos (ortogonales), como un taxi navegando por una ciudad con calles cuadriculadas (como Manhattan).
 - **Distancia de Chebyshev:** También conocida como la "distancia del Rey del ajedrez". Representa la máxima distancia a lo largo de cualquier eje coordenado. Asume que moverse en diagonal cuesta lo mismo que moverse en línea recta.

2. Distancias Geográficas (Mundo Real)

- Cuando los nodos son puntos geográficos en un mapa real y las distancias son lo suficientemente grandes como para que la curvatura de la Tierra importe.
 - **Distancia Haversine (o del Gran Círculo):** Calcula la distancia más corta entre dos puntos sobre la superficie de una esfera, dados sus valores de latitud y longitud. Ignora por completo que la Tierra es ligeramente achatada, asumiendo una esfera perfecta.
 - **Distancia de Vincenty (o Geodésica elipsoidal):** Una versión mucho más precisa que Haversine. Utiliza modelos elipsoidales de la Tierra (como el WGS 84). Es iterativa y computacionalmente más pesada.

3. Distancias Topológicas y de Red (Ingeniería Práctica)

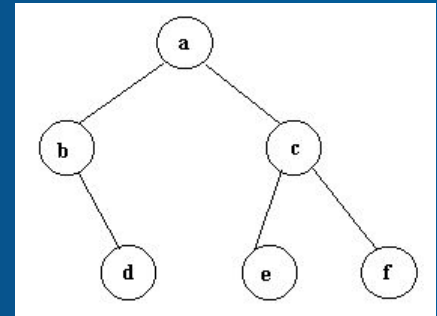
- En el ruteo urbano o logístico, la línea recta casi nunca existe.
- Los vehículos están limitados a una red de caminos (un grafo subyacente).
 - **Distancia de Red (Red de Calles):** La distancia real viajando a través del sistema de carreteras disponible. No se calcula con una fórmula matemática directa entre dos puntos (X,Y), sino resolviendo un sub-problema de Dijkstra/UCS sobre el mapa de la ciudad para encontrar el camino más corto entre el nodo A y el nodo B.
 - **Distancia Asimétrica (Tiempo de viaje/Costo direccional):** Transforma el TSP clásico en un ATSP (Asymmetric TSP). Aquí, la distancia del nodo A al B no es igual que del B al A ($\text{dist}(A,B) \neq \text{dist}(B,A)$).

TDA Árbol

ES
D

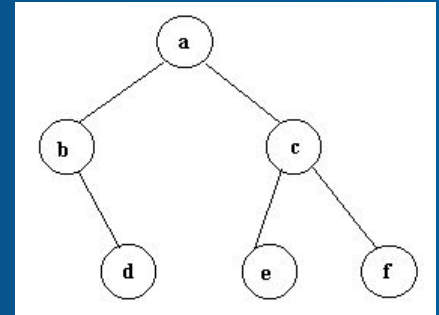
TDA Árbol

- Es una estructura de datos que puede ser definida como:
 - Una estructura vacía o...
 - Un elemento o clave de información (**nodo**) más un número finito de estructuras tipo árbol, disjuntos, llamados subárboles.



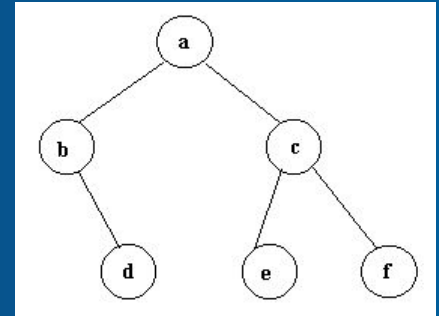
TDA Árbol

- Según el número de estructuras tipo árbol que compone a cada nodo del árbol, el árbol recibe un nombre:
 - **Binario:** el número de estructuras de cada nodo es inferior o igual a 2.
 - **N-ario:** el número de estructuras de al menos un nodo es superior a 2.
- Por lo tanto, a diferencia de la lista y sus derivados, el árbol es una estructura no secuencial.



Observación

- En la computación se utiliza mucho una estructura de datos denominada árboles binarios.
- Son árboles que tienen 0, 1 ó 2 descendientes como máximo.

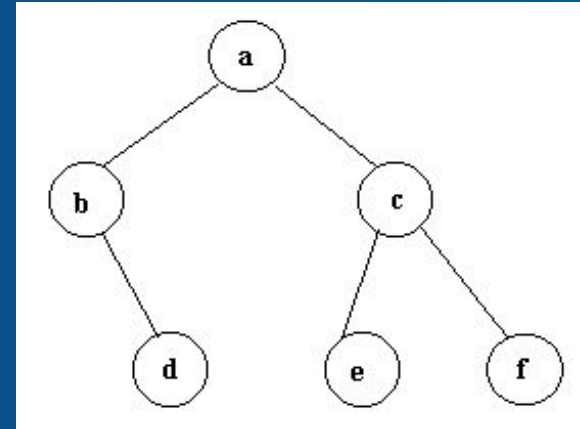


Nomenclatura de los árboles



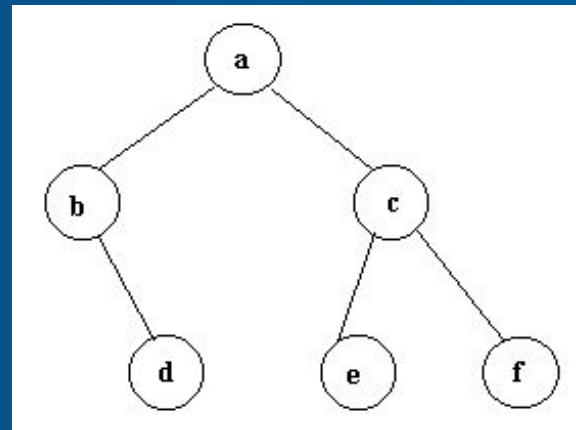
Nomenclatura de los árboles

- **Raíz:** es aquel elemento que no tiene antecesor (en la imagen de ejemplo sería el nodo “a”).
- **Rama:** arista entre dos nodos.
- **Antecesor:** un nodo X es antecesor de un nodo Y si por alguna de las ramas de X se puede llegar a Y.
- **Sucesor:** un nodo X es sucesor de un nodo Y si por alguna de las ramas de Y se puede llegar a X.



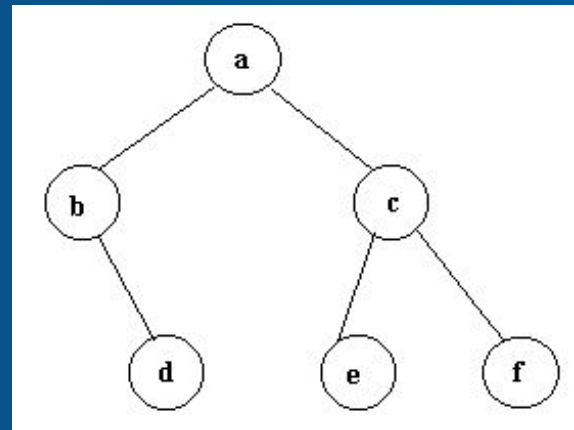
Nomenclatura de los árboles

- **Grado de un nodo:** el número de descendientes directos que tiene. Por ejemplo:
 - **c** tiene grados 2.
 - **d** tiene grado 0.
 - **a** tiene grado 2.
- **Hoja:** nodo que no tiene descendientes: grado 0. Por ejemplo:
 - **d** si es hoja.
 - **a** no es hoja.
 - **f** si es hoja.



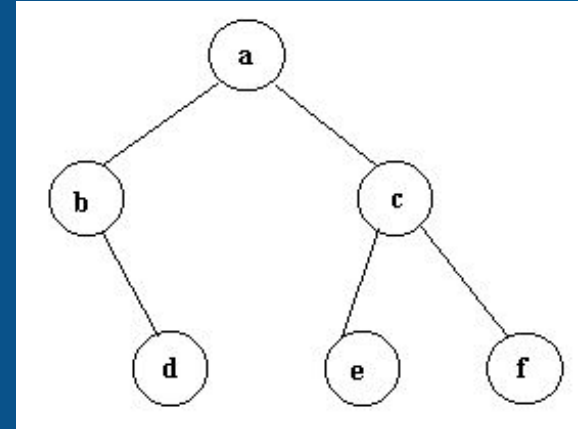
Nomenclatura de los árboles

- **Nodo interno:** aquel que tiene al menos un descendiente.
 - **b** si es un nodo interno.
 - **e** no es un nodo interno.
 - **c** si es un nodo interno.
- **Nivel:** número de nodos que hay que recorrer para llegar desde la raíz a un nodo. Por ejemplo:
 - El nivel del nodo **a** es **1**.
 - El nivel del nodo **c** es **2**.
 - El nivel del nodo **e** es **3**.



Nomenclatura de los árboles

- **Altura:** corresponde con el nivel más alto del árbol. En el ejemplo, la altura del árbol es 3.
- **Anchura:** es el mayor valor del número de nodos que hay entre cada uno de los niveles. En la figura, **la anchura es 3**.





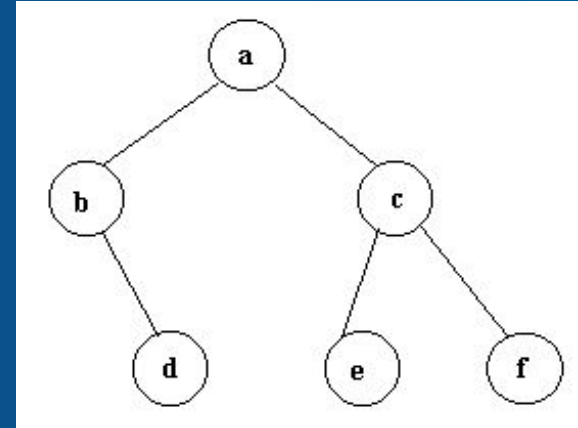
Recorrido de un árbol binario

Recorrido de un árbol binario

- Existen dos tipos de recorridos:
 - Recorrido en *profundidad*.
 - Recorrido en preorden.
 - Recorrido en inorden.
 - Recorrido en postorden.
 - Recorrido en *anchura* o *a nivel*.

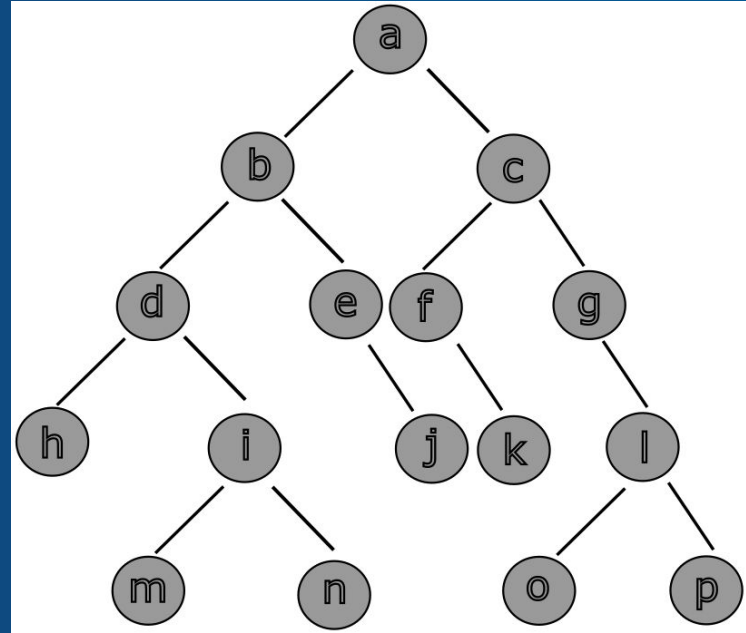
Recorrido en preorden

- Consiste en recorrer el árbol con la siguiente secuencia:
 - Visitar el nodo actual
 - Después visitar el subárbol izquierdo.
 - Después visitar el subárbol derecho.
- Es un proceso recursivo.
- El recorrido del ejemplo en preorden sería:
 - ***a, b, d, c, e y f.***

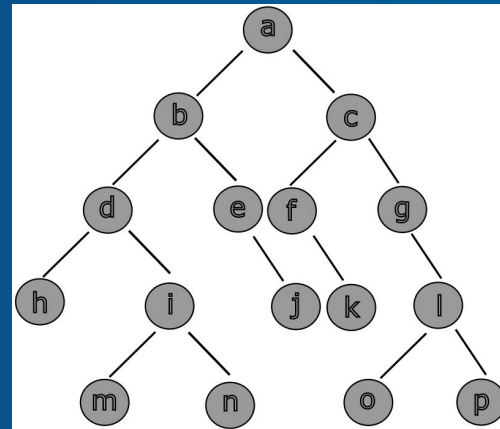


Adaptar la búsqueda en preorden a nuestro ejemplo de C

```
void preorden(arbolBin_t *a){  
    if(a!=NULL){  
        printf("%c\n",a->elem);  
        preorden(a->izq);  
        preorden(a->der);  
    }  
}
```



Adaptarlo a nuestro ejemplo de C



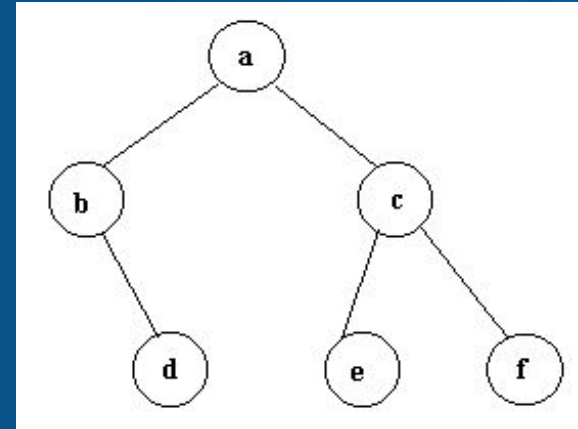
```
void preorden(arbolBin_t *a){  
    if(a!=NULL){  
        printf("%c\n",a->elem);  
        preorden(a->izq);  
        preorden(a->der);  
    }  
}
```



Se inicia el recorrido en preorden:
a b d h i m n e j c f k g l o p

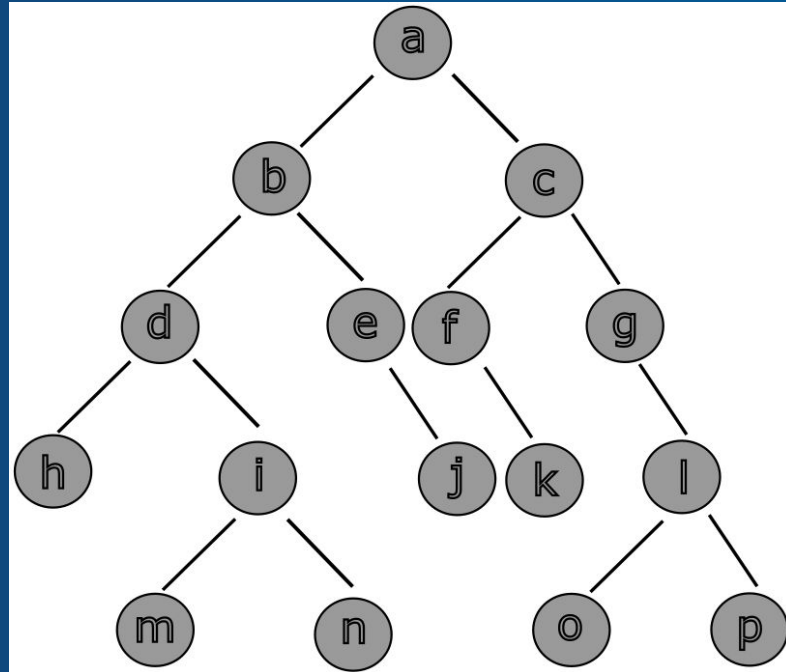
Recorrido en *inorden* u *orden central*

- Se visita en la secuencia:
 - Primero se recorre el subárbol izquierdo.
 - Luego se recorre el nodo actual.
 - Después se visita el subárbol derecho.
- En el ejemplo, el orden de las visitas serían en el orden: ***b, d, a, e, c y f.***



Adaptar la búsqueda en inorden a nuestro ejemplo de C

```
void inorden(arbolBin_t *a){  
    if(a!=NULL){  
        inorden(a->izq);  
        printf("%c ",a->elem);  
        inorden(a->der);  
    }  
}
```

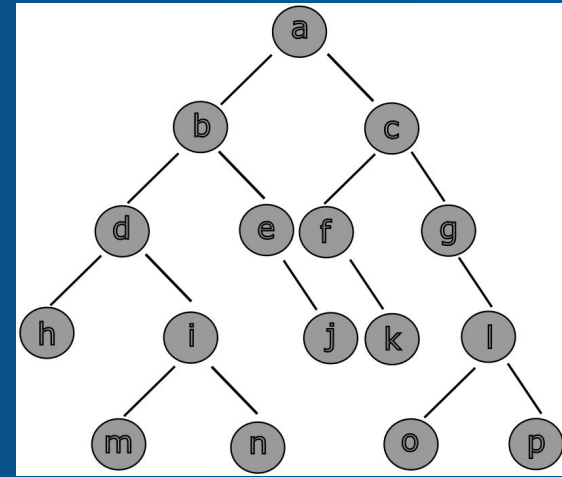


Adaptar la búsqueda en inorden a nuestro ejemplo de C

```
void inorden(arbolBin_t *a){  
    if(a!=NULL){  
        inorden(a->izq);  
        printf("%c ",a->elem);  
        inorden(a->der);  
    }  
}
```

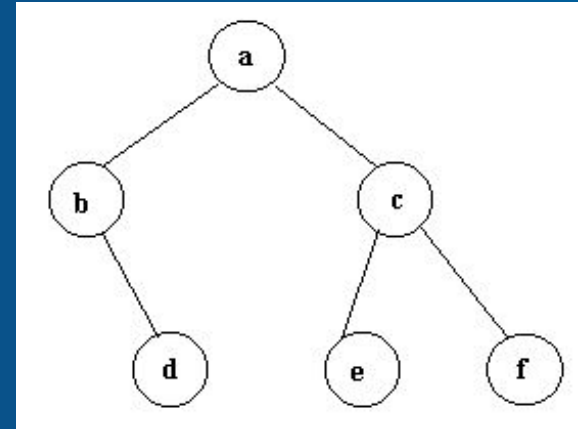


Se inicia el recorrido en inorden
h d m i n b e j a f k c g o l p



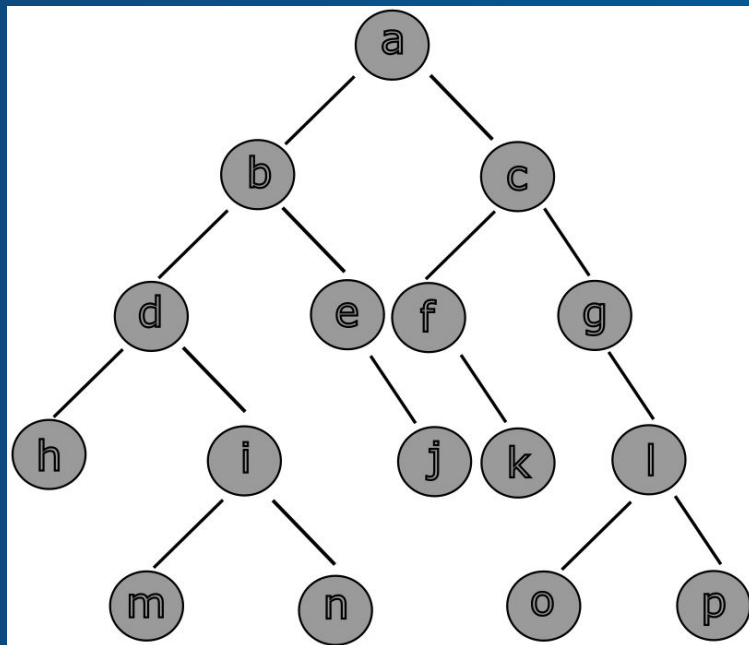
Recorrido en postorden

- Consiste en recorrer el árbol con la siguiente secuencia:
 - Primero se visita el subárbol izquierdo.
 - Luego se visita el subárbol derecho.
 - Por último se visita el nodo actual.
- En el ejemplo, el orden de las visitas serían en el orden: ***d, b, e, f, c*** y ***a***.



Adaptar la búsqueda en postorden a nuestro ejemplo de C

```
void postorden(arbolBin_t *a){  
    if(a!=NULL){  
        postorden(a->izq);  
        postorden(a->der);  
        printf("%c ",a->elem);  
    }  
}
```

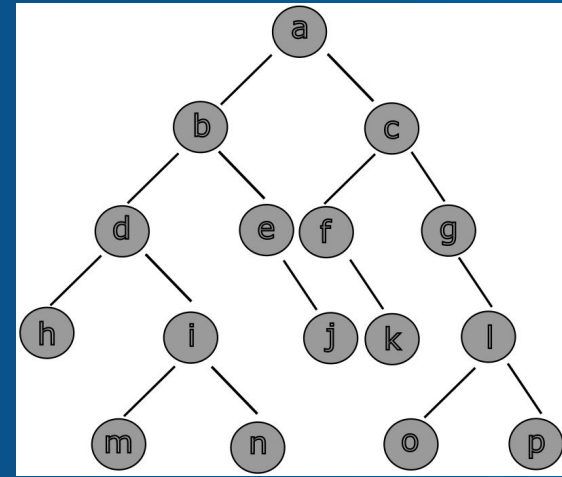


Adaptar la búsqueda en postorden a nuestro ejemplo de C

```
void postorden(arbolBin_t *a){  
    if(a!=NULL){  
        postorden(a->izq);  
        postorden(a->der);  
        printf("%c ",a->elem);  
    }  
}
```

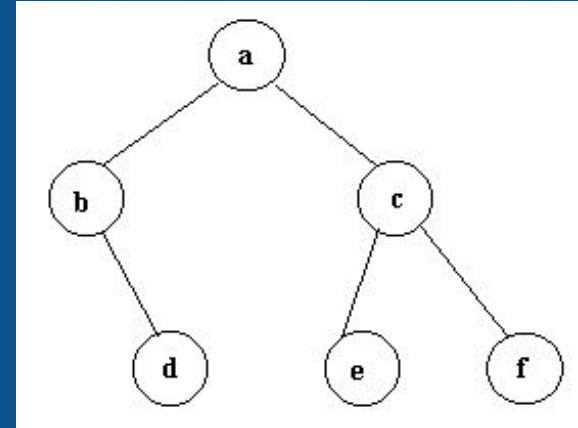


Se inicia el recorrido en postorden
h m n i d j e b k f o p l g c a



Recorrido en amplitud

- Consiste en ir visitando el árbol por niveles:
 - Primero se visitan los nodos de nivel 1 (la raíz).
 - Después los nodos de nivel 2.
 - Así hasta recorrer todo el árbol.
- Para el ejemplo, el recorrido del árbol por amplitud sería: a, b, c, d, e, f.



Recorrido en amplitud

- El recorrido en amplitud se realiza de manera iterativa, a partir de una *cola auxiliar*.

Se inicia el recorrido en amplitud
a b c d e f g h i j k l m n o p

```
void amplitud(arbolBin_t *a){
    cola_t *cola;
    arbolBin_t *aux;
    if (a != NULL){
        crear(&cola);
        encolar(&cola, a);
        while(!esVacia(cola)) {
            desencolar(&cola, &aux);
            printf("%c ", aux->elem);
            if(aux->izq != NULL){
                encolar(&cola, aux->izq);
            }
            if(aux->der != NULL){
                encolar(&cola, aux->der);
            }
        }
    }
}
```

Árbol Binario de Búsqueda

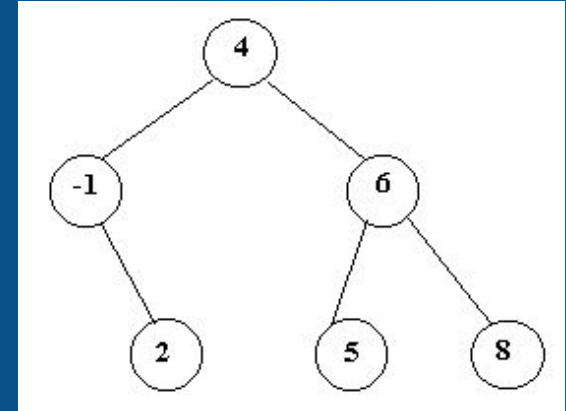


Árbol Binario de Búsqueda

- Un árbol binario de búsqueda es aquel que es compuesto por:
 - Una estructura vacía o...
 - Un elemento o clave de información más un número finito, **máximo dos**, de estructuras de tipo árbol, **disjuntos**, llamados subárboles y además cumplen con:
 - Todas las claves del subárbol izquierdo al nodo son menos que la clave del nodo.
 - Todas las claves del subárbol derecho al nodo son mayores a la clave del nodo.
 - Ambos subárboles son árboles binarios de búsqueda.

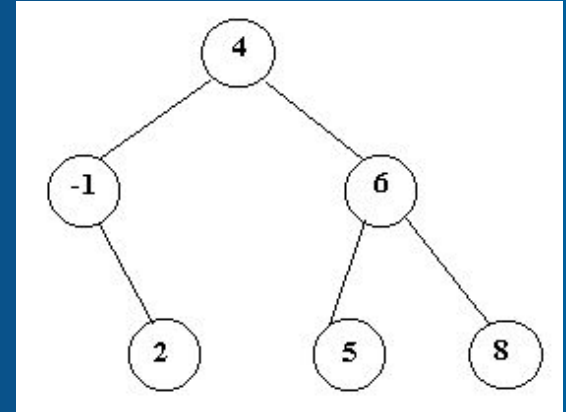
Ejemplo de árbol binario de búsqueda

- El ejemplo consiste en un árbol binario de búsqueda de números enteros.
- Dada la raíz “4”:
 - Las claves del subárbol izquierdo son menores que 4.
 - Las claves del subárbol derecho son mayores que 4.
- Esto se cumple también para ***todos los subárboles.***



Ventaja

- ***Son en general mucho más rápidos*** para localizar un elemento que una lista enlazada.

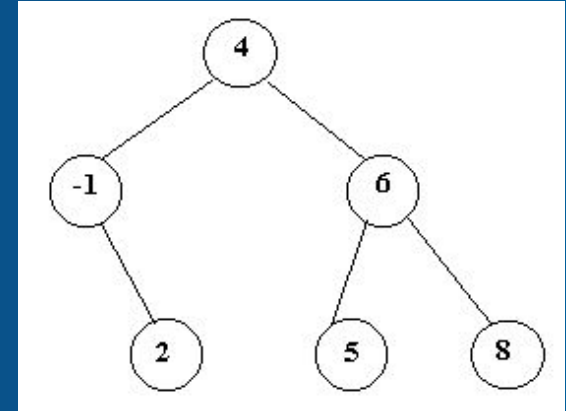


Árbol perfectamente equilibrado

- Se realiza analizando cada uno de los nodos del árbol.
- Significa que la diferencia entre el número de nodos del subárbol izquierdo y derecho es como máximo -1 o 1.
- Si se cumple lo anterior, el número de comparaciones necesarias para localizar una clave es aproximadamente de $\log_2(n)$ en el peor de los casos.

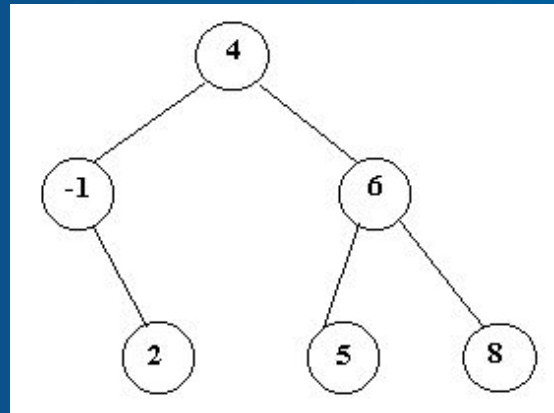
Inserción de elementos a un árbol binario de búsqueda

- El algoritmo de inserción para este tipo de árboles tiene la ventaja, sobre los arreglos ordenados, donde se emplearía búsqueda binaria, de que no necesita hacer una reubicación de los elementos de la estructura para que este siga ordenada.



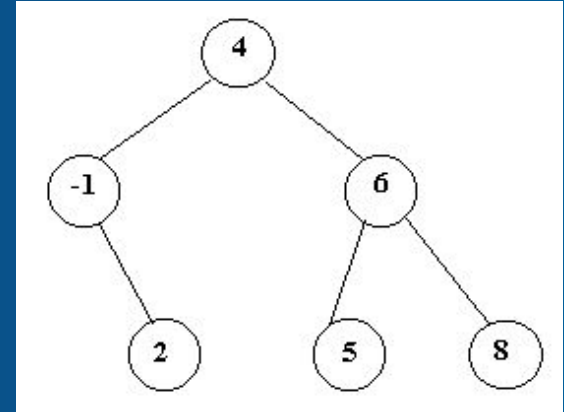
Inserción de elementos a un árbol binario de búsqueda

- Por ejemplo:
 - Insertar la clave 7 en el árbol se requiere avanzar por el árbol hasta llegar a la clave 8, e introducir la nueva clave en el subárbol izquierdo a 8.



Eliminado de elementos a un árbol binario de búsqueda

- Es un poco más complejo que el proceso de inserción, sin embargo, es más eficiente que el borrado de un arreglo ordenado.



Borrado

- El proceso de borrado de un árbol binario de búsqueda es complicado.
- Hay que comprobar que el árbol siga manteniendo sus características luego del borrado.
- Existen tres casos:
 1. El nodo no tiene descendientes.
 2. El nodo tiene descendientes por una rama.
 3. El nodo tiene descendientes en ambas ramas.

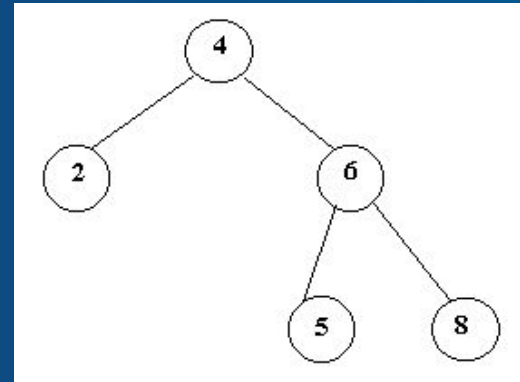
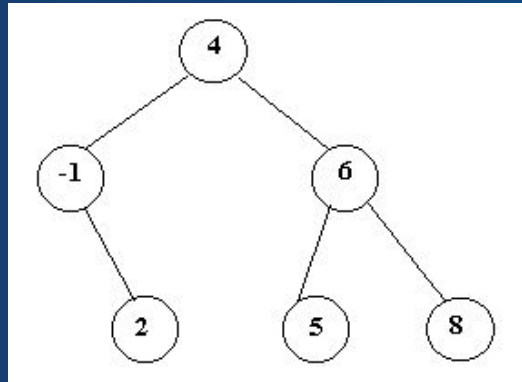
El nodo no tiene descendientes

- Simplemente se borra.

El nodo tiene descendiente por una rama

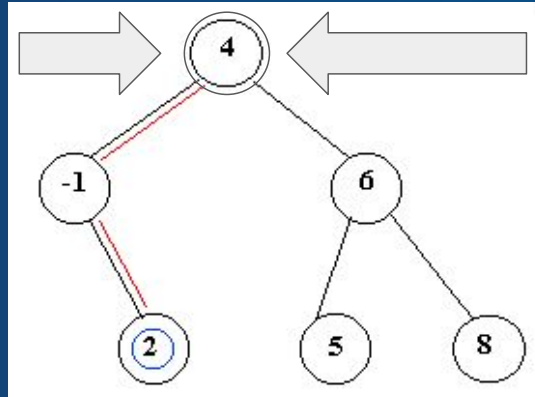
Si el nodo tiene al menos un descendiente por ***una sola rama***, se borra dicho nodo y su primer descendiente se asigna como hijo del padre del nodo borrado.

En el ejemplo se puede ver qué ocurre si se borra el nodo -1.



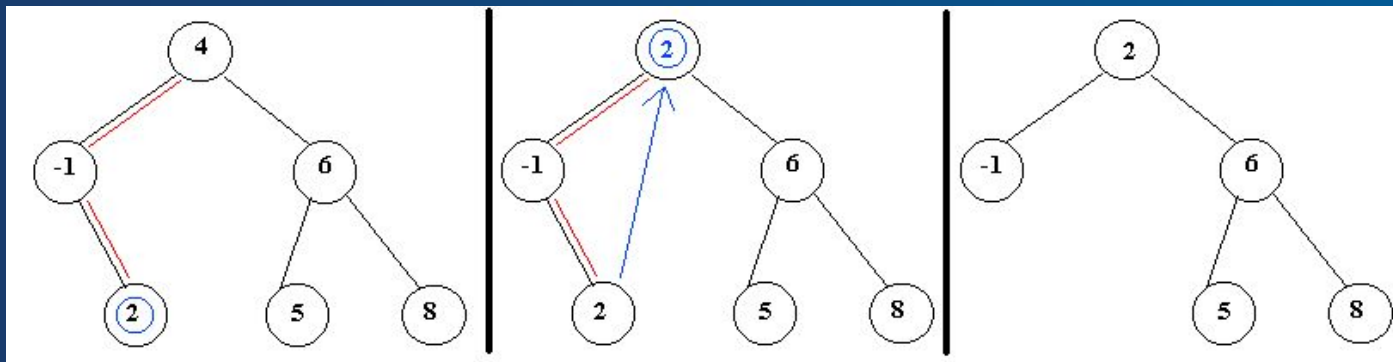
El nodo tiene al menos un descendiente por cada rama

- Al borrar un nodo, es necesario mantener la coherencia de los enlaces, además de mantener la estructura de árbol binario de búsqueda.
- La solución consiste en sustituir la información del nodo que se borra por el de una de las hojas, y borrar a continuación dicha hoja.



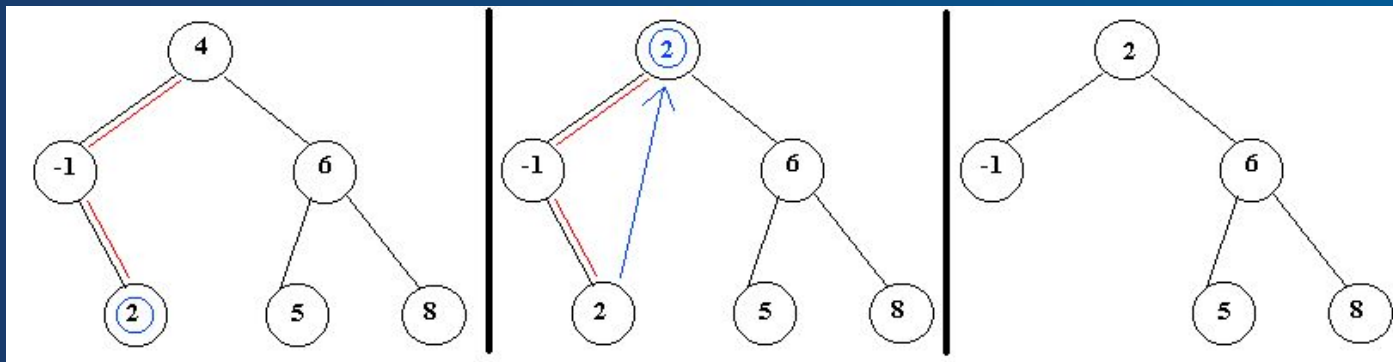
El nodo tiene al menos un descendiente por cada rama

- La hoja elegida debe poseer alguna de las siguientes claves:
 - La **mayor** de las claves **menores** al nodo que se borra. Si se quiere borrar el nodo 4, se sustituirá por el nodo de la clave 2.
 - La **menor** de las claves **mayores** al nodo que se borra. Si se quiere borrar el nodo 4, se sustituirá por el nodo de la clave 5.



El nodo tiene al menos un descendiente por cada rama

- El ejemplo anterior realiza la sustitución por la mayor de las claves menores.
- Se desciende primer a la izquierda del nodo que se va a borrar, y después se avanza siempre a la derecha hasta encontrar un nodo hoja.



Ejemplo de creación de un Árbol Binario de Búsqueda



Ejemplo de creación de un Árbol Binario de Búsqueda

- Suponer que introduciremos las claves de forma ascendente:
 - Ejemplo: 1, 2, 3, 4, 5 y 6.
- Los pasos serían:
 1. Se crea un árbol cuya raíz tiene la clave 1.
 2. Se inserta la clave 2 en el subárbol derecho de 1.
 3. Se inserta la clave 3 en el subárbol derecho de 2.
 4. Continuando las inserciones, se ve que el árbol degenera en una lista secuencial.

Ejercicio en clases 1

- Almacenar en un árbol de búsqueda binaria los elementos del conjunto N, que son ingresados como sigue:
 - $N = \{10, 9, 1, 2, 4, 5, 3, 8, 7, 6\}$

Ejercicio en clases 2

- Almacenar en un árbol de búsqueda binaria los elementos del conjunto N, que son ingresados como sigue:
 - {50, 30, 70, 20, 40, 60, 80, 10, 35, 45, 65, 75, 85}

Ejercicio en clases 3

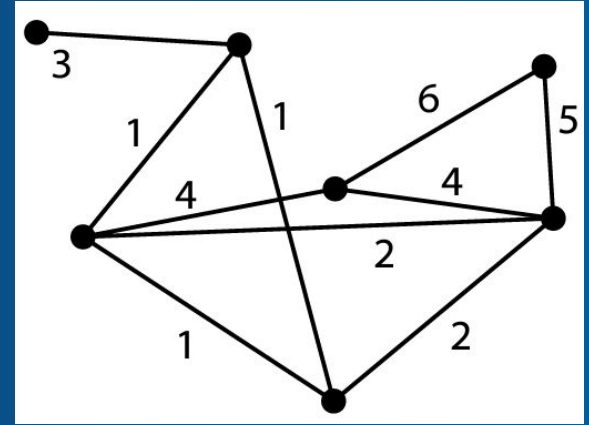
- Dado el siguiente conjunto de claves, construye un árbol binario de búsqueda insertando los elementos en el orden dado:
 - {50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45, 55, 65, 75, 90}
- Construye el árbol binario de búsqueda a partir de los datos dados.
- Realiza los siguientes recorridos:
 - Preorden.
 - Inorden.
 - Postorden.
 - Por amplitud.

Árbol generador mínimo o de expansión mínima

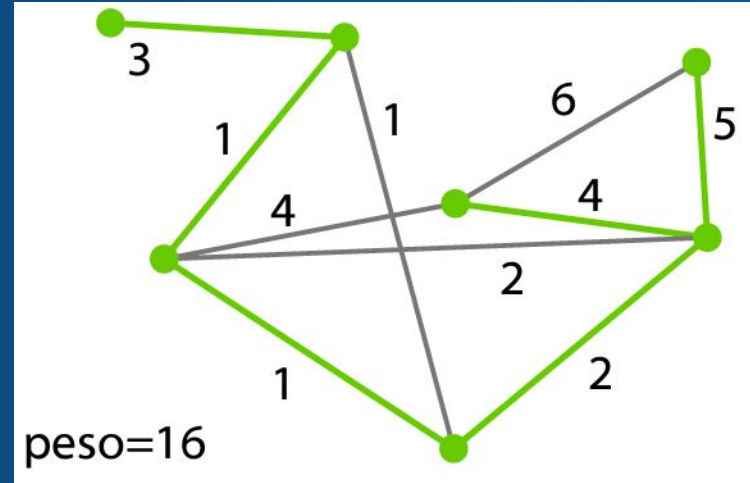
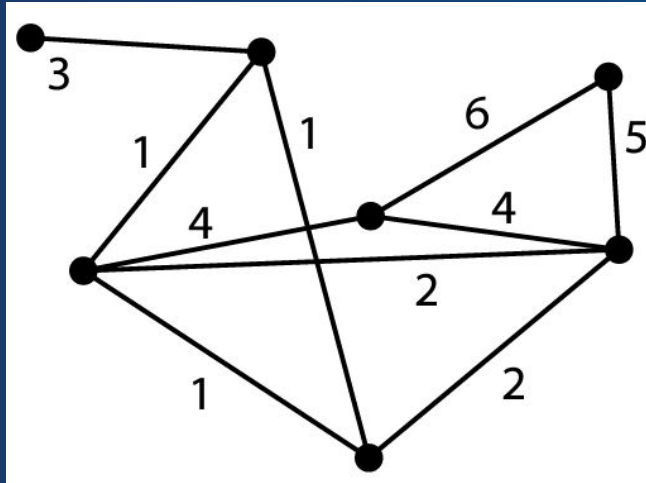


Árbol de expansión mínima

- Un árbol generador de un grafo es:
 - Un subgrafo conexo del grafo.
 - Contiene a todos los vértices.
 - Es un árbol.
- Un árbol de expansión mínima en un grafo pesado, es un árbol generador de peso mínimo.



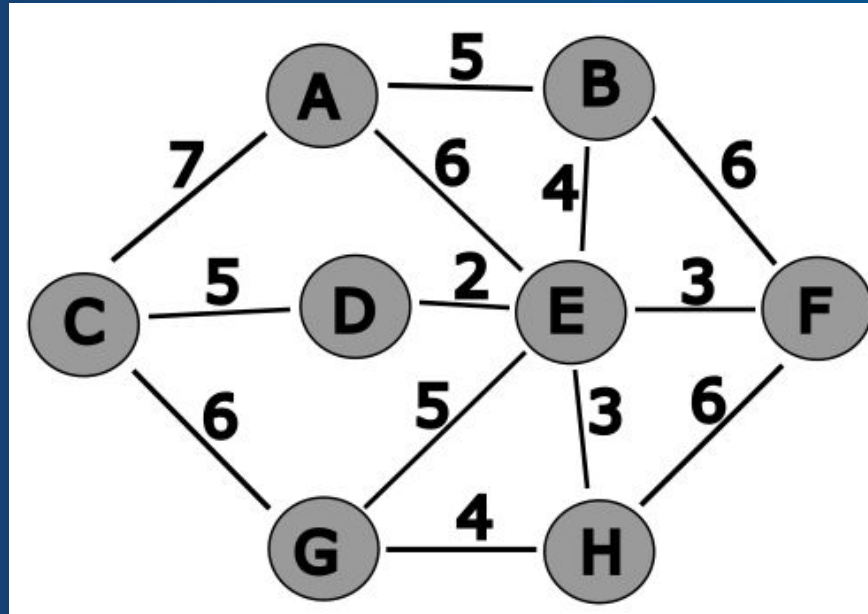
Árbol de expansión mínima



Ejercicio aplicado 1

- Philip es dueño de una invernadero lleno de paltos. Tiene pensado instalar un sistema de regadío que le evite perder 3 horas todas las mañanas para regar sus amados arbolitos.
- Las unidades de medida están en metros y el metro de cañería vale \$4.000. Puede omitir el precio de los codos, tee, teflón, etc.
- ¿Cuál sería la forma menos costosa de suministrar agua a cada árbol?

¿Cuál sería la forma menos costosa de suministrar agua a cada árbol?

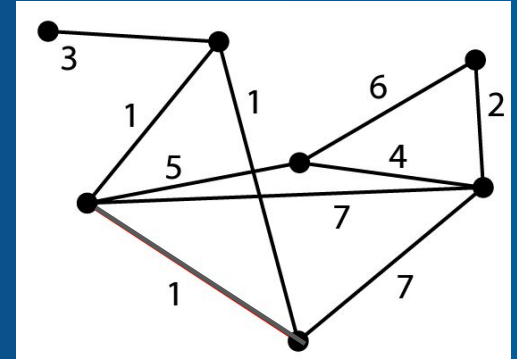


Algoritmo de Kruskal

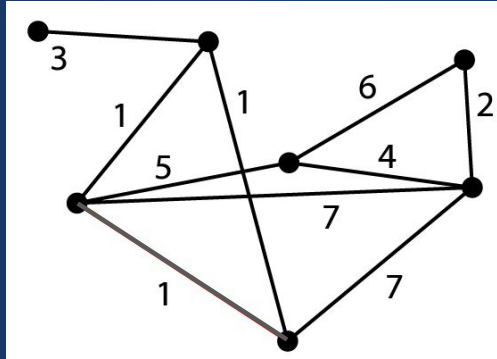


Algoritmo de Kruskal para AGM

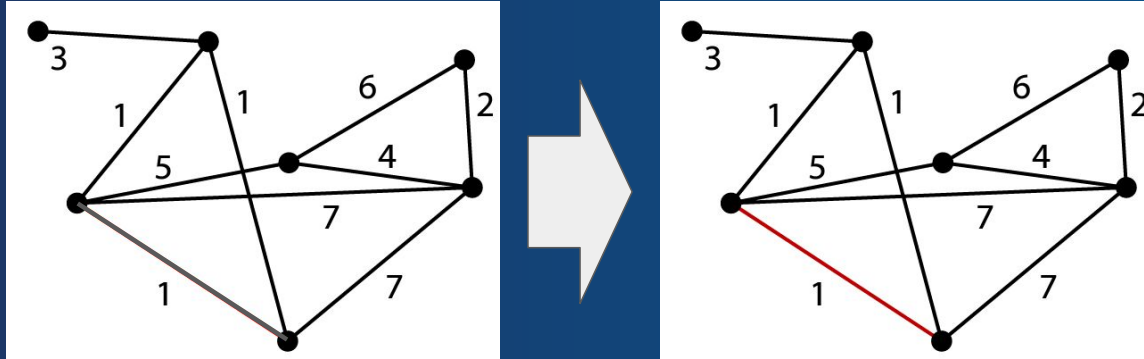
- Comienza con un subgrafo que comparta los vértices con el grafo original, pero sin la inclusión de ninguna arista.
- Agregar iterativamente una arista de peso mínimo arbitraria que no forme ciclos con las demás aristas del conjunto.
- El algoritmo termina cuando se han agregado $n-1$ aristas.



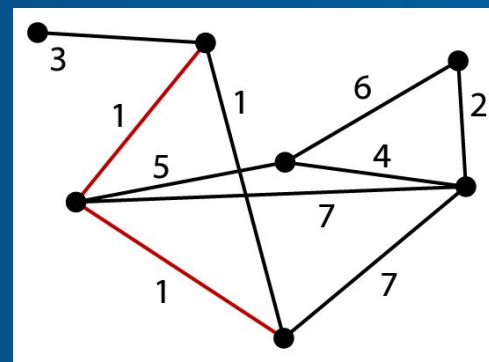
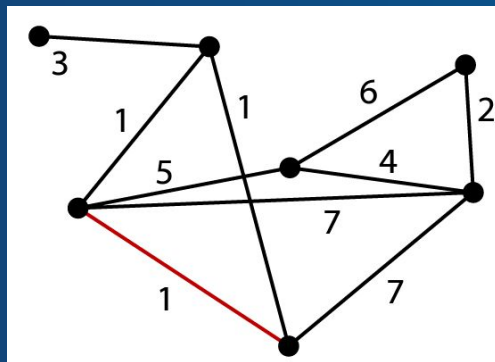
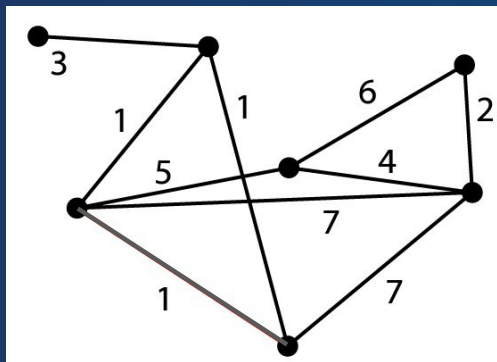
Algoritmo de Kruskal ejemplo



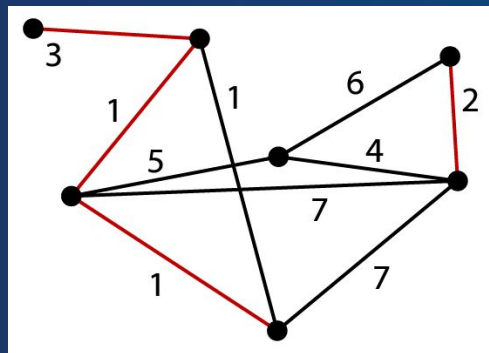
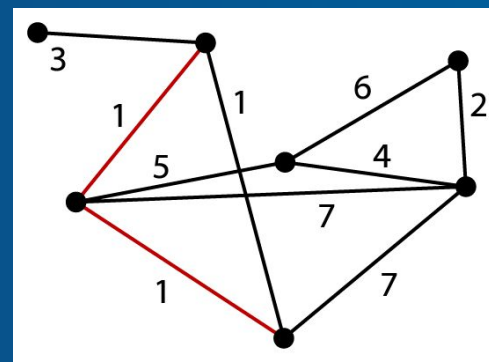
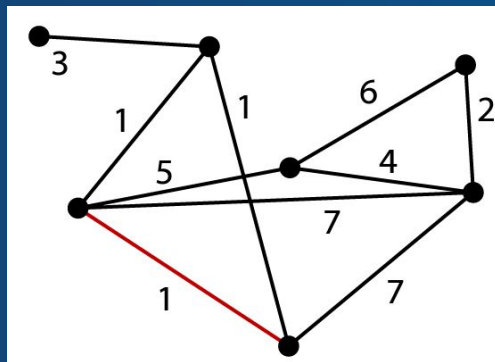
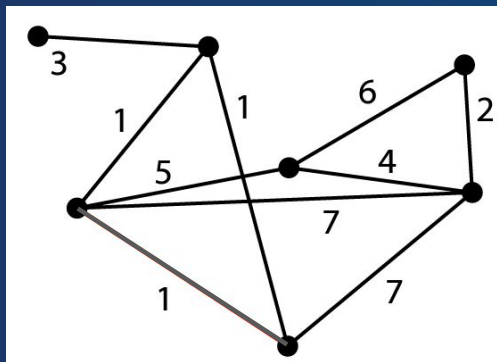
Algoritmo de Kruskal ejemplo



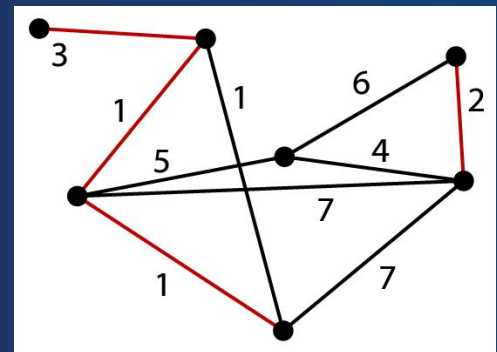
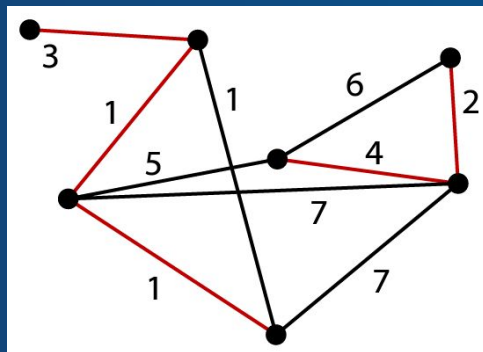
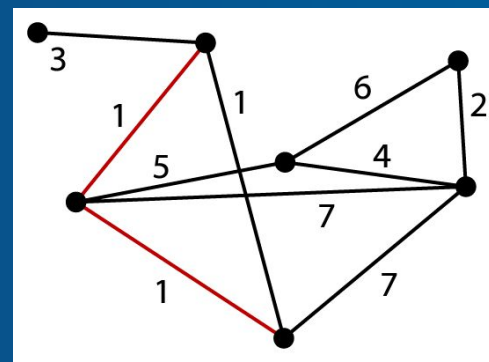
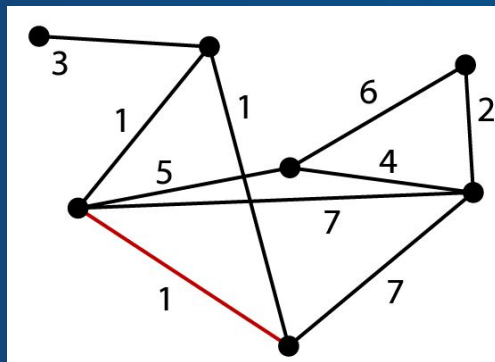
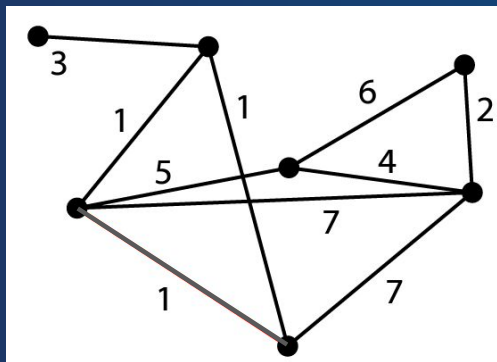
Algoritmo de Kruskal ejemplo



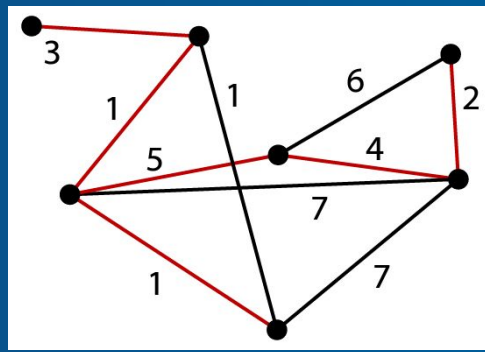
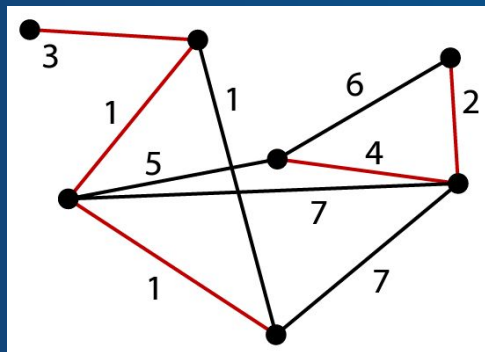
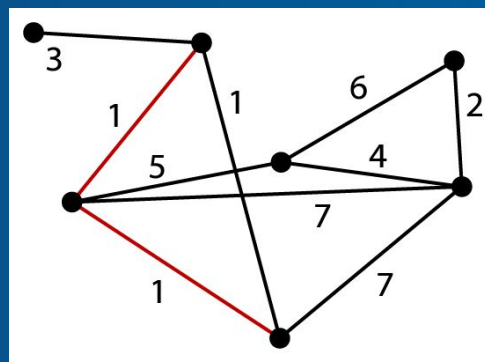
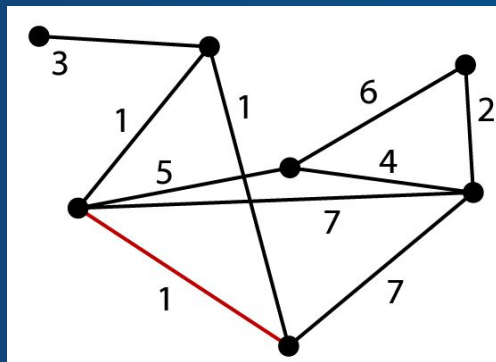
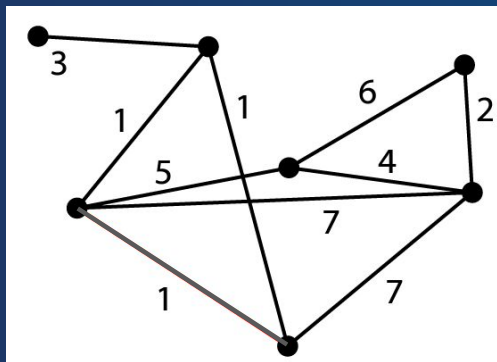
Algoritmo de Kruskal ejemplo



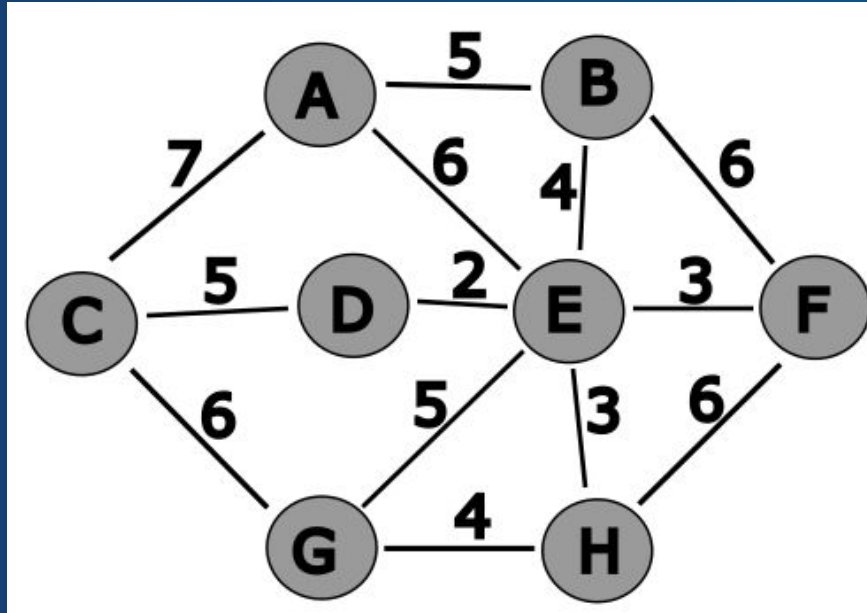
Algoritmo de Kruskal ejemplo



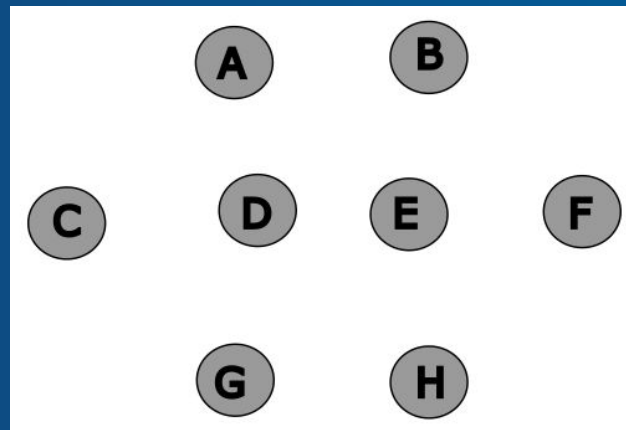
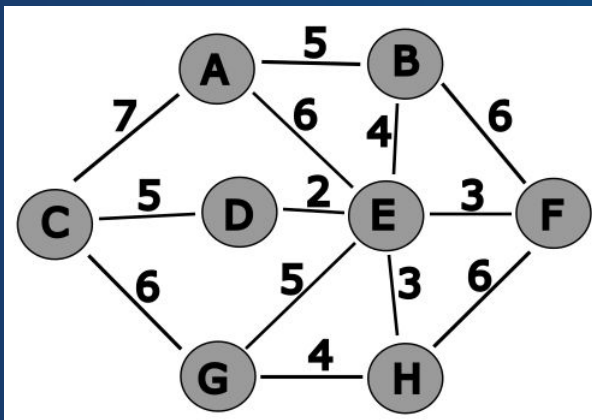
Algoritmo de Kruskal ejemplo



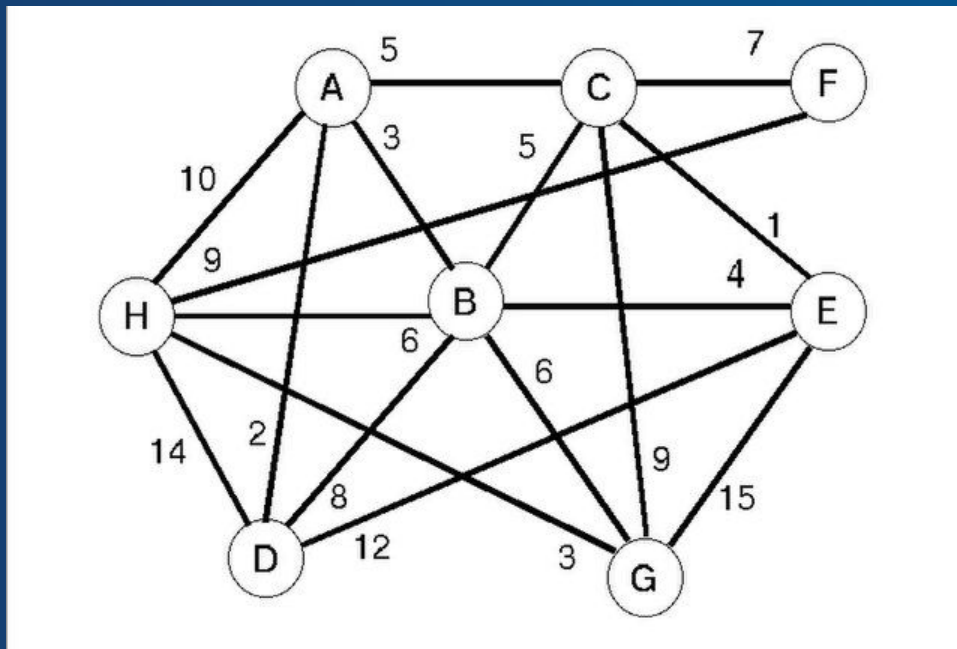
Aplicar Kruskal para el árbol planteado anteriormente



Algoritmo de Kruskal para AGM



Ejercicio 1: Aplicar Kruskal para generar un árbol de expansión mínima



Algoritmo de Prim

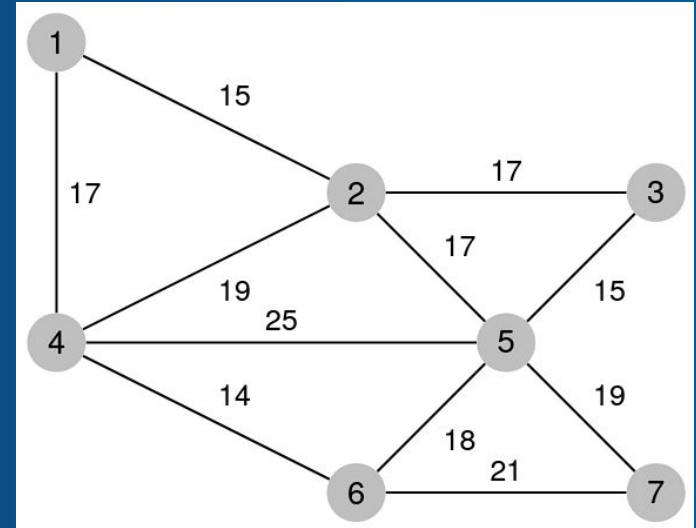


Algoritmo de Prim

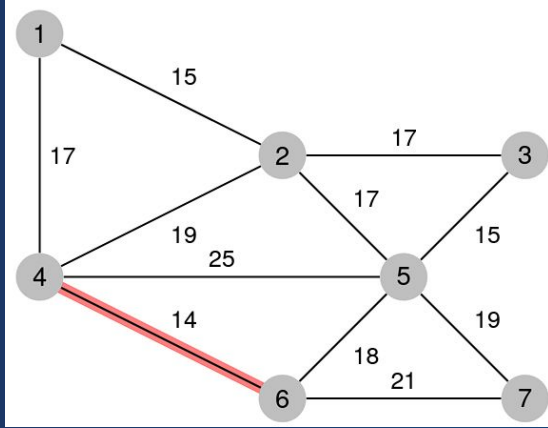
- Algoritmo que permite la determinación de un árbol de expansión mínima.
- Aplicaciones: distribuciones de tendidos eléctricos, telefónicos, entre muchos otros.
- A diferencia de Kruskal, Prim comienza desde un vértice origen e iterativamente se va extendiendo el árbol generado.

Algoritmo de Prim

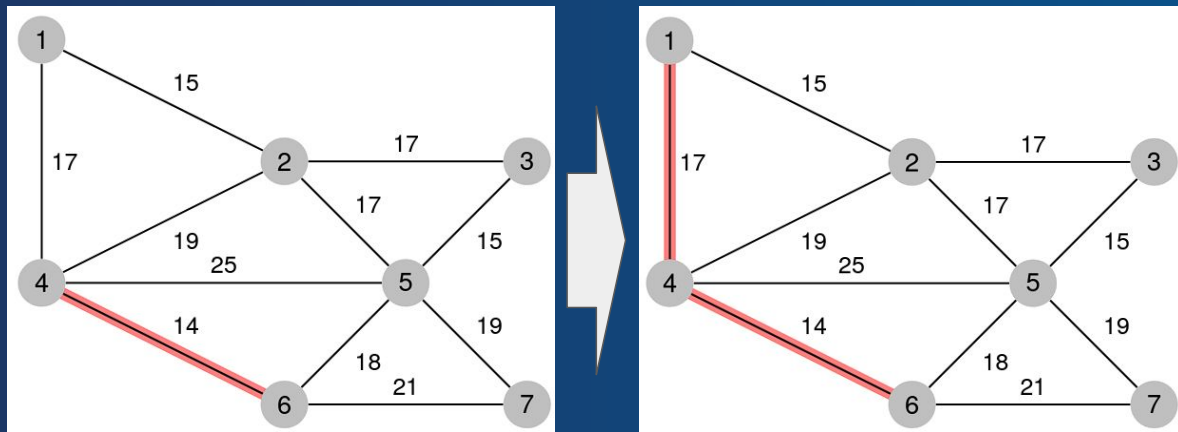
- Inicialmente, es seleccionado de manera arbitraria un vértice para comenzar.
- En cada paso se une el árbol ya existente con alguno de los vértices aún no alcanzados, seleccionando la arista de menor costo capaz de incorporar un nuevo vértice.



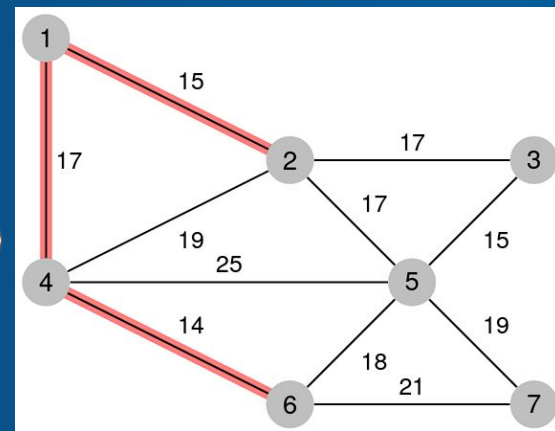
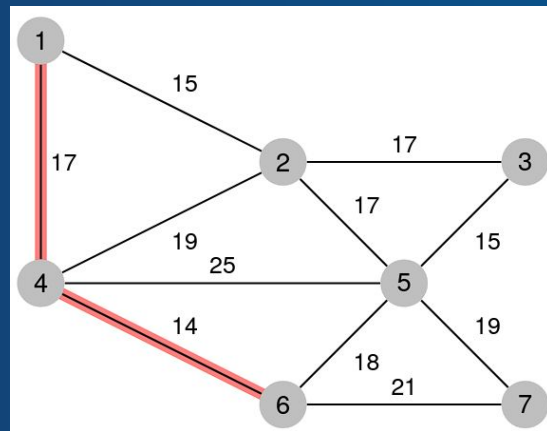
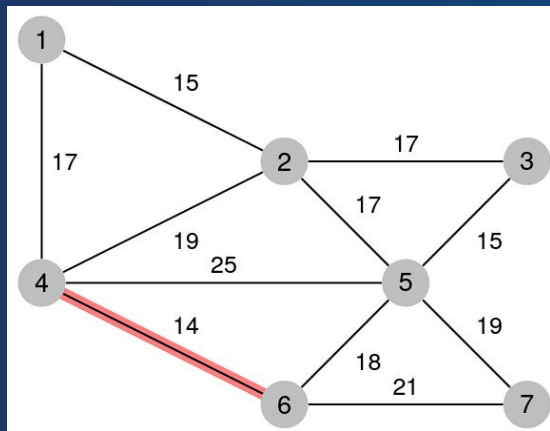
Ejemplo Aplicación de Prim



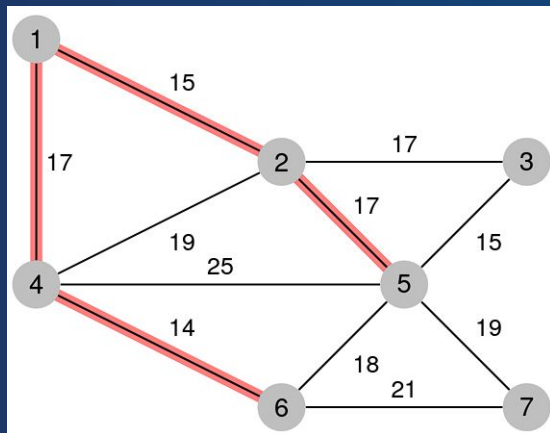
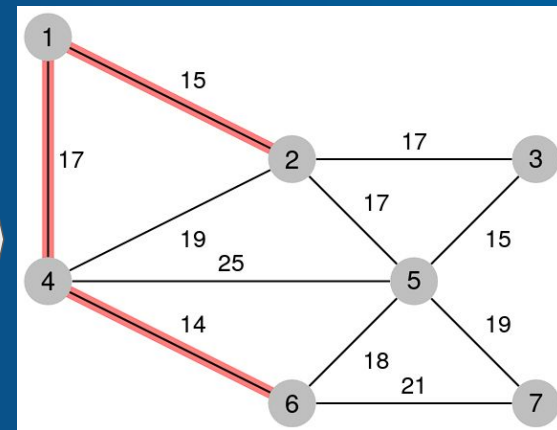
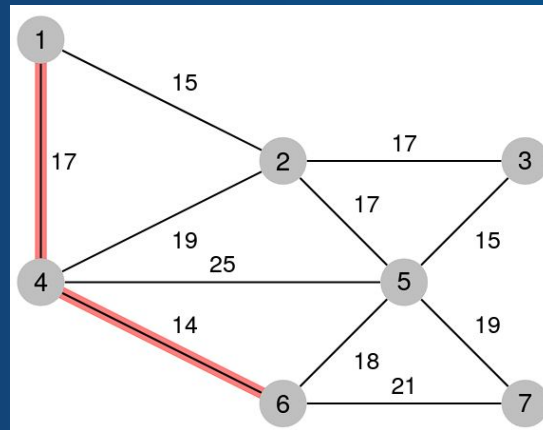
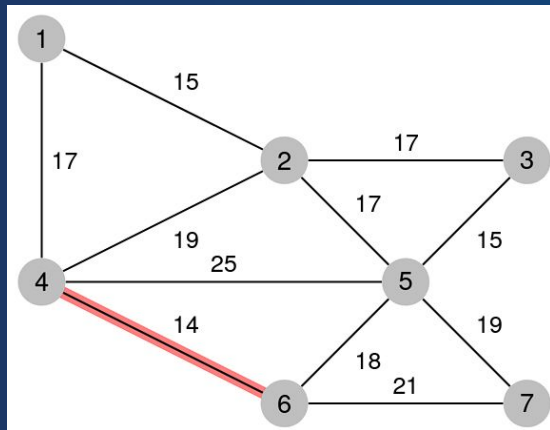
Ejemplo Aplicación de Prim



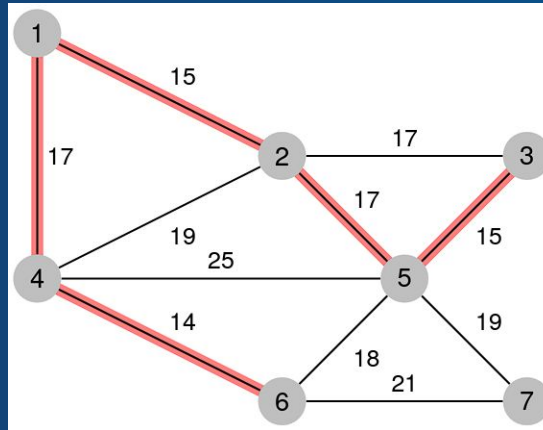
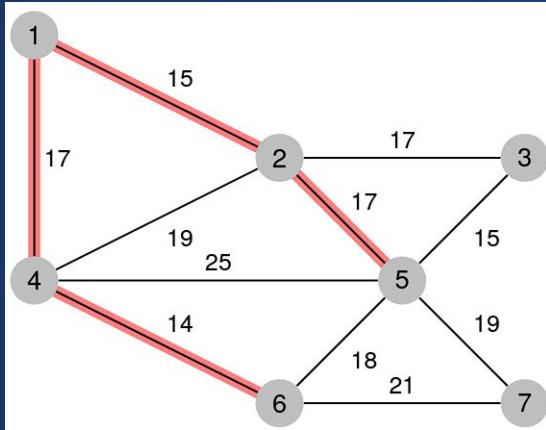
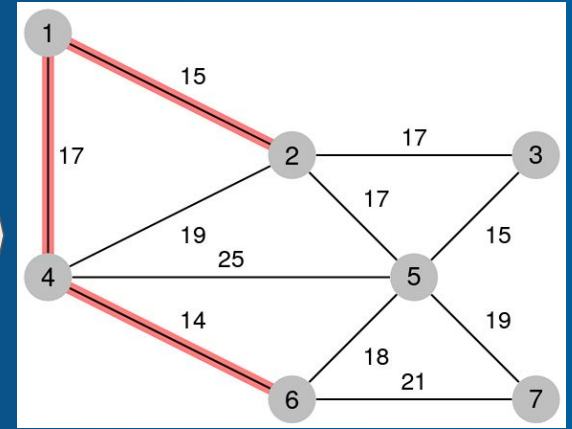
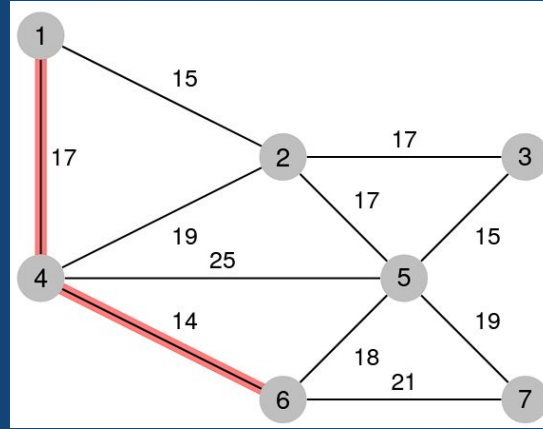
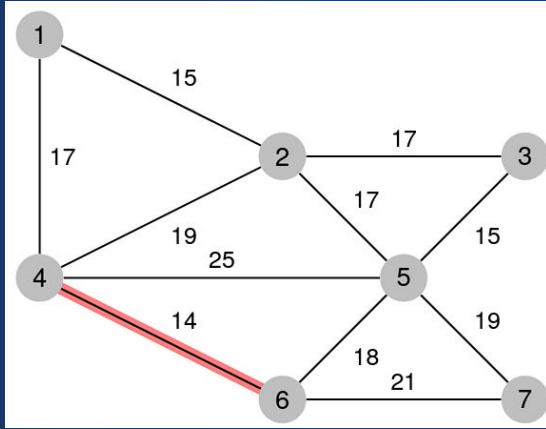
Ejemplo Aplicación de Prim



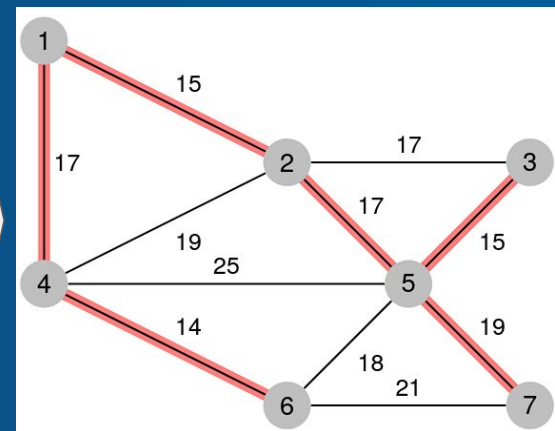
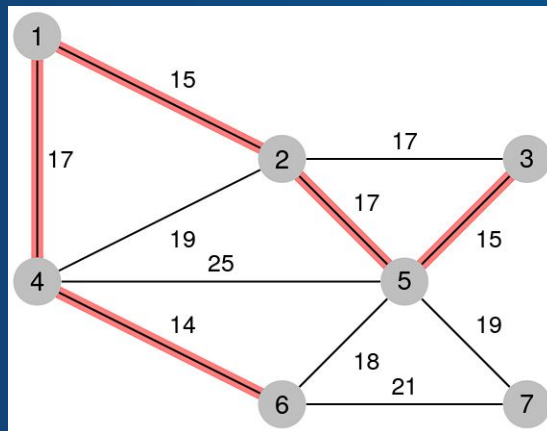
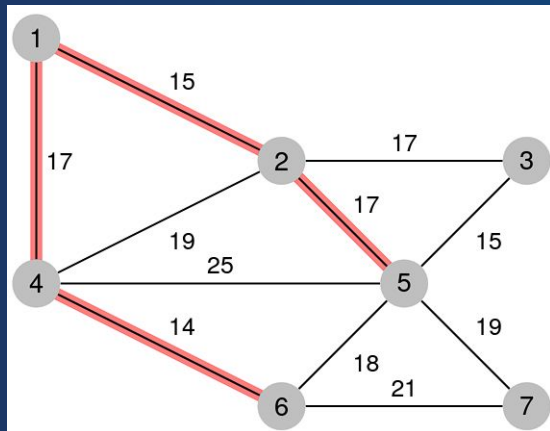
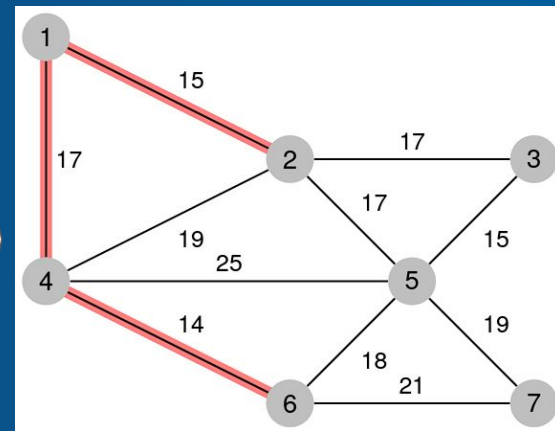
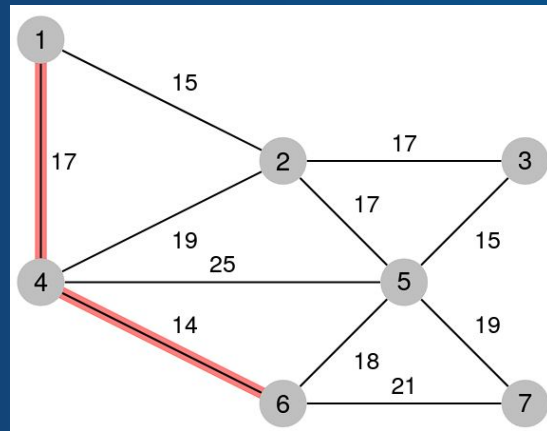
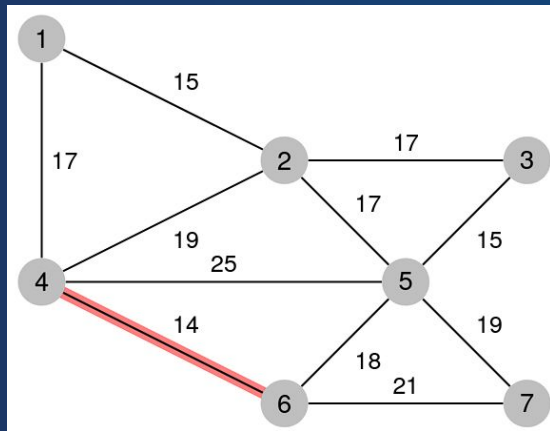
Ejemplo Aplicación de Prim



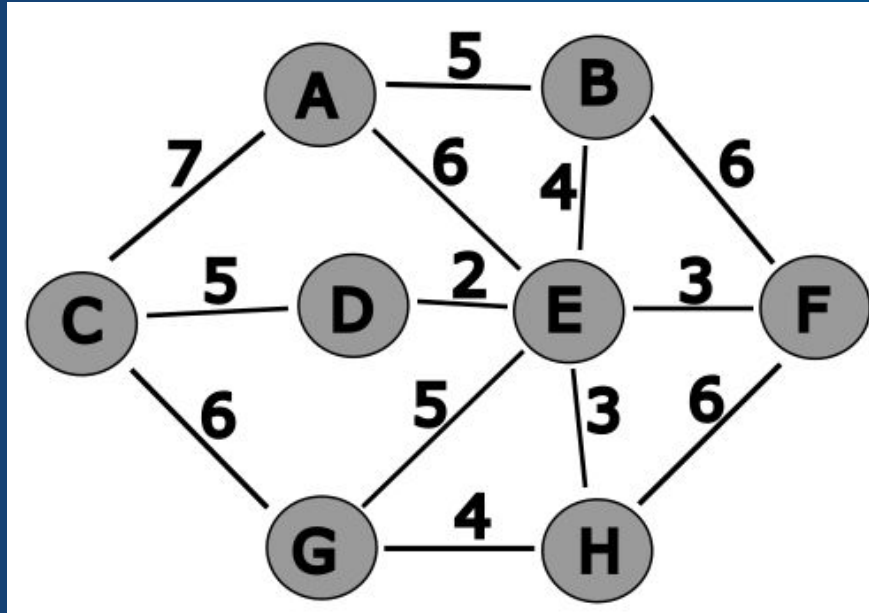
Ejemplo Aplicación de Prim



Ejemplo Aplicación de Prim



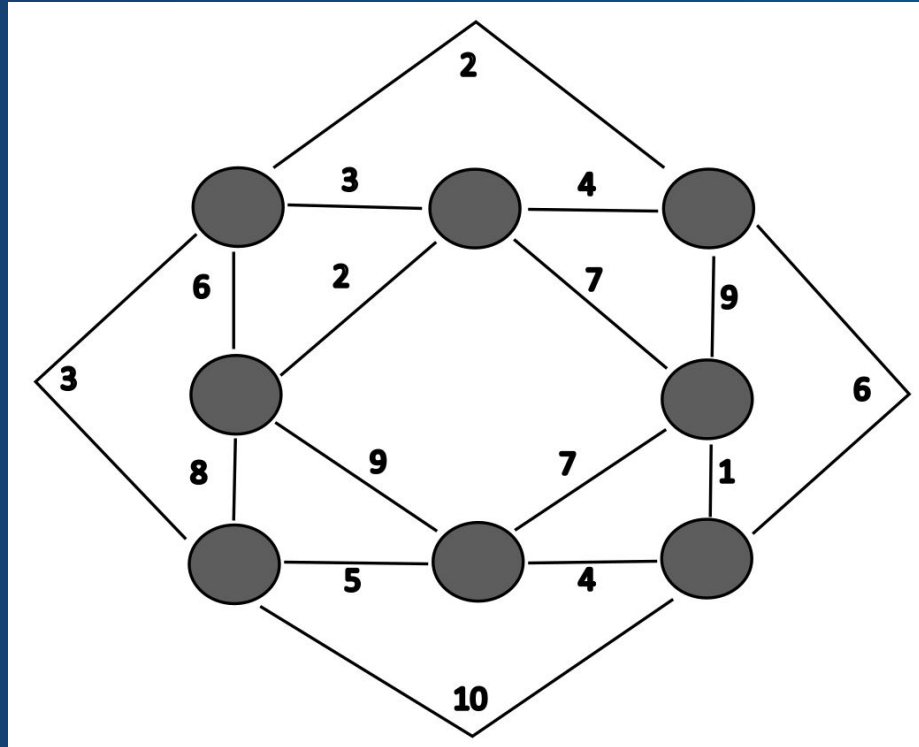
Aplicar Prim para el siguiente grafo



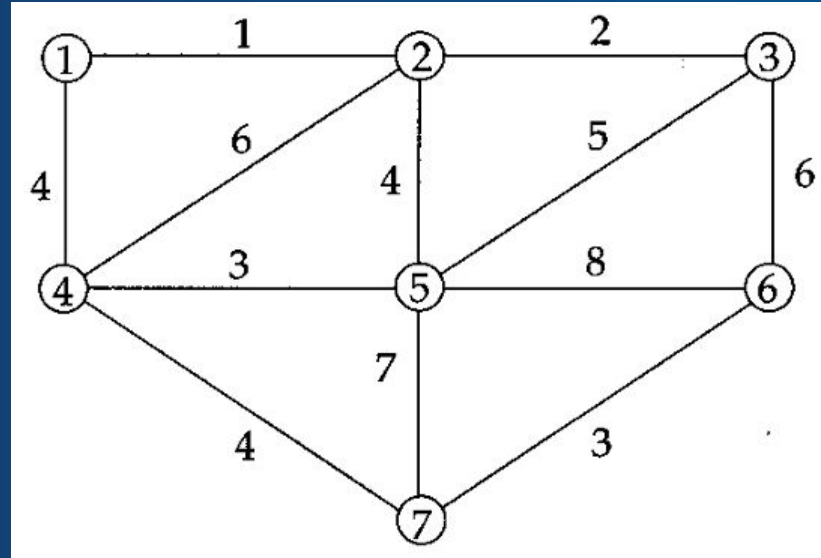
Ejercicios

EFESOS

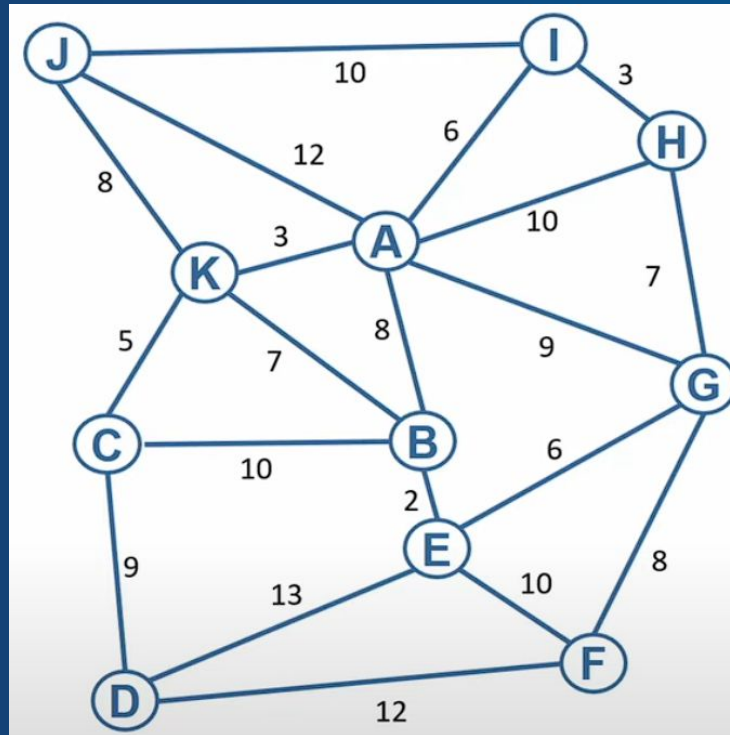
Ejercicio: Hallar Árbol De Expansión mínima con Kruskal y Prim



Hallar árbol de mínima expansión a partir de Kruskal y Prim



Hallar árbol de mínima expansión a partir de Kruskal y Prim



Métodos vistos en EdD

EdD

Algoritmo de Dijkstra



Algoritmo de Dijkstra

- Fue creado por el científico Edsger W. Dijkstra en 1956.
- Lo inventó en solo una tarde, mientras pensaba cómo modelar rutas en un mapa para una computadora.
- Dijkstra mismo dijo que “no le costó mucho hacerlo...”, pero terminó siendo uno de los algoritmos más usados en la informática moderna.

¿Qué es el algoritmo de Dijkstra?

- Es un método para encontrar el camino más corto entre dos puntos en un mapa (o en una red de conexiones).
- Imagina que estás en una ciudad y quieres llegar al hospital usando la ruta más rápida posible. El algoritmo de Dijkstra te ayuda a encontrar esa ruta usando lógica matemática.

¿Cómo funciona?

- Poco a poco va construyendo la ruta más eficiente, sin necesidad de probar todas:
 1. Empieza en el punto de origen (por ejemplo, tu casa).
 2. Mira todos los caminos que puedes tomar desde ahí.
 3. Guarda el costo (o la distancia) para llegar a los lugares cercanos.
 4. Luego, sigue avanzando a los lugares con el menor costo acumulado, actualizando siempre cuál es el camino más corto hasta el momento.
 5. Repite este proceso hasta llegar al destino.

Aplicaciones en la vida real

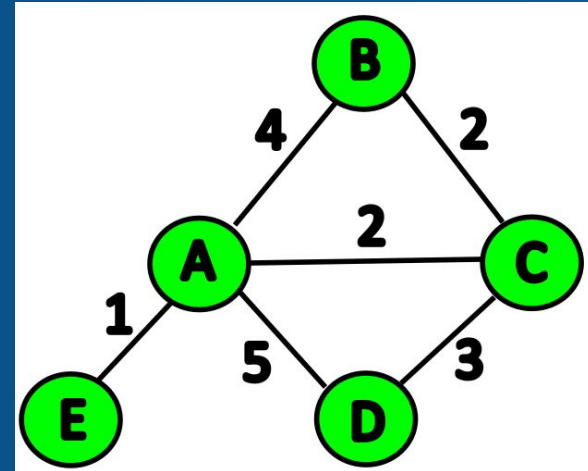
- **Google Maps y GPS:** para buscar la ruta más rápida.
- **Redes de computadoras:** para enviar información por el camino más rápido entre servidores.
- **Videojuegos:** para que los personajes o enemigos se muevan inteligentemente en un mapa.
- **Logística y transporte:** para optimizar rutas de entrega.

¿Dónde podemos utilizar Dijkstra?

- Dijkstra se usa en grafos ponderados (con pesos), donde se quiere encontrar el camino más corto desde un nodo origen a todos los demás.
- Puede funcionar en:
 - Grafos dirigidos
 - Grafos no dirigidos
 - Grafos con pesos positivos

Ejemplo

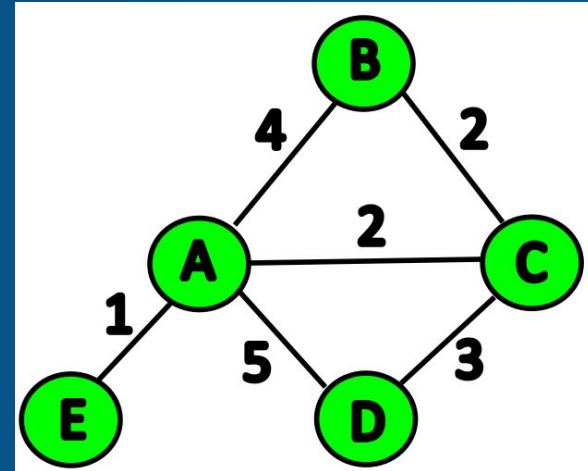
- Pesos de los caminos:
 - $A-B = 4$
 - $A-C = 2$
 - $A-D = 5$
 - $A-E = 1$
 - $B-C = 2$
 - $C-D = 3$
- ¿Cual es el camino más corto desde A hacia todos los demás nodos?



Dijkstra paso a paso

1. Inicializamos distancias desde A:

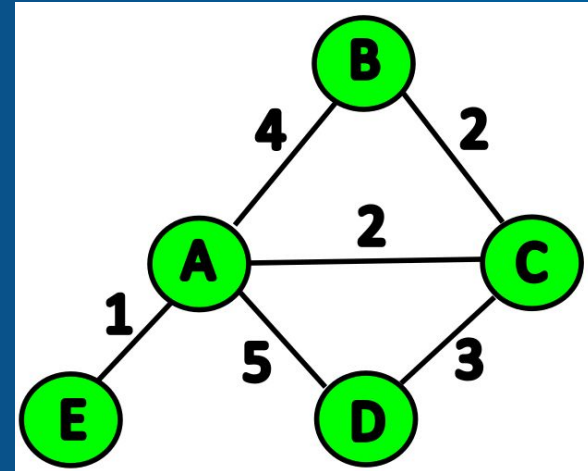
- A: 0 (inicio)
- B: ∞
- C: ∞
- D: ∞
- E: ∞



Dijkstra paso a paso

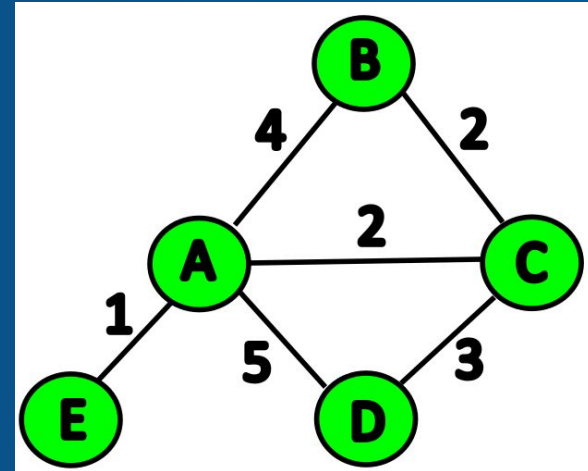
2. Desde A, actualizamos vecinos:

- B: 4
- C: 2
- D: 5
- E: 1



Dijkstra paso a paso

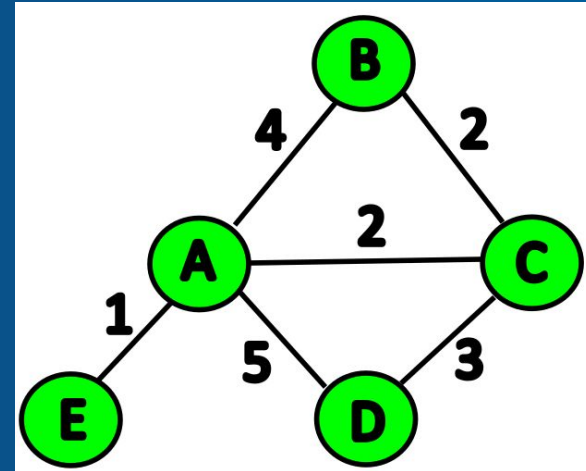
3. Vamos al nodo más cercano no visitado $\rightarrow$ E (1):
 - Desde E no hay conexiones nuevas.



Dijkstra paso a paso

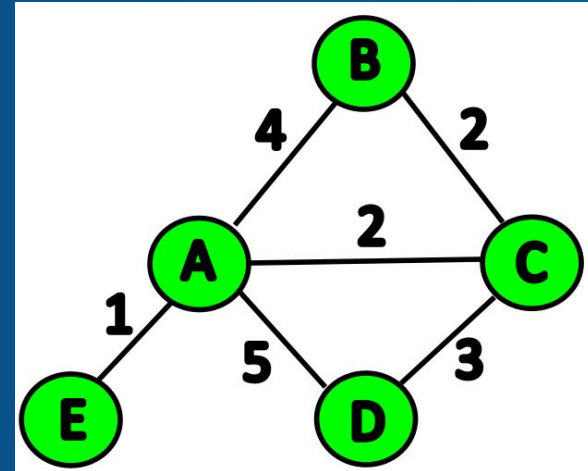
4. Siguiente más cercano $\rightarrow C$ (2):

- C-D cuesta 3, y como 2 (hasta C) + 3 = 5 $\rightarrow$ D ya tiene 5, no cambia.
- C-B cuesta 2 $\rightarrow$ 2 + 2 = 4 $\rightarrow$ mismo caso anterior, valor 4.



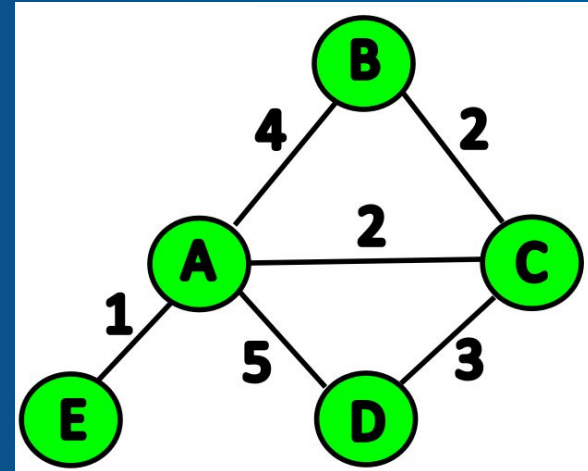
Dijkstra paso a paso

5. Siguiendo $\rightarrow B$ (4):
- Ya no mejora nada.



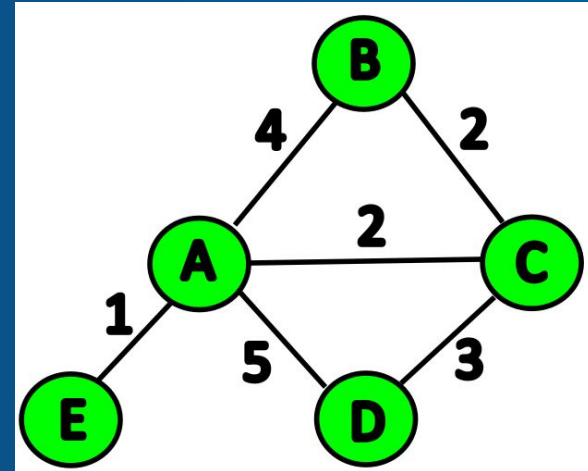
Dijkstra paso a paso

6. Siguiendo $\rightarrow D$ (5):
- Nada nuevo.



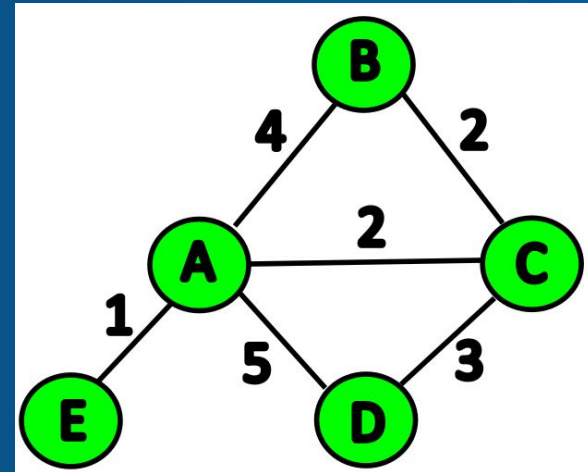
Resultado final (caminos más cortos desde A)

- $A \rightarrow A = 0$
- $A \rightarrow E = 1$
- $A \rightarrow C = 2$
- $A \rightarrow B = 4$
- $A \rightarrow D = 5$



Caminos más cortos

- A-E
- A-C
- A-C-B
- A-C-D



Ejemplo un poco más complejo

- Encontrar los caminos más cortos desde A a todos los demás nodos usando Dijkstra.

Desde	Hacia	Peso
A	B	7
A	C	9
A	E	14
B	C	10
B	F	15
C	D	2
D	E	9
E	F	9
D	F	11

Dijkstra: Paso 0

- Inicializar distancias
 - Distancia desde A a sí misma = 0
 - Todas las demás = ∞
 - Nodo visitado: ninguno

Nodo	Distancia desde A	Anterior
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-

Desde	Hacia	Peso
A	B	7
A	C	9
A	E	14
B	C	10
B	F	15
C	D	2
D	E	9
E	F	9
D	F	11

Paso 1: Desde A

- Vecinos:
 - B: $0 + 7 = 7$
 - C: $0 + 9 = 9$
 - E: $0 + 14 = 14$
- Visitado: A

Nodo	Distancia desde A	Anterior
A	0	-
B	7	A
C	9	A
D	∞	-
E	14	A
F	∞	-

Desde	Hacia	Peso
A	B	7
A	C	9
A	E	14
B	C	10
B	F	15
C	D	2
D	E	9
E	F	9
D	F	11

Paso 2: Nodo más cercano → B (7)

- Vecinos:
 - C: $7 + 10 = 17$ (no mejora C que ya está en 9)
 - F: $7 + 15 = 22$
- Visitados: A, B.

Nodo	Distancia desde A	Anterior
A	0	-
B	7	A
C	9	A
D	∞	-
E	14	A
F	22	B

Desde	Hacia	Peso
A	B	7
A	C	9
A	E	14
B	C	10
B	F	15
C	D	2
D	E	9
E	F	9
D	F	11

Paso 3: Nodo más cercano → C (9)

- Vecinos:
 - D: $9 + 2 = 11$
- Visitados: A, B, C

Nodo	Distancia desde A	Anterior
A	0	-
B	7	A
C	9	A
D	11	C
E	14	A
F	22	B

Desde	Hacia	Peso
A	B	7
A	C	9
A	E	14
B	C	10
B	F	15
C	D	2
D	E	9
E	F	9
D	F	11

Paso 4: Nodo más cercano → D (11)

- Vecinos:
 - E: $11 + 9 = 20$ (no mejora)
 - F: $11 + 11 = 22$ (igual que antes)
- Visitados: A, B, C, D

Nodo	Distancia desde A	Anterior
A	0	-
B	7	A
C	9	A
D	11	C
E	14	A
F	22	B

Desde	Hacia	Peso
A	B	7
A	C	9
A	E	14
B	C	10
B	F	15
C	D	2
D	E	9
E	F	9
D	F	11

Paso 5: Nodo más cercano → E (14)

- Vecinos:
 - F: $14 + 9 = 23$ (no mejora F = 22)
- Visitados: A, B, C, D, E

Nodo	Distancia desde A	Anterior
A	0	-
B	7	A
C	9	A
D	11	C
E	14	A
F	22	B

Desde	Hacia	Peso
A	B	7
A	C	9
A	E	14
B	C	10
B	F	15
C	D	2
D	E	9
E	F	9
D	F	11

Paso 6: Nodo más cercano → F (22)

- Ya no hay mejoras posibles.
- Visitados: A, B, C, D, E, F

Nodo	Distancia desde A	Anterior
A	0	-
B	7	A
C	9	A
D	11	C
E	14	A
F	22	B

Desde	Hacia	Peso
A	B	7
A	C	9
A	E	14
B	C	10
B	F	15
C	D	2
D	E	9
E	F	9
D	F	11

Resultados finales: Caminos más cortos desde A

- Sin Dijkstra, calcular todos los caminos más cortos sería muy lento, especialmente si tuviéramos 100 ciudades.
- Dijkstra encuentra las mejores rutas automáticamente, con eficiencia y sin errores humanos.

Nodo destino	Distancia	Ruta más corta
A	0	A
B	7	A → B
C	9	A → C
D	11	A → C → D
E	14	A → E
F	22	A → B → F A → C → D → F (ambos valen 22)

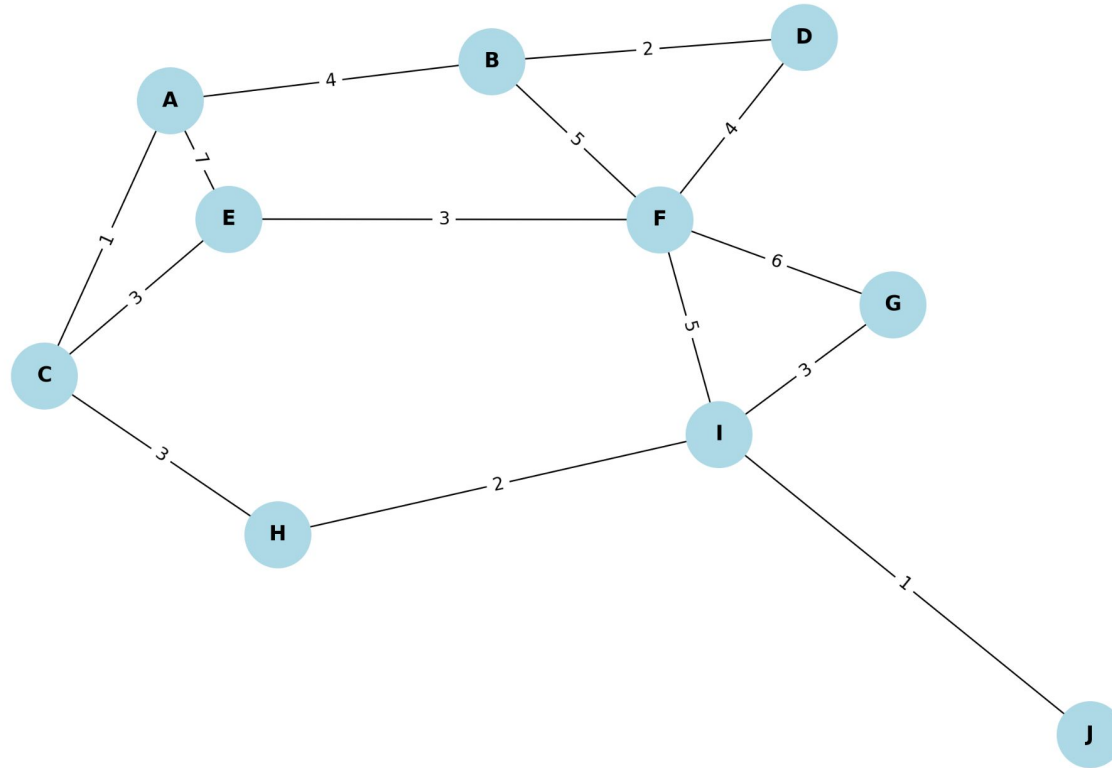
Otro ejemplo



**Encontrar la ruta más
corta desde A a todos
los demás nodos
usando Dijkstra**

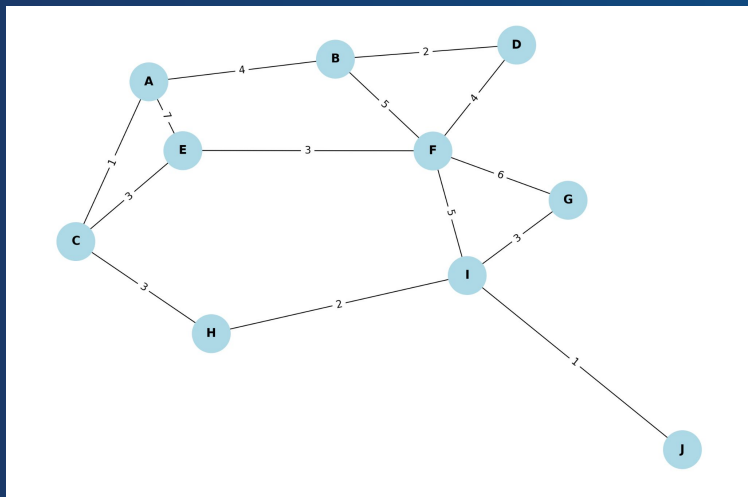
Nodo	Conectado a	Peso
A	B	4
A	C	1
A	E	7
B	D	2
B	F	5
C	H	3
C	E	3
D	F	4
E	F	3
F	G	6
F	I	5
G	I	3
H	I	2
I	J	1

Representación gráfica



Inicializar distancias

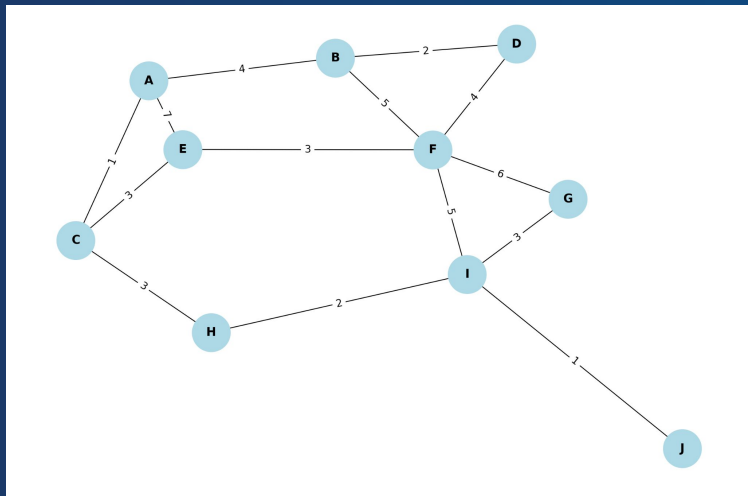
- Distancia desde A a sí misma = 0
- Todas las demás = ∞
- Nodo visitado: ninguno



Nodo	Distancia desde A	Anterior
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-
G	∞	-
H	∞	-
I	∞	-
J	∞	-

Iteración 1: Desde A (distancia 0)

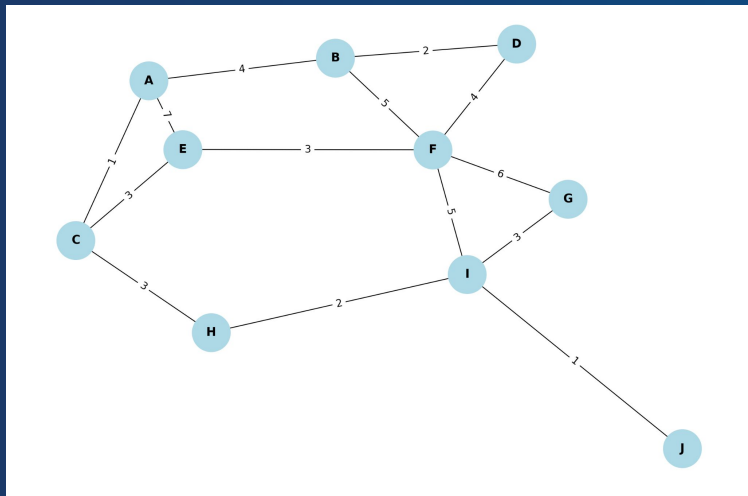
- Vecinos:
 - B: $0 + 4 = 4$
 - C: $0 + 1 = 1$
 - E: $0 + 7 = 7$



Nodo	Distancia desde A	Anterior
A	0	-
B	4	A
C	1	A
D	∞	-
E	7	A
F	∞	-
G	∞	-
H	∞	-
I	∞	-
J	∞	-

Iteración 2: Desde C (1)

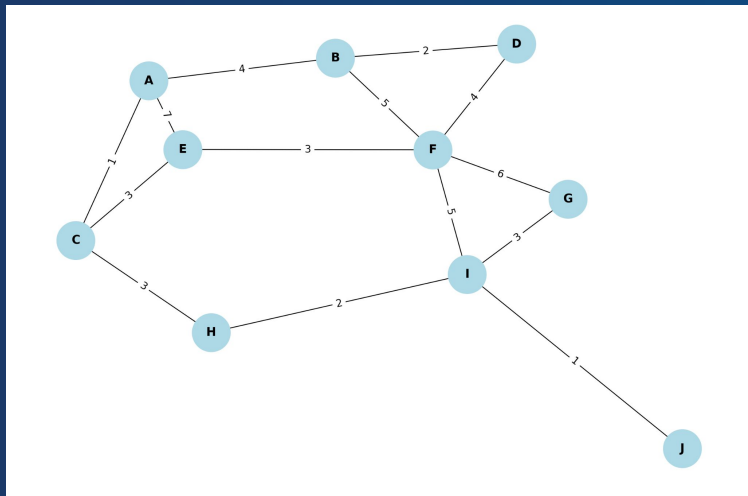
- Vecinos:
 - H: $1 + 3 = 4$
 - E: $1 + 3 = 4$ (mejora E = 7 $\rightarrow$ 4)



Nodo	Distancia desde A	Anterior
A	0	-
B	4	A
C	1	A
D	∞	-
E	4	C
F	∞	-
G	∞	-
H	4	C
I	∞	-
J	∞	-

Iteración 3: Desde B (4)

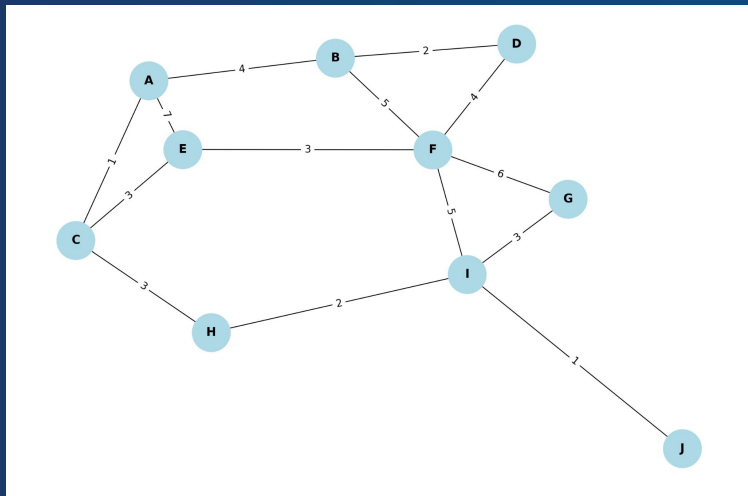
- Vecinos:
 - D: $4 + 2 = 6$
 - F: $4 + 5 = 9$



Nodo	Distancia desde A	Anterior
A	0	-
B	4	A
C	1	A
D	6	B
E	4	C
F	9	B
G	∞	-
H	4	C
I	∞	-
J	∞	-

Iteración 4: Desde E (4)

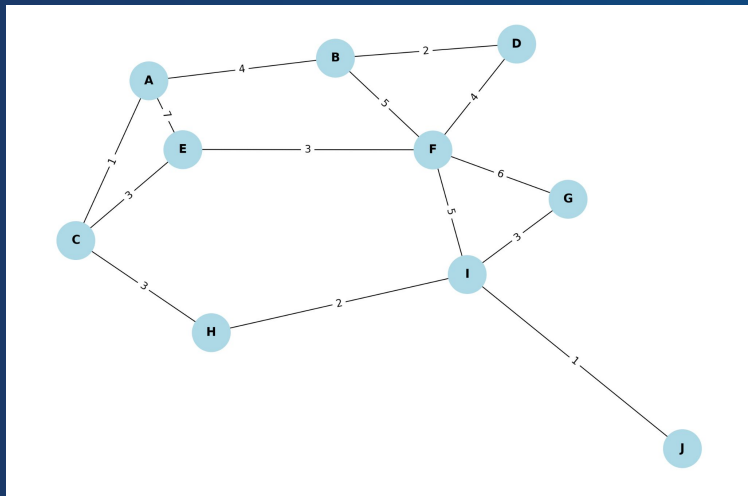
- Vecinos:
 - F: $4 + 3 = 7 \rightarrow$ mejora $F = 9 \rightarrow 7$



Nodo	Distancia desde A	Anterior
A	0	-
B	4	A
C	1	A
D	6	B
E	4	C
F	7	E
G	∞	-
H	4	C
I	∞	-
J	∞	-

Iteración 5: Desde H (4)

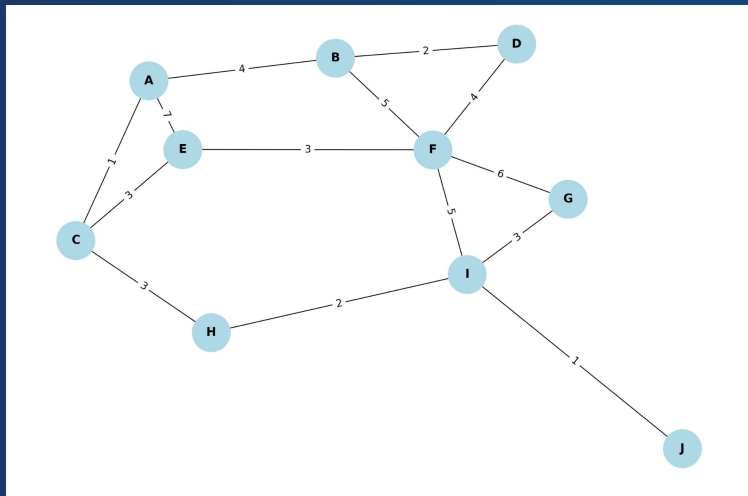
- Vecinos:
 - I: $4 + 2 = 6$



Nodo	Distancia desde A	Anterior
A	0	-
B	4	A
C	1	A
D	6	B
E	4	C
F	7	E
G	∞	-
H	4	C
I	6	H
J	∞	-

Iteración 6: Desde D (6)

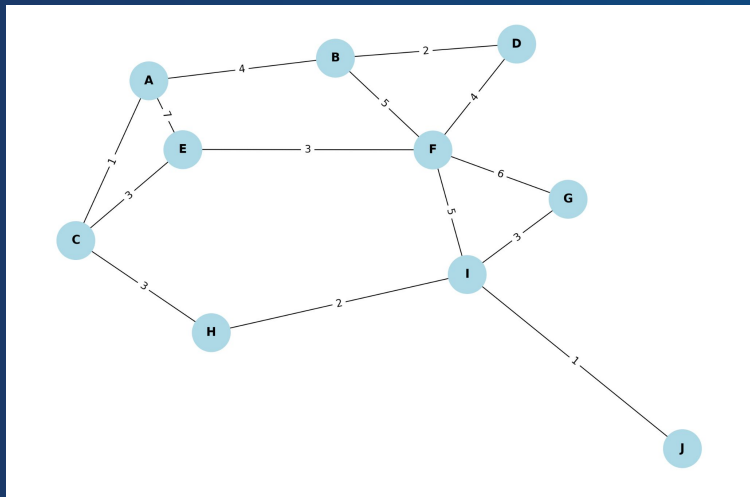
- Vecinos:
 - F: $6 + 4 = 10$ (no mejora F = 7)



Nodo	Distancia desde A	Anterior
A	0	-
B	4	A
C	1	A
D	6	B
E	4	C
F	7	E
G	∞	-
H	4	C
I	6	H
J	∞	-

Iteración 7: Desde I (6)

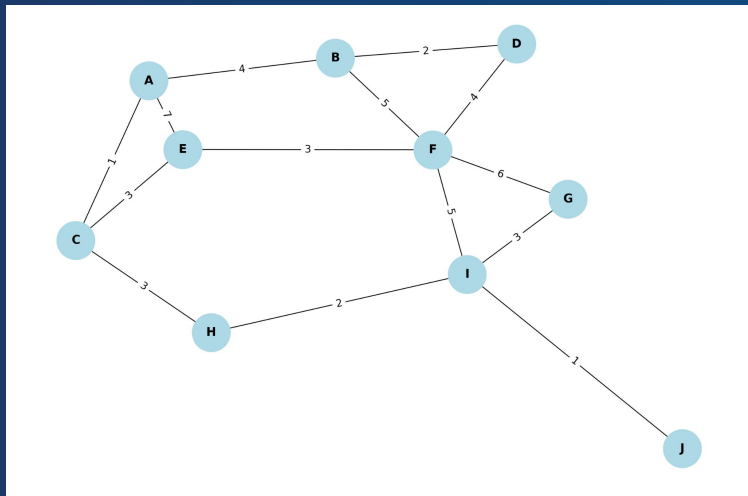
- Vecinos:
 - J: $6 + 1 = 7$
 - G: $6 + 3 = 9$



Nodo	Distancia desde A	Anterior
A	0	-
B	4	A
C	1	A
D	6	B
E	4	C
F	7	E
G	9	I
H	4	C
I	6	H
J	7	I

Iteración 8: Desde F (7)

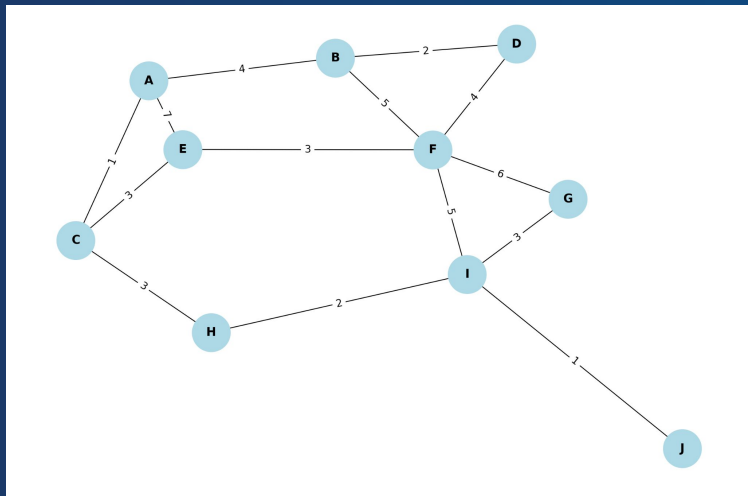
- Vecinos:
 - G: $7 + 6 = 13$ (ya está en 9, no mejora)



Nodo	Distancia desde A	Anterior
A	0	-
B	4	A
C	1	A
D	6	B
E	4	C
F	7	E
G	9	I
H	4	C
I	6	H
J	7	I

Iteración 9: Desde J (7)

- Sin mejoras



Nodo	Distancia desde A	Anterior
A	0	-
B	4	A
C	1	A
D	6	B
E	4	C
F	7	E
G	9	I
H	4	C
I	6	H
J	7	I

Resultado final

Nodo	Distancia	Ruta más corta
A	0	A
B	4	$A \rightarrow B$
C	1	$A \rightarrow C$
D	6	$A \rightarrow B \rightarrow D$
E	4	$A \rightarrow C \rightarrow E$
F	7	$A \rightarrow C \rightarrow E \rightarrow F$
G	9	$A \rightarrow C \rightarrow E \rightarrow F \rightarrow I \rightarrow G$
H	4	$A \rightarrow C \rightarrow H$
I	6	$A \rightarrow C \rightarrow H \rightarrow I$
J	7	$A \rightarrow C \rightarrow H \rightarrow I \rightarrow J$

Dijkstra en C

```
int main() {
    int grafo[V][V] = {
        // A B C D E F G H I J K L M
        { 0, 4, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0 }, // A
        { 4, 0, 1, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0 }, // B
        { 0, 1, 0, 2, 0, 6, 0, 0, 0, 0, 0, 0, 0 }, // C
        { 0, 5, 2, 0, 0, 1, 3, 0, 0, 0, 0, 0, 0 }, // D
        { 2, 0, 0, 0, 0, 4, 0, 5, 0, 0, 0, 0, 0 }, // E
        { 0, 0, 6, 1, 4, 0, 2, 0, 0, 0, 0, 0, 0 }, // F
        { 0, 0, 0, 3, 0, 2, 0, 0, 7, 0, 0, 0, 0 }, // G
        { 0, 0, 0, 0, 5, 0, 0, 0, 1, 6, 0, 0, 0 }, // H
        { 0, 0, 0, 0, 0, 0, 7, 1, 0, 3, 2, 0, 0 }, // I
        { 0, 0, 0, 0, 0, 0, 0, 6, 3, 0, 0, 8, 0 }, // J
        { 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 3 }, // K
        { 0, 0, 0, 0, 0, 0, 0, 0, 0, 8, 1, 0, 4 }, // L
        { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 4, 0 } // M
    };

    dijkstra(grafo, 0); // Nodo A = índice 0
    return 0;
}
```

```
C:\Users\phili\OneDrive\Escritorio>gcc dijkstra.c -o disk
```

```
C:\Users\phili\OneDrive\Escritorio>disk
```

Nodo	Distancia desde A	Ruta
A	0	A
B	4	A -> B
C	5	A -> B -> C
D	7	A -> B -> C -> D
E	2	A -> E
F	6	A -> E -> F
G	8	A -> E -> F -> G
H	7	A -> E -> H
I	8	A -> E -> H -> I
J	11	A -> E -> H -> I -> J
K	10	A -> E -> H -> I -> K
L	11	A -> E -> H -> I -> K -> L
M	13	A -> E -> H -> I -> K -> M

OPT-2 y OPT-3

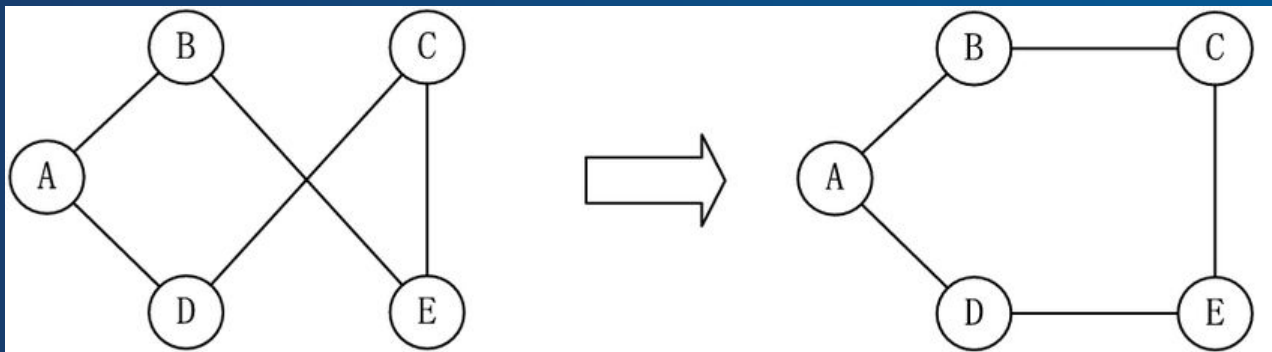


OPT-2 y OPT-3

- Estos algoritmos no construyen una solución desde cero como Prim o Kruskal, sino que mejoran una ruta ya existente.
- Se usan en problemas como el TSP.

OPT-2 (Two-opt)

- **Idea principal:** Se eliminan dos aristas de la ruta y se reconecta el camino intercambiando los extremos, lo que equivale a revertir un subcamino.
- Se prueban múltiples combinaciones de dos aristas.
- Si se encuentra una mejora (menor distancia total), se aplica el cambio.
- Se repite hasta que no haya más mejoras posibles.

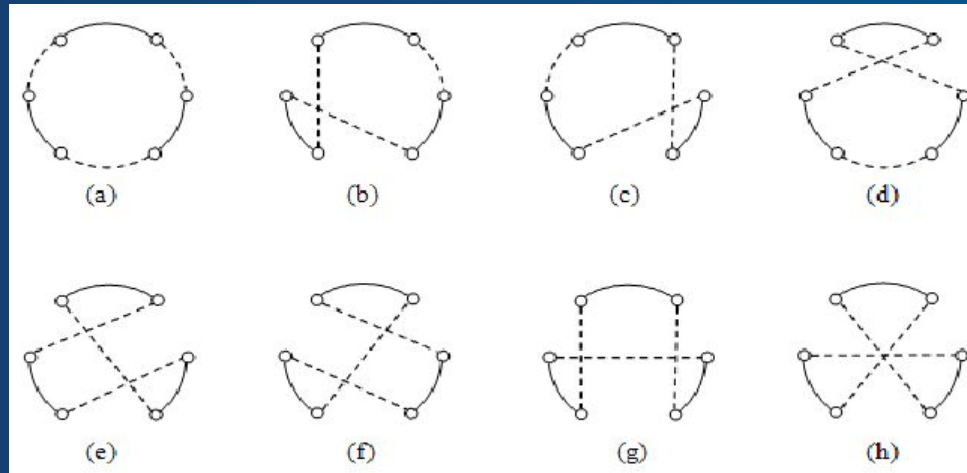


OPT-2 (Two-opt): Ejemplo

- Ruta inicial: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$
- Quito aristas $B \rightarrow C$ y $D \rightarrow E$
- Revierto el subcamino $C \rightarrow D \rightarrow$ queda: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow A$

OPT-3 (Three-opt)

- Extiende la idea de OPT-2, removiendo tres aristas en lugar de dos.
- Hay más combinaciones posibles, pero también mayor capacidad de mejora.
- Es más costoso computacionalmente, pero mejora rutas que OPT-2 no puede.



Algoritmo de Inserción



Algoritmo de Inserción

- Es un algoritmo constructivo usado para el TSP.
- Comienza con una solución parcial y va insertando nodos uno a uno.
 1. Se parte con un ciclo pequeño (usualmente 2 o 3 nodos).
 2. Se elige el nodo más cercano (o más lejano, según la variante) que no está en la ruta.
 3. Se inserta el nodo en el lugar donde menos aumente el costo total.
 4. Repetir hasta insertar todos los nodos.

Variantes

- **Nearest Insertion:** inserta el nodo más cercano al ciclo parcial.
- **Farthest Insertion:** inserta el nodo más lejano (favorece una cobertura más temprana).
- **Random Insertion:** nodo al azar, buena base para GRASP.

Ejemplo de algoritmo de inserción por el más cercano

Nodo	X	Y
1	0	0
2	2	1
3	4	0
4	6	2
5	5	5
6	3	6
7	1	5
8	0	3
9	2	3
10	3	3

Paso 0 (inicialización)

- Elegimos un nodo inicial arbitrario: Nodo 1
- Buscamos el nodo más cercano: Nodo 2
- Ruta inicial: $1 \rightarrow 2 \rightarrow 1$

	1	2	3	4	5	6	7	8	9	10
1	0	2.24	4.00	6.32	7.81	6.71	5.10	3.00	3.61	4.24
2	2.24	0	2.00	4.12	5.00	5.10	4.12	2.83	2.00	2.24
3	4.00	2.00	0	2.83	5.10	6.08	5.39	5.00	3.61	3.16
4	6.32	4.12	2.83	0	3.16	4.47	5.39	6.32	4.47	3.61
5	7.81	5.00	5.10	3.16	0	2.24	4.00	5.83	3.61	2.24
6	6.71	5.10	6.08	4.47	2.24	0	2.24	3.61	2.00	1.41
7	5.10	4.12	5.39	5.39	4.00	2.24	0	2.24	1.41	2.24
8	3.00	2.83	5.00	6.32	5.83	3.61	2.24	0	2.00	3.00
9	3.61	2.00	3.61	4.47	3.61	2.00	1.41	2.00	0	1.00
10	4.24	2.24	3.16	3.61	2.24	1.41	2.24	3.00	1.00	0

Paso 1 (elegir nodo más cercano a la ruta)

- Candidatos: 3–10
- El más cercano: Nodo 3
- Insertamos donde menos aumente el costo: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$

	1	2	3	4	5	6	7	8	9	10
1	0	2.24	4.00	6.32	7.81	6.71	5.10	3.00	3.61	4.24
2	2.24	0	2.00	4.12	5.00	5.10	4.12	2.83	2.00	2.24
3	4.00	2.00	0	2.83	5.10	6.08	5.39	5.00	3.61	3.16
4	6.32	4.12	2.83	0	3.16	4.47	5.39	6.32	4.47	3.61
5	7.81	5.00	5.10	3.16	0	2.24	4.00	5.83	3.61	2.24
6	6.71	5.10	6.08	4.47	2.24	0	2.24	3.61	2.00	1.41
7	5.10	4.12	5.39	5.39	4.00	2.24	0	2.24	1.41	2.24
8	3.00	2.83	5.00	6.32	5.83	3.61	2.24	0	2.00	3.00
9	3.61	2.00	3.61	4.47	3.61	2.00	1.41	2.00	0	1.00
10	4.24	2.24	3.16	3.61	2.24	1.41	2.24	3.00	1.00	0

Paso 2

- Candidatos 4–10
- Más cercano a la ruta actual
(1–2–3–1) → Nodo 4
- Insertamos en la mejor posición

	1	2	3	4	5	6	7	8	9	10
1	0	2.24	4.00	6.32	7.81	6.71	5.10	3.00	3.61	4.24
2	2.24	0	2.00	4.12	5.00	5.10	4.12	2.83	2.00	2.24
3	4.00	2.00	0	2.83	5.10	6.08	5.39	5.00	3.61	3.16
4	6.32	4.12	2.83	0	3.16	4.47	5.39	6.32	4.47	3.61
5	7.81	5.00	5.10	3.16	0	2.24	4.00	5.83	3.61	2.24
6	6.71	5.10	6.08	4.47	2.24	0	2.24	3.61	2.00	1.41
7	5.10	4.12	5.39	5.39	4.00	2.24	0	2.24	1.41	2.24
8	3.00	2.83	5.00	6.32	5.83	3.61	2.24	0	2.00	3.00
9	3.61	2.00	3.61	4.47	3.61	2.00	1.41	2.00	0	1.00
10	4.24	2.24	3.16	3.61	2.24	1.41	2.24	3.00	1.00	0

Paso 3

- Quedan 5–10
- Más cercano: Nodo 10
- Se inserta entre los nodos donde genera menor aumento
- ... y así sucesivamente hasta incluir los 10 nodos.

	1	2	3	4	5	6	7	8	9	10
1	0	2.24	4.00	6.32	7.81	6.71	5.10	3.00	3.61	4.24
2	2.24	0	2.00	4.12	5.00	5.10	4.12	2.83	2.00	2.24
3	4.00	2.00	0	2.83	5.10	6.08	5.39	5.00	3.61	3.16
4	6.32	4.12	2.83	0	3.16	4.47	5.39	6.32	4.47	3.61
5	7.81	5.00	5.10	3.16	0	2.24	4.00	5.83	3.61	2.24
6	6.71	5.10	6.08	4.47	2.24	0	2.24	3.61	2.00	1.41
7	5.10	4.12	5.39	5.39	4.00	2.24	0	2.24	1.41	2.24
8	3.00	2.83	5.00	6.32	5.83	3.61	2.24	0	2.00	3.00
9	3.61	2.00	3.61	4.47	3.61	2.00	1.41	2.00	0	1.00
10	4.24	2.24	3.16	3.61	2.24	1.41	2.24	3.00	1.00	0

Posible resultado

- $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 10 \rightarrow 9 \rightarrow 8 \rightarrow 7 \rightarrow 1$

Aclaración

- En cada paso del algoritmo, hace dos cosas:
 - Selecciona el nodo no visitado más cercano a algún nodo de la ruta actual (esto depende de la variante de inserción: más cercana, más lejana, etc.).
 - Buscas la mejor posición para insertarlo en la ruta actual, es decir, aquella que minimiza el aumento del costo total al agregar ese nodo entre dos nodos consecutivos de la ruta.

Ejercicio: Aplicar algoritmo de inserción

```
adj_matrix = [  
    [0, 12, 25, 18, 30, 22, 17, 28, 35, 20],  
    [12, 0, 14, 20, 16, 28, 24, 19, 27, 31],  
    [25, 14, 0, 11, 24, 15, 21, 26, 18, 29],  
    [18, 20, 11, 0, 13, 17, 23, 14, 32, 25],  
    [30, 16, 24, 13, 0, 19, 12, 22, 20, 27],  
    [22, 28, 15, 17, 19, 0, 16, 21, 14, 18],  
    [17, 24, 21, 23, 12, 16, 0, 15, 26, 19],  
    [28, 19, 26, 14, 22, 21, 15, 0, 17, 13],  
    [35, 27, 18, 32, 20, 14, 26, 17, 0, 11],  
    [20, 31, 29, 25, 27, 18, 19, 13, 11, 0]  
]
```


***GRASP: Greedy Randomized
Adaptive Search Procedure***

GRASP

- Es un método que pertenece a la categoría de metaheurísticas.
- Su nombre viene de “Procedimientos de búsqueda basados en funciones Greedy aleatorizadas adaptativas”.
- Se basa en el procedimiento de búsqueda greedy, aleatorio y adaptativo, que puede ser aplicado a una amplia gama de problemas de optimización, garantizando una buena solución, aunque no necesariamente la óptima.

Historia de GRASP

- Fue desarrollado al final de la década de los ochenta.
- El término fue introducido por Feo y Resende en 1995 como una nueva técnica metaheurística de propósito general.
- Este tipo de implementaciones son generalmente robustas y se hace difícil encontrar aplicaciones donde el método funcione mal.

Algoritmo GRASP

1. **Inicialización:** selecciona un conjunto inicial de soluciones candidatas aleatorias mediante un criterio Greedy.
2. **Iteraciones:**
 - a. **Construcción de la solución:** fase donde se genera una solución parcial o completa.
 - b. **Mejora local:** Aplica una búsqueda local para mejorar la solución actual.
 - c. **Actualización de la mejor solución:** si la solución actual es mejor a la registrada, esta la reemplaza.
3. **Criterio de parada:** número de iteraciones o por no mejora de solución.

Parámetros del Algoritmo GRASP

1. Tamaño de la Lista Restringida de Candidatos (RCL): $k = 3$. En cada paso de la fase constructiva se consideran los dos candidatos con mejor evaluación y se selecciona uno de ellos de manera aleatoria, incorporando diversificación al proceso de construcción de la solución. También existe la notación con α (alfa).
2. Número de iteraciones: 1 (utilizada únicamente con fines ilustrativos para mostrar detalladamente el funcionamiento del algoritmo).
3. Búsqueda local: algoritmo 2-OPT, aplicado posteriormente a la solución construida para mejorar la calidad de la ruta mediante intercambios de aristas que reduzcan el costo total.

Ejemplo de GRASP aplicado a TSP

- Misión: buscar un circuito que minimice la distancia total.

	A	B	C	D	E	F
A	-	2	9	10	7	4
B	2	-	6	4	3	8
C	9	6	-	5	4	3
D	10	4	5	-	3	6
E	7	3	4	3	-	2
F	4	8	3	6	2	-

Paso 1: Construcción aleatoria-greedy

- Usamos una versión aleatoria del algoritmo de inserción.
 1. Elegir una ciudad al azar como punto de partida: Ruta = [A].

Paso 1: Construcción aleatoria-greedy

- Usamos una versión aleatoria del algoritmo de inserción.
 1. Elegir una ciudad al azar como punto de partida: Ruta = [A].
 2. Agregar ciudades cercanas a A (greedy con aleatoriedad):
 - Desde A: B(2), F(4), C(9), D(10).
 - Tomamos las 3 más cercanas $\rightarrow$ B, F, C
 - Se selecciona una al azar entre estas 3 (supongamos que sale F).
 - La ruta actuar resulta: [A, F].

Paso 1: Construcción aleatoria-greedy

3. Agregamos otra ciudad cercana a A o F:
 - a. Desde A: B (2), E (7), C (9), D (10)
 - b. Desde F: C (3), E (2), D (6)
 - c. Candidatos: B, E, C, D
 - d. Seleccionamos 3 más cercanos: E (2), B (2), C (3)
 - e. Elegimos aleatoriamente: B
 - f. Ruta actual: [A, F, B]

Paso 1: Construcción aleatoria-greedy

4. **Variante opcional:** Insertamos el resto de ciudades una por una en la posición que menos aumente la distancia total.
 - a. Ciudades restantes: C, D, E
 - b. Probamos insertar cada una en cada hueco (entre dos nodos) y elegimos la mejor (mínimo aumento de distancia)
 - c. Repetimos hasta completar el ciclo.
5. Ruta construida: $A \rightarrow F \rightarrow B \rightarrow E \rightarrow D \rightarrow C \rightarrow A$
 - a. Costo: $A-F (4) + F-B (8) + B-E (3) + E-D (3) + D-C (5) + C-A (9) = 32$

Paso 2: Mejora local con OPT-2

Revisa cada par de aristas y evalúa si invertirlas reduce la distancia total.
Si encuentra una mejora, realiza la inversión y repite.

1. Analizamos pares de aristas:

- A-F y B-E
- F-B y E-D
- B-E y D-C
- ...

Paso 2: Mejora local con OPT-2

2. Intercambiamos extremos si reduce el costo. Por ejemplo:
 - a. Reemplazamos F-B y E-D por F-E y B-D
 - b. Nuevas conexiones: $A \rightarrow F \rightarrow E \rightarrow B \rightarrow D \rightarrow C \rightarrow A$
 - c. Nuevas distancias:
 - i. $A-F (4) + F-E (2) + E-B (3) + B-D (4) + D-C (5) + C-A (9) = 27$
3. Seguimos iterando mientras encontremos mejoras.

Paso 3: Repetimos varias veces

- La gracia de GRASP es que repite varias veces este proceso, cada vez con una construcción aleatoria diferente, y se queda con la mejor ruta encontrada.
- Ejemplo

Iteración	Ruta construida	Costo final (con OPT-2)
1	A-F-B-E-D-C-A	27
2	A-B-C-F-E-D-A	30
3	A-E-B-F-C-D-A	26
...	...	...

Conclusión sobre el GRASP

- GRASP = Construcción aleatoria + mejora local
- Ideal para:
 - Problemas NP-hard como TSP
 - Evitar quedarse atrapado en mínimos locales
 - Obtener buenas soluciones sin explorar todo el espacio

Pseudocódigo de GRASP

```
algorithm GRASP( $f(\cdot)$ ,  $g(\cdot)$ , MaxIterations, Seed)
1   $x_{best} := \emptyset$ ;  $f(x_{best}) := +\infty$ ;
2  for  $k = 1, 2, \dots, \text{MaxIterations} \rightarrow$ 
3     $x := \text{ConstructGreedyRandomizedSolution}(\text{Seed}, g(\cdot))$ ;
4    if ( $x$  not feasible) then
5       $x := \text{repair}(x)$ ;
6    endif
7     $x := \text{LocalSearch}(x, f(\cdot))$ ;
8    if ( $f(x) < f(x_{best})$ ) then
9       $x_{best} := x$ ;
10   endif
11 endfor;
12 return( $x_{best}$ );
end GRASP
```

Otro ejemplo de GRASP



GRASP para el TSP con 10 ciudades (A–J)

- Utilizaremos:
 1. Matriz de distancias.
 2. Fase de construcción aleatoria-greedy (inserción aleatoria).
 3. Fase de mejora local con OPT-2.
 4. Repetimos varias veces para obtener la mejor ruta.

	A	B	C	D	E	F	G	H	I	J
A	-	29	20	21	16	31	100	12	4	31
B	29	-	15	29	28	40	72	21	29	41
C	20	15	-	15	14	25	81	9	23	27
D	21	29	15	-	4	12	92	12	25	13
E	16	28	14	4	-	16	94	9	20	16
F	31	40	25	12	16	-	95	24	36	3
G	100	72	81	92	94	95	-	90	101	99
H	12	21	9	12	9	24	90	-	15	25
I	4	29	23	25	20	36	101	15	-	35
J	31	41	27	13	16	3	99	25	35	-

2. Fase Constructiva (Greedy Randomized Insertion)

- Elegimos una ciudad inicial aleatoria y luego insertamos ciudades usando una lista de candidatos ordenada por cercanía pero eligiendo aleatoriamente entre las más cercanas.

	A	B	C	D	E	F	G	H	I	J
A	-	29	20	21	16	31	100	12	4	31
B	29	-	15	29	28	40	72	21	29	41
C	20	15	-	15	14	25	81	9	23	27
D	21	29	15	-	4	12	92	12	25	13
E	16	28	14	4	-	16	94	9	20	16
F	31	40	25	12	16	-	95	24	36	3
G	100	72	81	92	94	95	-	90	101	99
H	12	21	9	12	9	24	90	-	15	25
I	4	29	23	25	20	36	101	15	-	35
J	31	41	27	13	16	3	99	25	35	-

2. Fase Constructiva (Greedy Randomized Insertion)

- Un ejemplo podría ser:
 - a. Ciudad inicial: A -> [A].
 - b. Se eligen las 3 más cercanas, y de ellas, se elige una aleatoriamente: I.

	A	B	C	D	E	F	G	H	I	J
A	-	29	20	21	16	31	100	12	4	31
B	29	-	15	29	28	40	72	21	29	41
C	20	15	-	15	14	25	81	9	23	27
D	21	29	15	-	4	12	92	12	25	13
E	16	28	14	4	-	16	94	9	20	16
F	31	40	25	12	16	-	95	24	36	3
G	100	72	81	92	94	95	-	90	101	99
H	12	21	9	12	9	24	90	-	15	25
I	4	29	23	25	20	36	101	15	-	35
J	31	41	27	13	16	3	99	25	35	-

2. Fase Constructiva (Greedy Randomized Insertion)

c. Siguiente candidatos desde A e I:

- Desde A: H (12), E (16), C (20)
- Desde I: H (15), E (20), C (23)
- Mejores: H, E, C $\rightarrow$ al azar sale: H
- Ruta actual $\rightarrow$ [A, I, H]

	A	B	C	D	E	F	G	H	I	J
A	-	29	20	21	16	31	100	12	4	31
B	29	-	15	29	28	40	72	21	29	41
C	20	15	-	15	14	25	81	9	23	27
D	21	29	15	-	4	12	92	12	25	13
E	16	28	14	4	-	16	94	9	20	16
F	31	40	25	12	16	-	95	24	36	3
G	100	72	81	92	94	95	-	90	101	99
H	12	21	9	12	9	24	90	-	15	25
I	4	29	23	25	20	36	101	15	-	35
J	31	41	27	13	16	3	99	25	35	-

2. Fase Constructiva (Greedy Randomized Insertion)

- d. Se insertan los nodos restantes, comprobando su contribución entre cada par de la ruta actual, con el fin de minimizar el aumento total. Se continúa hasta que estén las 10 ciudades consideradas en la ruta.

Ruta inicial GRASP: A → I → H → C → D → E → F → J → B → G →
A

3. Mejora local con OPT-2

- Cambio entre $B \rightarrow G$ y $J \rightarrow B \rightarrow$ ¿Y si cambiamos el orden de visita a B y J?
- Cambio entre $F \rightarrow J$ y $D \rightarrow E$
- Se prueba cada par de aristas no adyacentes y se calcula si al revertir el subcamino mejora el total.
- Posible ruta final:
 - Ruta mejorada: $A \rightarrow I \rightarrow H \rightarrow C \rightarrow D \rightarrow F \rightarrow J \rightarrow E \rightarrow B \rightarrow G \rightarrow A$
 - Coste: 262

4. Repetir iteraciones

- GRASP se basa en repetir la construcción aleatoria + mejora.

Iteración	Ruta construida	Costo OPT-2
1	A-I-H-C-D-E-F-J-B-G-A	294 → 262
2	A-H-C-I-D-E-F-B-G-J-A	310 → 265
3	A-C-H-D-I-F-E-B-J-G-A	301 → 257
4	A-I-H-D-E-C-B-F-J-G-A	307 → 266

- Mejor ruta final: $A \rightarrow C \rightarrow H \rightarrow D \rightarrow I \rightarrow F \rightarrow E \rightarrow B \rightarrow J \rightarrow G \rightarrow A$ con costo 257.

Ejercicios



Ejercicio 1: Aplicar Dijkstra para determinar caminos mínimos

Nodo	Vecino	Peso
A	B	6
A	C	2
A	D	4
B	D	3
B	E	1
C	D	1
C	F	7
D	G	5
E	H	8
F	G	4
G	H	2

Ejercicio 2: Aplicar Dijkstra para determinar caminos mínimos

[illegible]

Ejercicio: Utilizar OPT-2 y OPT-3 para mejorar solución

- Un técnico de mantenimiento debe visitar 15 máquinas distribuidas en una planta industrial. El técnico parte desde la máquina 1 y debe recorrer todas las demás una sola vez, retornando finalmente al punto de inicio (ciclo cerrado).
- El orden inicial de visita fue generado usando una heurística simple (inserción más cercana). Sin embargo, el supervisor sospecha que la ruta puede mejorarse.
- Se entrega la lista de coordenadas (x, y) de las máquinas y la ruta inicial.

Máquina	X	Y
1	5	5
2	15	5
3	25	5
4	35	5
5	45	5
6	45	15
7	35	15
8	25	15
9	15	15
10	5	15
11	5	25
12	15	25
13	25	25
14	35	25
15	45	25

Código ejemplo Dijkstra

```
#include <stdio.h>

#define V 13
#define INF 9999

void dijkstraRec(int grafo[V][V], int dist[V], int padre[V], int visitado[V], int u){
    visitado[u] = 1;

    for (int v = 0; v < V; v++) {
        if (!visitado[v] && grafo[u][v] > 0) {
            int nuevaDist = dist[u] + grafo[u][v];
            if (nuevaDist < dist[v]) {
                dist[v] = nuevaDist;
                padre[v] = u;
            }
        }
    }

    // Elegir siguiente nodo no visitado con menor distancia
    int min = INF, next = -1;
    for (int i = 0; i < V; i++)
        if (!visitado[i] && dist[i] < min)
            min = dist[i], next = i;

    if (next != -1)
        dijkstraRec(grafo, dist, padre, visitado, next);
}
```

```
void imprimirRuta(int padre[], int j) {
    if (padre[j] == -1) {
        printf("%c", 'A' + j);
        return;
    }
    imprimirRuta(padre, padre[j]);
    printf(" -> %c", 'A' + j);
}

void dijkstra(int grafo[V][V], int origen) {
    int dist[V], padre[V], visitado[V] = {0};

    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        padre[i] = -1;
    }
    dist[origen] = 0;

    dijkstraRec(grafo, dist, padre, visitado, origen);

    printf("Nodo\tDistancia desde A\tRuta\n");
    for (int i = 0; i < V; i++) {
        printf("%c\t\t%d\t\t", 'A' + i, dist[i]);
        imprimirRuta(padre, i);
        printf("\n");
    }
}
```

Código ejemplo GRASP

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 5          // Número de ciudades
#define RCL_SIZE 2   // Tamaño de la lista RCL (greedy-random)

// Matriz de distancias fija entre 5 ciudades
int dist[N][N] = {
    {0, 2, 9, 10, 7},
    {1, 0, 6, 4, 3},
    {15, 7, 0, 8, 3},
    {6, 3, 12, 0, 11},
    {10, 4, 8, 5, 0}
};

// Función para calcular el costo total de un tour
int calcularCosto(int tour[N]) {
    int costo = 0;
    for (int i = 0; i < N - 1; i++)
        costo += dist[tour[i]][tour[i + 1]];
    costo += dist[tour[N - 1]][tour[0]]; // Regreso al origen
    return costo;
}
```

```
// Greedy Randomized Construction
void construirSolucion(int tour[N]) {
    int visitado[N] = {0};
    int actual = rand() % N;
    tour[0] = actual;
    visitado[actual] = 1;

    for (int i = 1; i < N; i++) {
        int candidatos[N], costos[N], count = 0;

        // Construir lista de candidatos no visitados
        for (int j = 0; j < N; j++) {
            if (!visitado[j]) {
                candidatos[count] = j;
                costos[count] = dist[actual][j];
                count++;
            }
        }

        // Ordenar candidatos por costo (greedy)
        for (int j = 0; j < count - 1; j++) {
            for (int k = j + 1; k < count; k++) {
                if (costos[j] > costos[k]) {
                    int tmpC = costos[j], tmpN = candidatos[j];
                    costos[j] = costos[k]; candidatos[j] = candidatos[k];
                    costos[k] = tmpC; candidatos[k] = tmpN;
                }
            }
        }

        // Elegir aleatoriamente de la lista RCL
        int elegido = candidatos[rand() % (count < RCL_SIZE ? count : RCL_SIZE)];
        tour[i] = elegido;
        visitado[elegido] = 1;
        actual = elegido;
    }
}
```

Código ejemplo GRASP

```
// Aplicar 2-opt simple
void aplicar2opt(int tour[N]) {
    int mejora = 1;
    while (mejora) {
        mejora = 0;
        for (int i = 1; i < N - 2; i++) {
            for (int j = i + 1; j < N - 1; j++) {
                int a = tour[i - 1], b = tour[i];
                int c = tour[j], d = tour[j + 1];

                int delta = (dist[a][c] + dist[b][d]) - (dist[a][b] + dist[c][d]);
                if (delta < 0) {
                    // Invertir el segmento
                    for (int k = 0; k < (j - i + 1) / 2; k++) {
                        int temp = tour[i + k];
                        tour[i + k] = tour[j - k];
                        tour[j - k] = temp;
                    }
                    mejora = 1;
                }
            }
        }
    }
}
```

```
// Imprimir tour
void imprimirTour(int tour[N]) {
    for (int i = 0; i < N; i++)
        printf("%d -> ", tour[i]);
    printf("%d\n", tour[0]); // Volver al origen
}

int main() {
    srand(time(NULL));

    int mejorTour[N], tour[N], mejorCosto = 1e9;

    // GRASP: múltiples iteraciones
    for (int iter = 0; iter < 30; iter++) {
        construirSolucion(tour);
        aplicar2opt(tour);

        int costo = calcularCosto(tour);
        if (costo < mejorCosto) {
            mejorCosto = costo;
            for (int i = 0; i < N; i++) mejorTour[i] = tour[i];
        }
    }

    printf("Mejor tour encontrado:\n");
    imprimirTour(mejorTour);
    printf("Costo total: %d\n", mejorCosto);

    return 0;
}
```

Enfoque IA

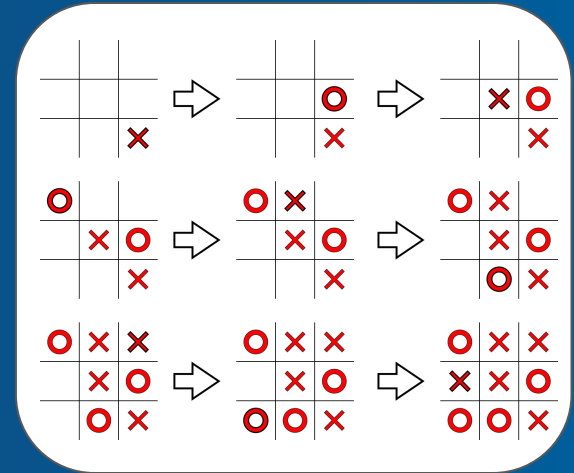
Enfoque IA

El espacio de estados



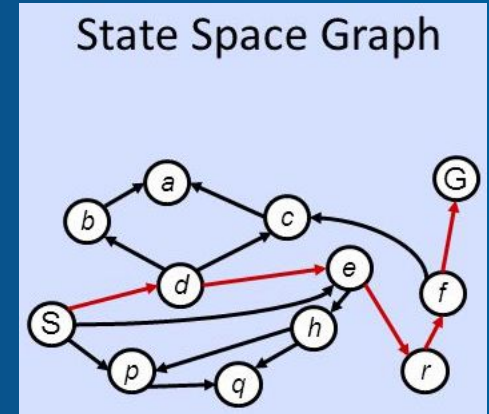
El Universo de lo Posible: El Espacio de Estados

- Es el conjunto de TODAS las configuraciones posibles en las que puede encontrarse nuestro problema desde el inicio hasta el fin.
- **El problema de la Explosión Combinatoria:** Un cubo Rubik tiene 43×10^{18} estados. El ajedrez tiene 10^{40} . En diseño de redes, el espacio de estados crece exponencialmente con cada cámara de inspección y ducto que agregamos al grafo.



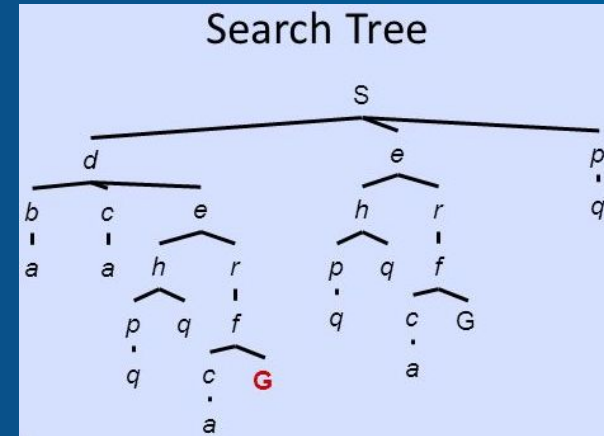
El Grafo (El Espacio de Estados)

- Es el mapa físico o matemático del problema. Las reglas del juego.
- **Naturaleza:** Es una entidad estática. Existe independientemente de si un algoritmo está corriendo o no.
- **Nodos Únicos:** Cada estado (situación) posible existe una sola vez en el grafo.
- **Tamaño:** Generalmente finito.



El Árbol de Búsqueda (La Ejecución)

- Es la traza histórica de las decisiones que está tomando el algoritmo.
- Naturaleza: Es dinámico. Se construye sobre la marcha y paso a paso en la memoria RAM.
- **Nodos Repetidos:** Un mismo estado del grafo puede aparecer en múltiples ramas del árbol si se llegó a él por distintos caminos (ciclos).
- **Tamaño:** Potencialmente infinito, incluso si el grafo es pequeño.



--	--

State Space Graph

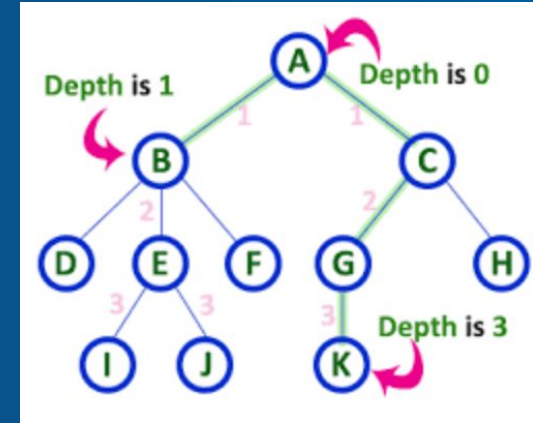
The graph illustrates a state space with nodes S, a, b, c, d, e, f, G, h, p, q, and r. The edges are as follows:

- Black edges: S to d, S to p, S to e, d to b, d to c, c to a, a to b, h to p, h to q, q to p, e to h, e to c, e to f, f to G.
- Red edges: S to d, d to e, e to r, r to f, f to G.

[illegible]

Anatomía de un Árbol de Búsqueda

- **Nodo Raíz:** El Estado Inicial (S_0). Profundidad 0.
- **Nodos Hoja:** Nodos que aún no han sido expandidos (Frontera) o callejones sin salida.
- **Profundidad (d):** El número de pasos (aristas) desde la raíz hasta un nodo específico.
- **Factor de Ramificación (b):** El número promedio de "hijos" que genera un nodo al expandirse. (Ej. Si en cada esquina puedo girar a 3 calles, $b=3$).



El motor de búsqueda

ESD

Algoritmo general de búsqueda en grafos (i)

1. Inicialización

- a. Frontera (Open List): Estructura de datos con los nodos descubiertos pero no revisados. Inicia solo con el Estado Inicial (S_0).
- b. Explorados (Closed List / Historial): Estructura vacía. Aquí guardaremos los nodos que ya procesamos para no repetir (evita los ciclos infinitos).

Algoritmo general de búsqueda en grafos (ii)

2. El Ciclo de Exploración (Mientras la Frontera NO esté vacía)
 - a. Extraer: Sacamos un nodo de la Frontera. (El criterio de extracción define si el algoritmo es BFS, DFS o UCS).
 - b. Prueba de Objetivo (Goal Test). ¿Este nodo es el nodo destino?
 - i. Si SÍ: ¡Detener el algoritmo! Devolvemos la ruta trazada.
 - ii. Si NO: Continuamos al paso C.
 - c. Archivar: Guardamos el nodo actual en la lista de Explorados.
 - d. Expandir: Usamos la Función de Transición para ver qué acciones son posibles desde aquí. Generamos los nodos "hijos" (vecinos).
 - e. Actualizar Frontera: Agregamos los hijos nuevos a la Frontera solo si no están ya en la Frontera ni en Explorados.

Algoritmo general de búsqueda en grafos (iii)

3. Condición de Fracaso

- a. Si la Frontera se queda vacía y nunca se cumplió la Prueba de Objetivo, el algoritmo devuelve "Falla" (no hay solución posible).

Formulación Formal del Problema



Formulación Formal del Problema

- **El desafío:** Un algoritmo general está vacío. Es un "cascarón". No sabe resolver nada hasta que le entregamos las reglas específicas del problema.
- **La Solución:** Para que la IA resuelva un problema de ingeniería, debemos definir matemáticamente 4 componentes. Si falta uno, no podemos programar:
 - Estado Inicial (S_0)
 - Modelo de Transición (Acciones)
 - Prueba de Objetivo (Goal Test)
 - Costo de Ruta ($g(n)$)

1.- El Estado Inicial o punto de partida

- Es la "fotografía" exacta de cómo se encuentra el mundo en el tiempo $t=0$, antes de que el algoritmo tome cualquier decisión.
- En el Árbol de Búsqueda: Corresponde al Nodo Raíz. Es el único elemento que insertamos en nuestra Frontera al inicio.
- Ejemplo en nuestro Caso de Estudio (Diseño de Red):
 - El mapa físico (grafo) de los ductos subterráneos está definido, pero ningún ducto tiene cable instalado.
 - Los servidores que debemos conectar están apagados y aislados.
 - Costo acumulado = 0.

2.- Acciones y Transición

- A. Las Acciones Disponibles: $Actions(S)$
 - a. Dado un estado actual S , ¿qué movimientos legales permite la física o la lógica del problema?
 - b. Importante: No todas las acciones están disponibles en todos los estados (Ej: No puedo girar a la izquierda si hay una pared).
- B. El Modelo de Transición: $Result(S, A)$
 - a. Es la función matemática. Si estoy en el estado S y ejecuto la acción A , ¿cuál es el estado exacto resultante S_{nuevo} ?

Visualizando el Estado Sucesor

- El Engranaje del Árbol: Generando Sucesores
 - [Estado Actual S] $\rightarrow$ (Ej. Servidor A conectado con Servidor B)
 - + [Acción A] $\rightarrow$ (Ej. Acción: Instalar cable entre B y nodo intermedio C)
 - = [Estado Sucesor S_{nuevo}] (Ej. Servidor A conectado con B, y B conectado con C)
- Cuando el algoritmo "expande" un nodo, lo que hace es tomar el estado S , listar todas las acciones legales, y aplicarles la función de transición para generar todos los sucesores posibles.

3.- Prueba de Objetivo (Goal Test)

- ¿Cómo sabemos que terminamos?
 - Se utiliza una función booleana que evalúa un estado y determina si cumple las condiciones para ser la solución final.
 - ¿Por qué es necesario? La computadora no "ve" el problema. No sabe que el laberinto terminó solo por mirar la salida. Necesita una comprobación matemática.
 - Ejemplo en el Diseño de Red (Steiner):
 - Falso: "Solo 3 de los 4 servidores están en la red."
 - Verdadero: "Existe un camino de conexiones ininterrumpidas que incluye al Servidor A, B, C y D."

4.- Costo de Ruta ($g(n)$)

- Costo de Acción y Costo de Ruta $g(n)$
 - Costo de Acción: $c(S, A, S')$
 - El costo individual de tomar una acción A para ir de S a S' . Siempre debe ser positivo (mayor o igual a cero).
 - Costo de Ruta Acumulado: $g(n)$
 - Es la suma aditiva de todos los costos de acción desde el Estado Inicial hasta el nodo actual n .
 - El Objetivo de la Optimización:
 - En ingeniería, casi siempre queremos encontrar la meta que minimice este valor $g(n)$.



Costos Reales vs. Teóricos

- En los libros de IA Básica (Costos Uniformes):
 - Cada paso en un laberinto o tablero cuesta exactamente 1 unidad.
 - Minimizar el costo es igual a minimizar la cantidad de pasos (saltos).
- En la Ingeniería (Costos Ponderados Reales):
 - Distancia: Metros de cable.
 - Dinero: Romper pavimento (costoso) vs. instalación aérea (barato).
 - Tiempo/Latencia: Retardo en milisegundos en redes de fibra óptica.
 - Consecuencia: ¡El camino con menos pasos no siempre es el más barato!

Resumen de Nuestro Caso de Estudio

- Problema de Nodos de Steiner (Redes): Conectar los servidores obligatorios en un campus universitario minimizando el costo.
 - 1. Estado Inicial: Ningún ducto seleccionado (grafo vacío).
 - 2. Acciones: Seleccionar un tramo de ducto adyacente a la red actual para instalar cable.
 - 3. Prueba de Objetivo: ¿Están los servidores obligatorios conectados entre sí mediante los ductos seleccionados?
 - 4. Costo de Ruta: La suma en dólares (basado en el tipo de terreno y distancia) de todos los ductos seleccionados hasta el momento.

Ejercicio Práctico de Formulación

- El Reto: Aspiradora Robótica Industrial.
- Contexto: Un robot aspiradora debe limpiar una bodega dividida en una cuadrícula (grilla). Hay zonas sucias y hay obstáculos (pilares).
- Instrucciones para los estudiantes: Trabajen en parejas y definan en papel los 4 componentes para que este robot limpie toda la bodega de forma óptima.
 - Estado Inicial
 - Acciones válidas
 - Prueba de objetivo
 - Costo de ruta

Búsqueda no informada



Búsqueda Ciega: Caminando en la Niebla

- Un algoritmo de búsqueda "no informada" (o ciega) es aquel que solo tiene acceso a la información proporcionada en la formulación del problema (los 4 pilares anteriores).
- Lo que SÍ sabe el algoritmo:
 - Dónde empezó (Estado Inicial).
 - Cuáles son las reglas (Acciones y Transición).
 - Cuánto ha gastado hasta ahora ($g(n)$).
 - Si un estado es la meta o no (Goal Test).
- Lo que NO sabe el algoritmo: A qué distancia o a qué costo está la meta desde su posición actual. No tiene una "brújula" ni un sentido de dirección.

Gestión de Memoria (Evitando el Colapso)

1. La Frontera (Open List / Lista de Pendientes)
 - a. Es el límite entre lo conocido y lo desconocido.
 - b. Contiene todos los nodos que ya descubrimos (los "hijos" generados), pero que aún no hemos evaluado ni expandido.
 - c. El algoritmo extrae su próximo movimiento de aquí.
2. Los Explorados (Closed List / Historial)
 - a. Contiene todos los nodos que ya evaluamos y expandimos.
 - b. Propósito Vital: Si el algoritmo genera un "hijo" que ya está en esta lista, lo descarta inmediatamente. Esto evita los temidos bucles infinitos en grafos con ciclos.

Estructuras de Datos: El Corazón del Algoritmo

- Pila (Stack):
 - Lógica LIFO (Last-In, First-Out). El último en entrar es el primero en salir.
- Cola (Queue):
 - Lógica FIFO (First-In, First-Out). El primero en entrar es el primero en salir.
- Cola de Prioridad (Priority Queue):
 - Los elementos se ordenan automáticamente según un valor numérico (peso/costo).

Detalle de las Métricas

- **Compleitud:** ¿Garantiza el algoritmo encontrar una solución si al menos una existe? (Y si no existe, ¿garantiza detenerse y avisarnos?).
- **Optimalidad:** Si hay múltiples soluciones posibles, ¿garantiza encontrar la ruta con el costo $g(n)$ más bajo posible?
- **Complejidad Temporal:** ¿Cuánto tarda en encontrar la solución? En IA, esto no se mide en segundos, sino en la cantidad de nodos que genera/expande. Se usa notación Big O ($O(\dots)$).
- **Complejidad Espacial:** ¿Cuánta memoria RAM requiere? En IA, esto es la cantidad máxima de nodos retenidos en memoria al mismo tiempo (el tamaño máximo de la Frontera + Explorados).

Detalle de las Métricas

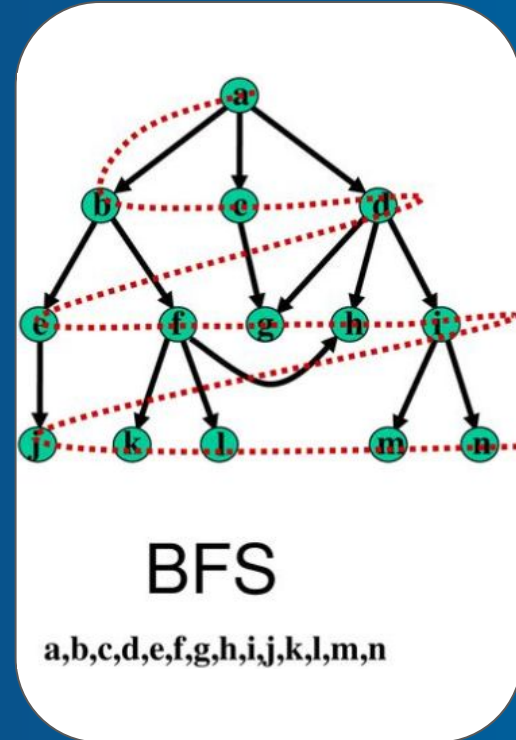
- Variables universales para medir el tiempo y espacio:
 - b = Factor de ramificación máximo (número de hijos por nodo).
 - d = Profundidad del nodo objetivo más superficial.



Búsqueda en Anchura (BFS - Breadth-First Search)

BFS: La estrategia de la onda expansiva

- Explora el árbol de búsqueda estrictamente por niveles (profundidad).
- No desciende al nivel de profundidad $d+1$ hasta haber revisado absolutamente todos los nodos del nivel d .
- Es como hacer un ping de red que salta a todos los routers a 1 salto de distancia, luego a todos los que están a 2 saltos, y así sucesivamente.

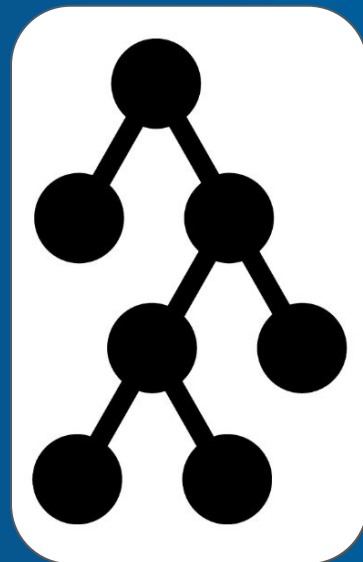


El secreto de BFS: La Cola FIFO

- ¿Cómo logramos que el algoritmo respete los niveles? Utilizando una estructura de datos tipo Cola (Queue).
- Regla FIFO (First In, First Out): El primer nodo que entra a la Frontera es el primero en ser extraído para su expansión.
- El efecto: Los nodos descubiertos recientemente (los hijos) se forman al final de la fila, obligando al algoritmo a procesar primero a los nodos más antiguos (los del nivel superior).

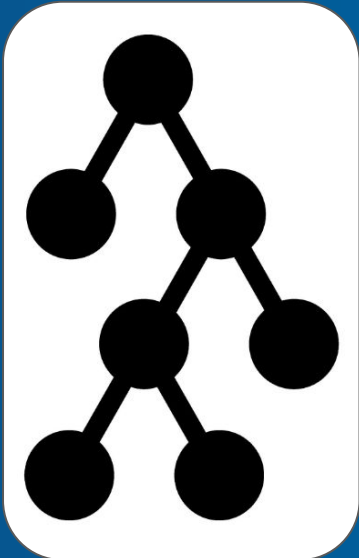
Ejecución paso a paso (Parte 1)

- Paso 1 (Nivel 0):
 - Frontera (Cola): [Nodo Raíz]
 - Acción: Extraemos la Raíz. Generamos sus 2 hijos y los metemos a la cola.
- Paso 2 (Nivel 1):
 - Frontera (Cola): [Hijo Izq, Hijo Der]
 - Acción: Extraemos el Hijo Izquierdo (entró primero).
 - Expansión: Generamos sus propios 2 hijos, los cuales se forman al final de la cola.



--	--

-



¿Es BFS útil en Ingeniería?: Ejemplo

- Ruta A (Directa): [Nodo Inicio] -----> [Nodo Meta]
 - **Pasos (Saltos):** 1
 - **Costo en dinero:** \$10.000 (Ej: Excavar roca)
- Ruta B (Con desvío): [Nodo Inicio] ---> [Nodo Intermedio] ---> [Nodo Meta]
 - **Pasos (Saltos):** 2
 - **Costo en dinero:** \$500 + \$500 = \$1.000 (Ej: Terreno blando)
- Evaluación Teórica de BFS:
 - **Complejidad:** Sí. (Siempre va a encontrar una forma de llegar a la meta).
 - **Optimalidad en Ingeniería:** NO. (BFS elegirá la Ruta A, gastando \$10.000, solo porque tiene "menos saltos").
 - Si, solo cuando todas las acciones cuestan lo mismo (grafo no ponderado).

El Talón de Aquiles de BFS: El devorador de memoria RAM

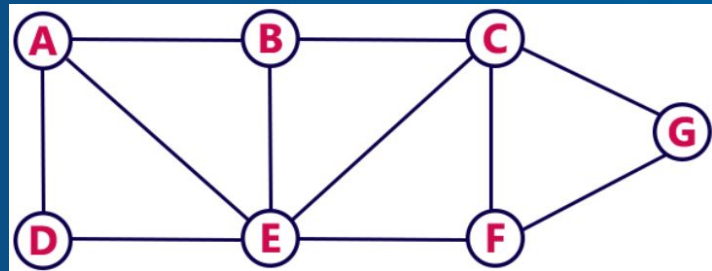
- Complejidad de la CPU (Tiempo): $O(b^d)$
 - b Es el número máximo de "hijos" (o vecinos) que puede tener un nodo.
 - d Es el número de "saltos" o pasos de distancia que hay desde el estado inicial (la raíz) hasta la solución más cercana (el objetivo)
- Matemática de la RAM (Espacio): $O(b^d)$
- **El Colapso:** Como BFS debe guardar todo un nivel en la "Fila" antes de avanzar, si en cada cruce hay 10 opciones ($b=10$) y la meta está a 5 cuadras ($d=5$):
 - Necesitaría mantener $10^5 = 100.000$ nodos en memoria RAM simultáneamente.

Ejemplo



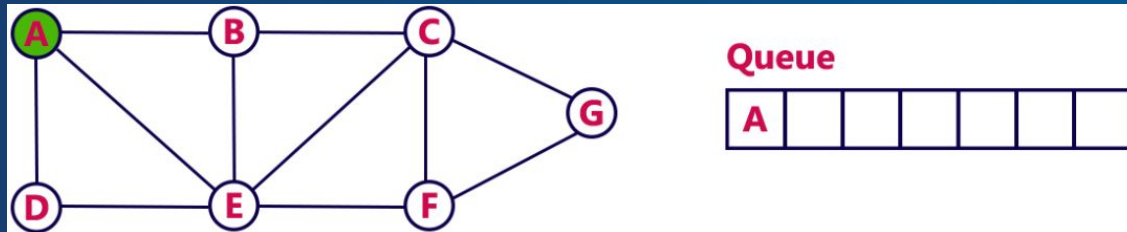
Ejemplo Práctico de BFS

- Nodos: A, B, C, D, E, F, G
- Reglas de nuestro motor para este ejemplo:
 - Iniciaremos el recorrido en el Nodo A.
 - Al expandir, si hay empate, insertaremos los nodos en la cola en orden alfabético inverso o según el orden de descubrimiento del grafo (D, E, B).
 - Usaremos una Cola (Queue) con la regla FIFO.



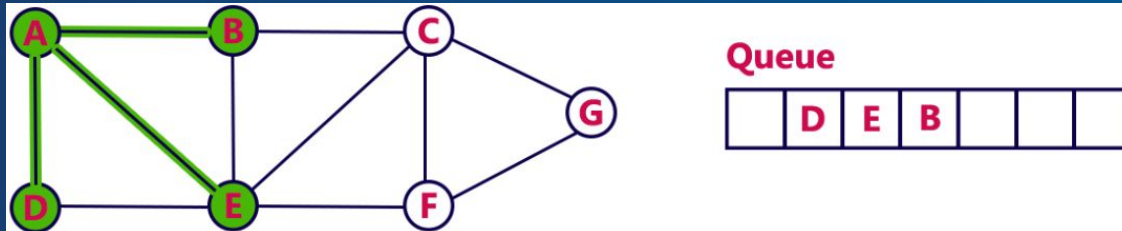
Paso 1 - Inicialización

- Paso 1: Seleccionar la Raíz
 - Grafo: El nodo A se marca en color verde (visitado).
 - Estado de la Cola (Frontera): [A]
- Acción del Algoritmo:
 - Seleccionamos el vértice A como punto de partida.
 - Marcamos A como visitado.
 - Insertamos A en la Cola.



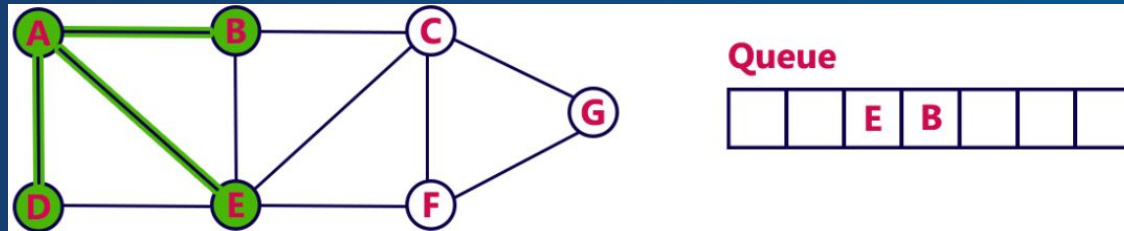
Paso 2 - Expandiendo el Nodo A

- Grafo: Las aristas de A hacia D, E y B se marcan (hijos descubiertos).
- Estado de la Cola: [D | E | B]
- Acción del Algoritmo:
 - Extraemos A de la cola (¡Adiós A!).
 - Visitamos todos los vértices adyacentes a A que NO hayan sido visitados: D, E, B.
 - Insertamos estos nuevos vértices en la Cola.



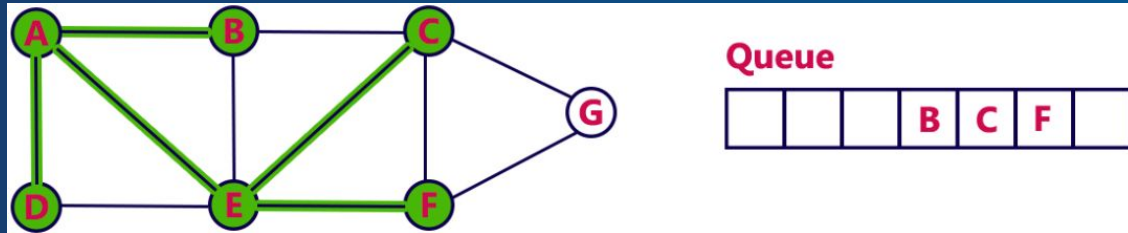
Paso 3 - Evaluando un callejón sin salida (D)

- Grafo: Nodo D en foco. Sus conexiones (A y E) ya están marcadas.
- Estado de la Cola: [E | B]
- Acción del Algoritmo:
 - Extraemos D de la parte frontal de la cola.
 - Visitamos los adyacentes de D no visitados.
 - ¡No hay ninguno! (A ya fue visitado y E ya está en la cola/visitado).
 - No se inserta nada nuevo en la cola.



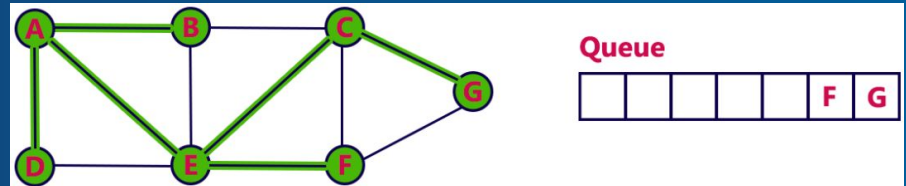
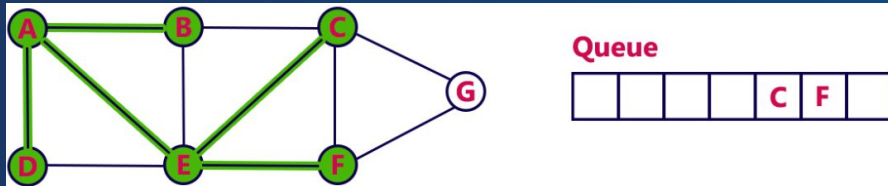
Paso 4 - Expandiendo el Nodo E

- Grafo: Nodo E en foco. Descubre a C y F.
- Estado de la Cola: [B | C | F]
- Acción del Algoritmo:
 - Extraemos E de la cola.
 - Visitamos los adyacentes de E no visitados: C y F. (Ignoramos A, B y D porque ya están procesados o en la cola).
 - Insertamos C y F al final de la Cola.



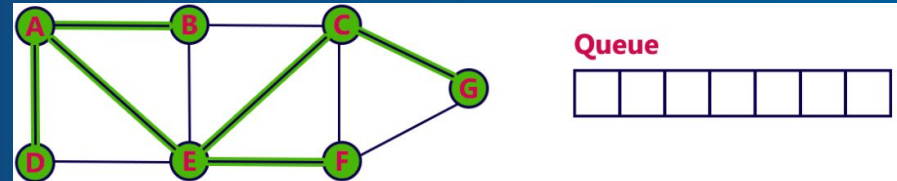
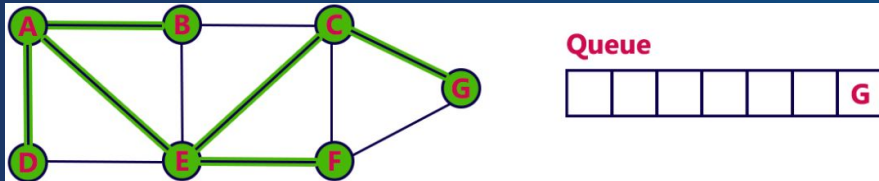
Pasos 5 y 6 - Avanzando por la cola (B y C)

- Grafo: El nodo G se descubre cuando se procesa C.
- Cola Paso 5 (Extraer B): [C | F] (B no aporta nodos nuevos).
- Cola Paso 6 (Extraer C): [F | G] (C descubre al nodo G).
- Acción del Algoritmo:
 - Paso 5: Extraemos B. Sus adyacentes (A, E, C) ya son conocidos. No aporta nada.
 - Paso 6: Extraemos C. Su único adyacente nuevo es G. Insertamos G al final de la cola.



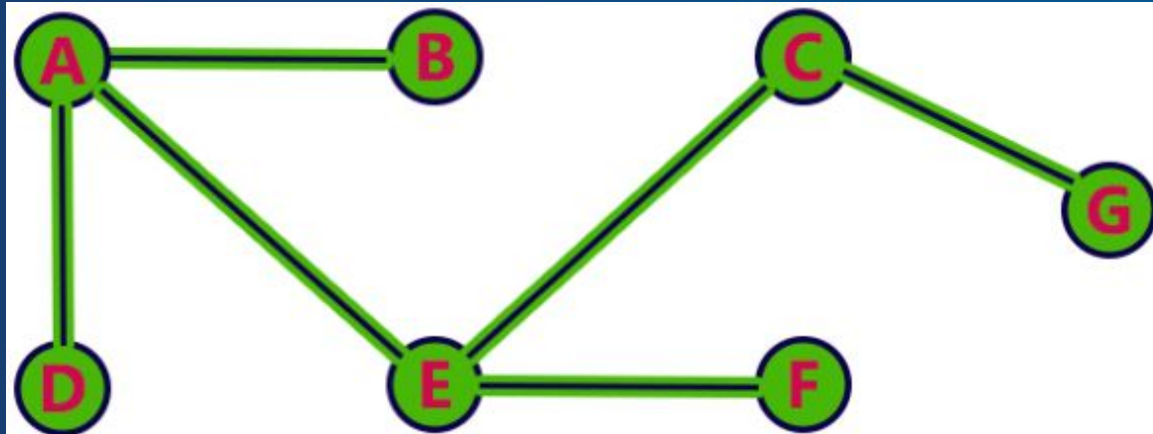
Pasos 7 y 8 - Vaciando la Cola y Fin

- Árbol de expansión resultante resaltado.
 - Cola Paso 7 (Extraer F): [G] (F no aporta nada).
 - Cola Paso 8 (Extraer G): [] (¡Cola Vacía!)
- Acción del Algoritmo:
 - Paso 7: Extraemos F. No hay adyacentes nuevos.
 - Paso 8: Extraemos G. No hay adyacentes nuevos.



Fin

- La cola está vacía.
- El proceso BFS se detiene.
- El resultado final es un Árbol de Expansión (Spanning Tree) sin ciclos.

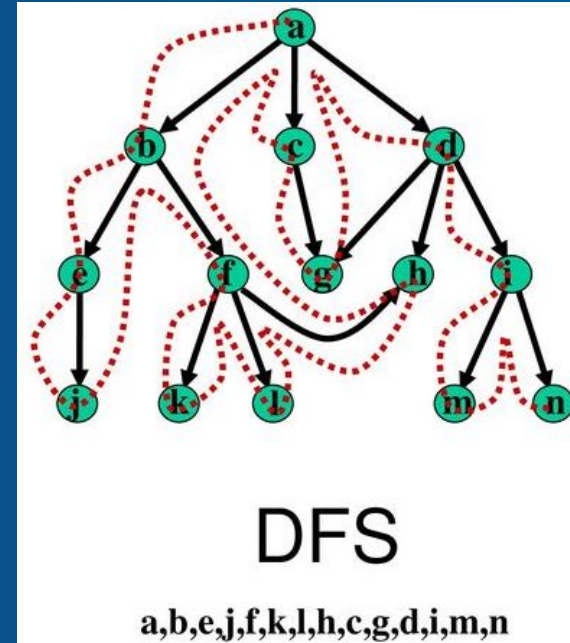


Búsqueda en Profundidad (DFS - Depth First Search)



Búsqueda en Profundidad (DFS)

- A diferencia de la onda expansiva, DFS es como explorar un laberinto con una linterna. Toma un camino y lo sigue hasta chocar con una pared o un callejón sin salida.
- Backtracking: Cuando llega al fondo y se da cuenta de que no es la meta, da un paso atrás (retrocede al nodo anterior) y prueba la siguiente opción disponible.
- Nunca revisa las ramas "hermanas" hasta haber agotado por completo la rama actual.



¿Qué diferencia tiene con preorden, inorden y postorden?

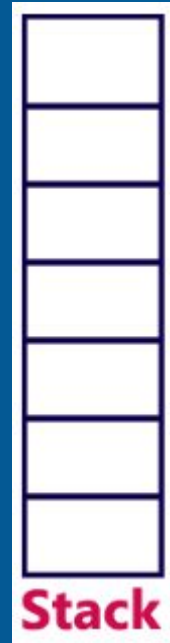
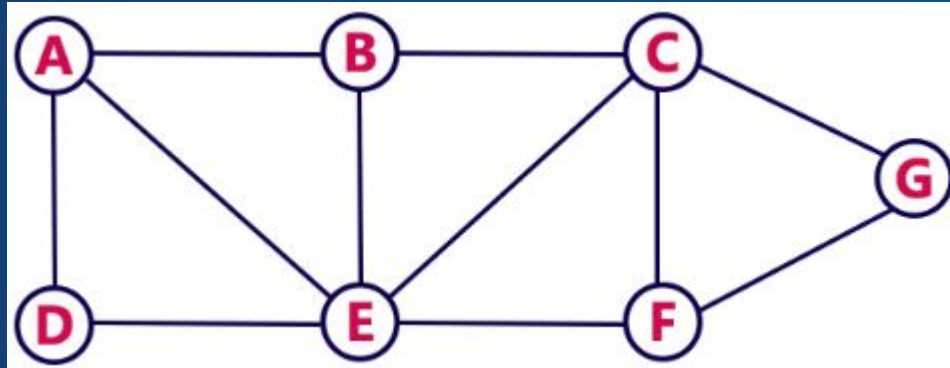
- DFS es una estrategia general de recorrido/búsqueda en grafos o árboles.
- preorden, inorden y postorden son formas específicas de recorrer un árbol binario, aunque los tres también son recorridos en profundidad (DFS).

La Mecánica de DFS (La Pila LIFO)

- Para lograr este comportamiento, cambiamos la estructura de nuestra Frontera de una Cola a una Pila (Stack).
- Regla LIFO: El último nodo en entrar a la Frontera es el primero en ser extraído.

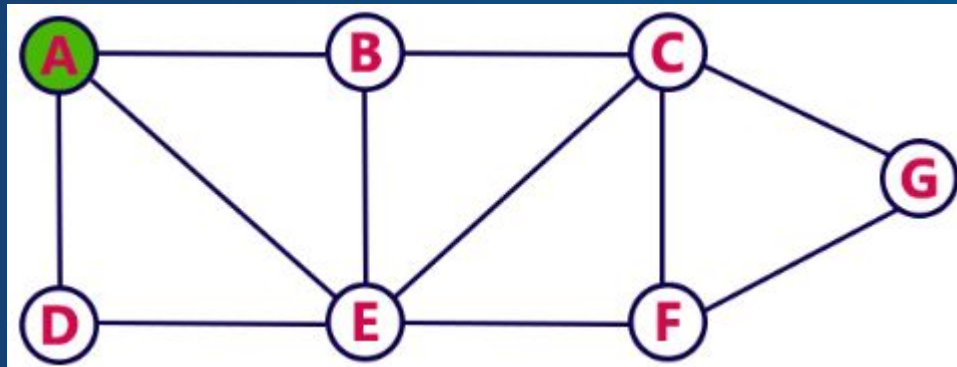
DFS: Ejecución paso a paso

- Resolveremos el mismo ejercicio, comenzando en A y terminando en G.
- Regla de desempate: Si hay múltiples vecinos no visitados, elegimos siempre el orden alfabético (puede ser otro criterio).
- Usaremos una Pila (Stack).



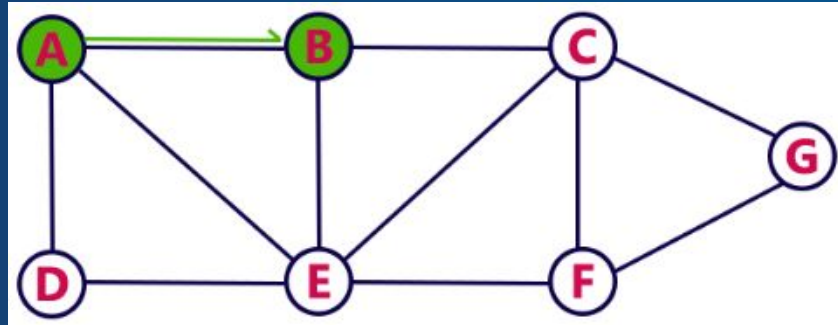
DFS: Paso 1 - Inicialización (Nodo A)

- El nodo A es visitado.
- Estado de la Pila: [A] (La base y la cima es A)
- Acción:
 - Marcamos A como visitado.
 - Insertamos A en la Pila.



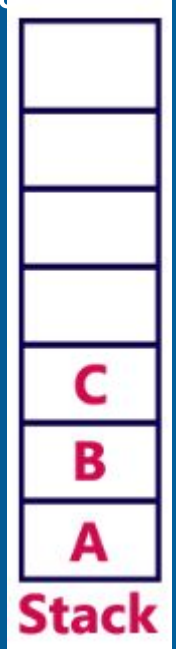
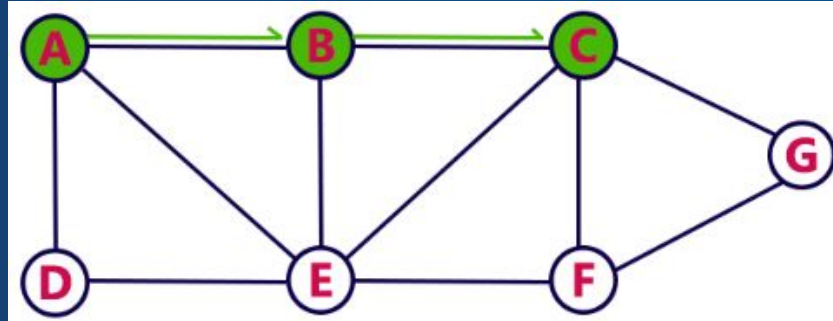
DFS: Paso 2 - Profundizando desde A hacia B

- Estado de la Pila: [B, A] (A abajo, B en la cima)
- Acción del Algoritmo:
 - El procesador mira la cima de la pila: es A.
 - ¿Tiene vecinos no visitados? Sí: B, D, E.
 - Por orden alfabético, elegimos un solo camino: B.
 - Marcamos B como visitado y lo apilamos encima de A.



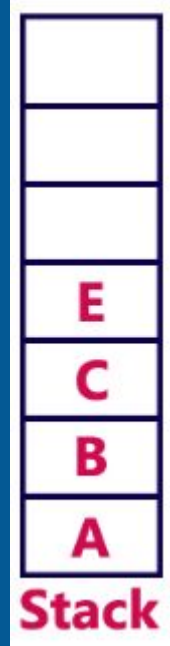
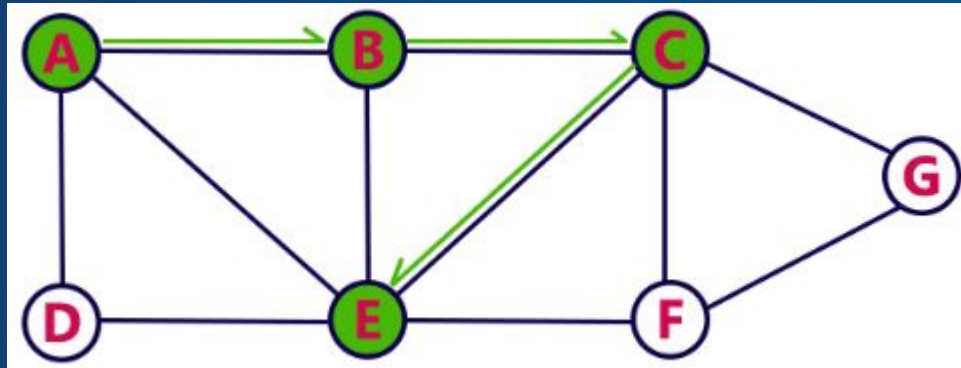
DFS: Paso 3 - Profundizando desde B hacia C

- Visitamos a un nodo adyacente de B que no haya sido visitado antes (C).
- Apilamos a C.



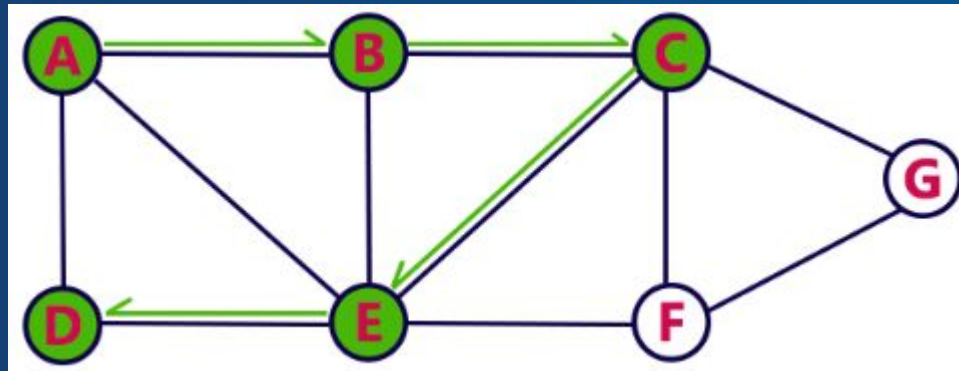
DFS: Paso 4 - Profundizando desde C hacia E

- Visita a un vértice adyacente de C que no haya sido visitado (E).
- Apilamos a E.



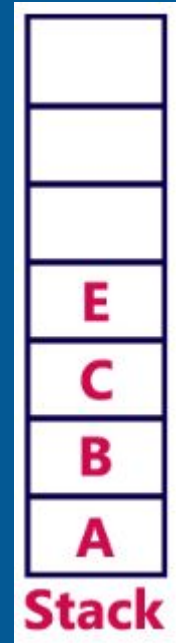
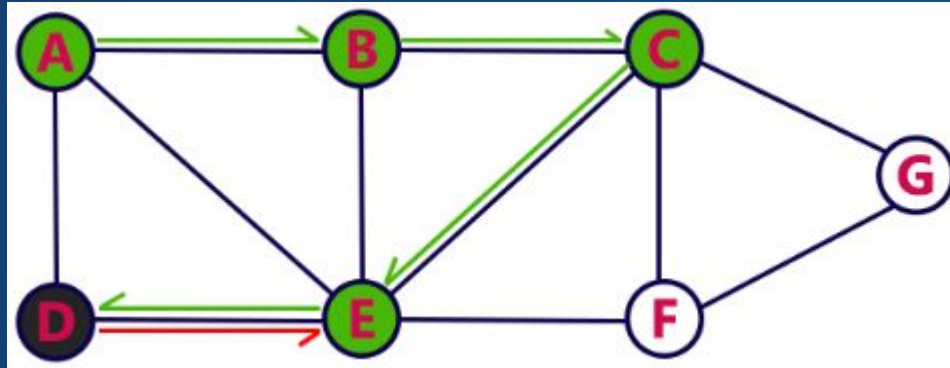
DFS: Paso 5 - Profundizando desde E hacia D

- Visitamos algún vértice adyacente a E que no haya sido visitado (D).
- Apilamos D.



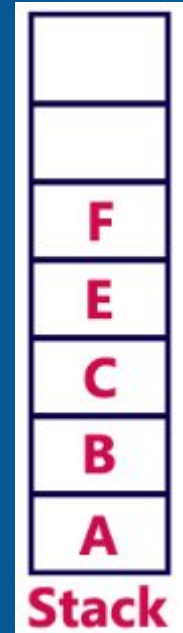
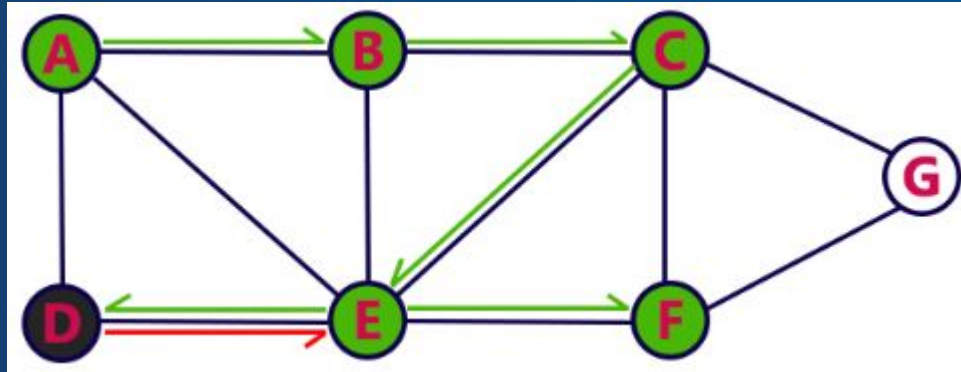
DFS: Paso 6 - Se acabaron las opciones de D

- Ya no tenemos vértices para visitar desde D.
- Se utiliza backtracking, desapilamos a D.



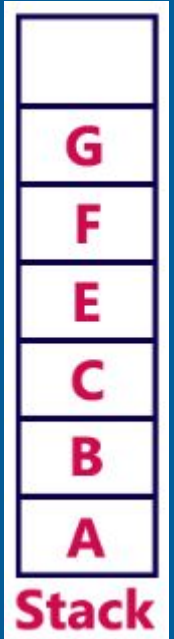
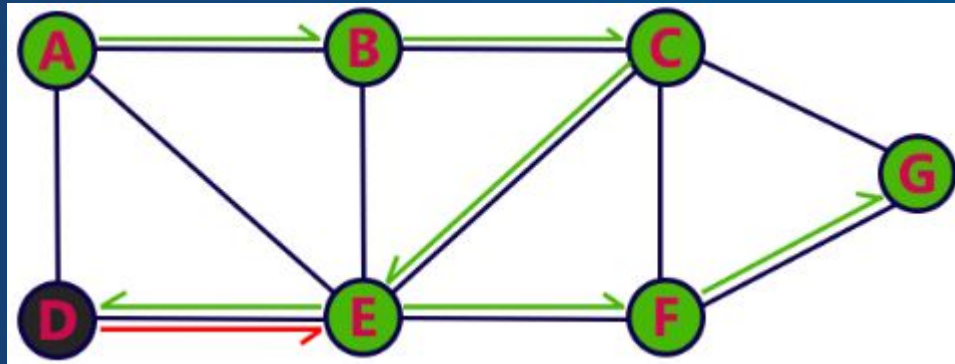
DFS: Paso 7 - Volvemos a buscar desde E (F)

- Visitamos algún vértice adyacente disponible de E que no haya sido visitado (F).
- Apilamos F.

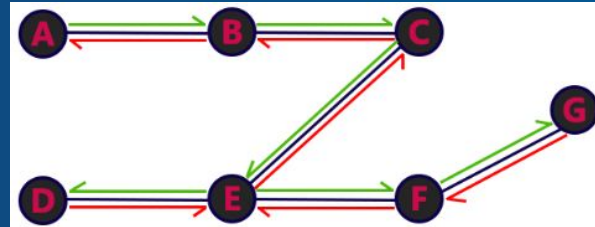
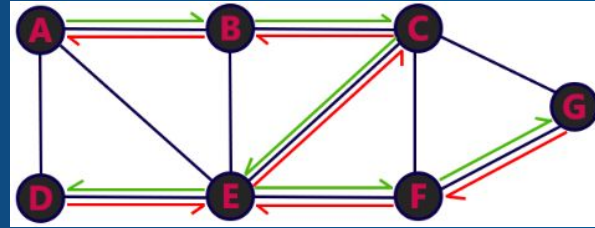
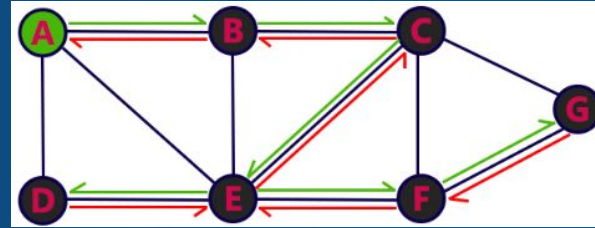
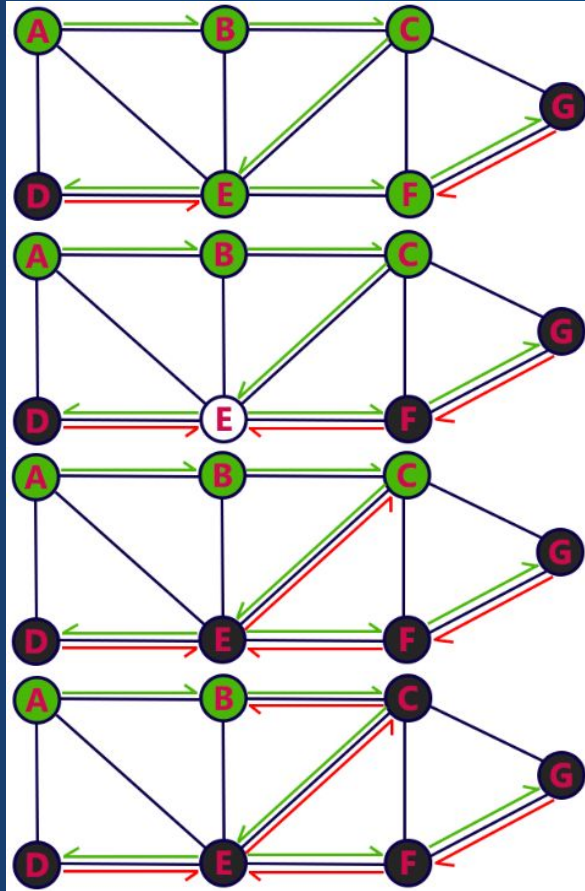


DFS: Paso 8 - Profundizando desde F hacia G

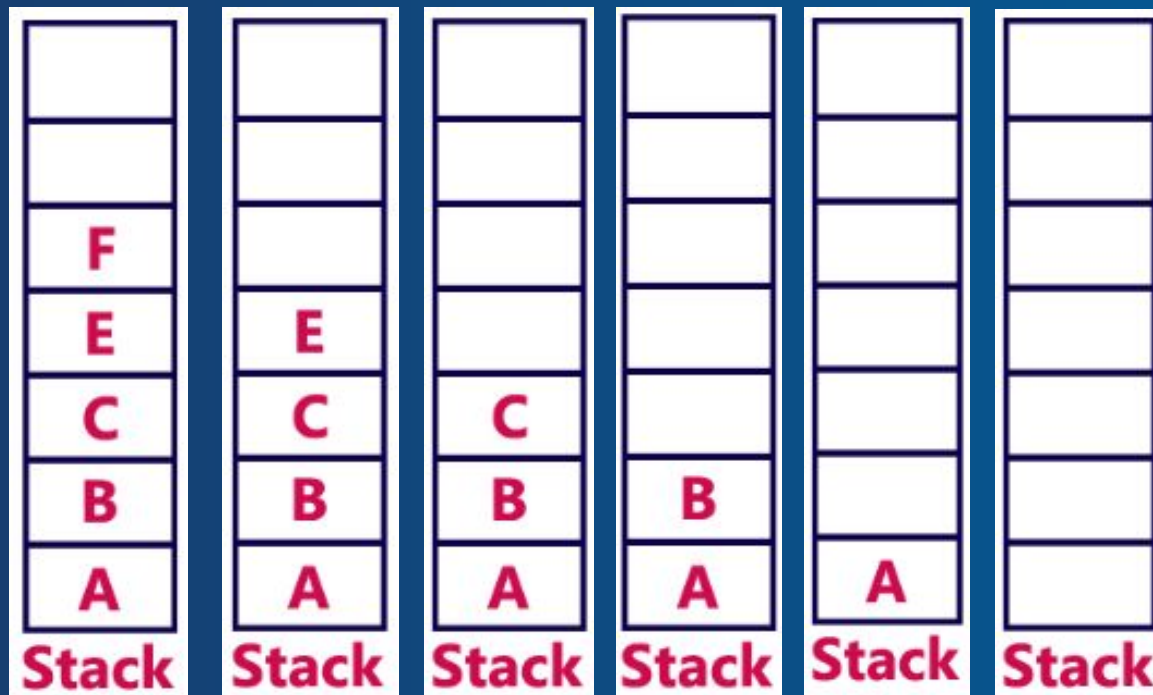
- Visitamos algún vértice adyacente de F que no haya sido visitado (G).
- Apilamos G.



DFS: Pasos finales (vaciado de la pila/Backtracking)



DFS: Pasos finales (vaciado de la pila)



Debilidades de DFS

- **Complejidad:** NO (Depende). Si el grafo tiene ciclos (y no programamos bien la lista de "Explorados"), DFS se quedará dando vueltas en ese ciclo para siempre, hundiéndose al infinito sin encontrar la meta.
 - Si el espacio de estados es infinito, DFS nunca volverá para explorar las otras ramas.
- **Optimalidad:** NO. DFS devuelve la primera solución que encuentra. Esa solución podría estar a 500 pasos de profundidad, ignorando por completo que en la rama derecha había una solución a solo 2 pasos de distancia.

La Gran Ventaja de DFS

- **El Milagro de la Memoria (Complejidad Espacial):** $O(b \cdot m)$
 - Donde b = ramificación, m = profundidad máxima.
- **Comparación Directa (Caso: $b=10$, profundidad 5):**
 - Memoria consumida por BFS (Onda expansiva): $10^5 = 100.000$ nodos esperando.
 - Memoria consumida por DFS (Linterna): $10 \times 5 = 50$ nodos.
- **Conclusión:** DFS solo necesita guardar en memoria la rama exacta por la que está caminando y los hermanos inmediatos de esa rama.

El Compromiso DFS-BFS (Uso Avanzado)

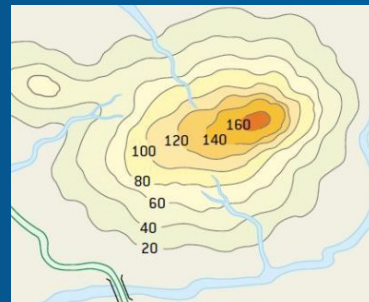
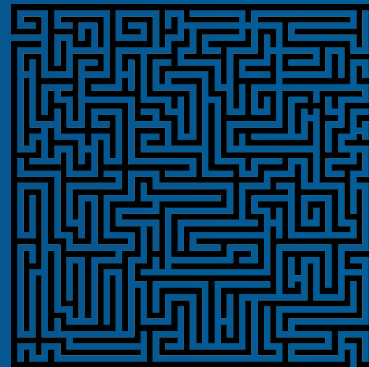


Búsqueda de Profundidad Iterativa (IDS)

- El dilema del ingeniero:
 - BFS: Encuentra la ruta más corta, pero colapsa la memoria RAM.
 - DFS: Gasta poquísima RAM, pero puede tomar la peor ruta o quedarse en un bucle infinito.
- La Solución Elegante: IDS (Iterative Deepening Search).
 - Ejecutamos DFS con un límite estricto de Profundidad = 1. Si no halla la meta, borramos la memoria.
 - Ejecutamos DFS con Profundidad = 2. Borramos la memoria.
 - Ejecutamos DFS con Profundidad = 3, y así sucesivamente.
- El Resultado: Obtenemos la optimalidad de BFS (explora por niveles) usando la bajísima memoria de DFS.

Búsqueda de Costo Uniforme (UCS)

- Mundo Real: Cuando un paso no es igual a otro
- Un laberinto clásico de videojuegos (todos los pasillos son iguales, cada paso cuesta 1).
- Un mapa topográfico con un río, una montaña y una carretera.
- El problema de BFS y DFS: Asumen que *Costo = Cantidad de pasos*.
- En ingeniería real: Tender fibra óptica en roca cuesta \$10,000/metro. Tenderla en un poste cuesta \$100/metro. Dar 1 paso en la roca es peor que dar 10 pasos por los postes.



Búsqueda de Costo Uniforme (UCS / Dijkstra)

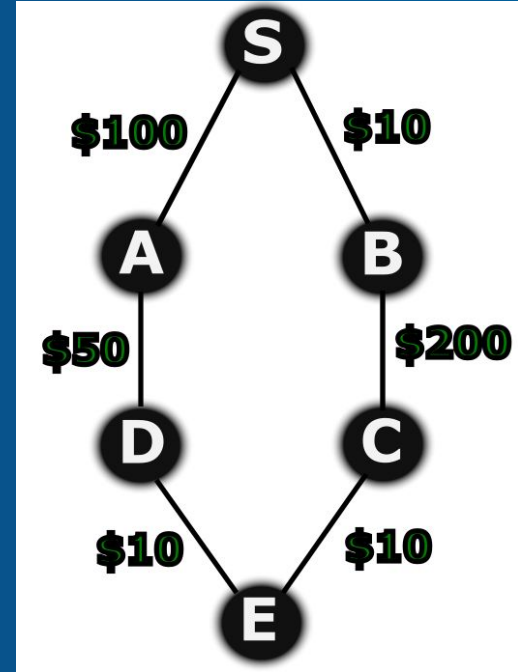
- ¿Qué es? Es la adaptación para Inteligencia Artificial del famoso Algoritmo de Dijkstra.
- La Regla de Oro: En lugar de expandir el nodo más superficial (BFS) o el más profundo (DFS), UCS siempre expande el nodo con el menor costo de ruta acumulado: $g(n)$.
- No le importa cuántos saltos haya dado; solo le importa cuánto "dinero" ha gastado desde el inicio hasta ese punto.

Mecánica de UCS

- El Cerebro de UCS: La Cola de Prioridad
- Estructura de Memoria: Ya no usamos pila ni cola. Usamos una Cola de Prioridad.
- ¿Cómo funciona? Cada vez que un nodo entra a la Frontera, no va ni al principio ni al final. Se inserta en la posición exacta que le corresponde según su valor $g(n)$, ordenado de menor a mayor.
- El procesador siempre extrae el nodo que esté en la cima, que garantizadamente será el más barato de toda la frontera.

UCS en Acción: El salto por costos

- Nodo Inicial S. Opciones: hacia A (Costo \$100) y hacia B (Costo \$10).
- Frontera ordenada: [B(\$10), A(\$100)].
- UCS expande B. Desde B, hay un camino hacia C (Costo \$200).
 - Costo total para llegar a C: $\$10 + \$200 = \$210$.
- Nueva Frontera: [A(\$100), C(\$210)].
- ¡El Salto! UCS se da cuenta de que seguir por B se volvió muy caro. ¡Se detiene, salta al otro lado del árbol, y comienza a explorar A!



Evaluación Teórica de UCS: El Costo de la Perfección

- Contenido visual de la diapositiva:
 - Completitud: Sí. (Si ningún costo sea negativo o cero).
 - Optimalidad: Sí. Es matemáticamente imposible que devuelva una ruta más cara si existe una más barata.
- Complejidad (Tiempo y RAM): $O(b^{1 + \lfloor (C^*)/\epsilon \rfloor}) \rightarrow$ Terrible.
 - Explora el mapa en "círculos concéntricos de costo" (como anillos de igual valor).
 - Problema: Al igual que BFS, UCS expande en todas direcciones tratando de agotar los caminos baratos. Gasta mucha RAM y mucho tiempo.

Resumen de Algoritmos Ciegos

Algoritmo	Consumo de RAM	¿Garantiza el mejor camino?	Ideal para...
BFS (Anchura)	Altísimo	Sí (solo si costo=1)	Problemas pequeños de pocos saltos.
DFS (Profundidad)	Muy Bajo	No (encuentra el primero)	Robots con poca RAM, explorar laberintos al azar.
IDS (Iterativa)	Muy Bajo	Sí (solo si costo=1)	Grafos gigantes donde no hay RAM pero necesitamos la ruta más corta.
UCS (Dijkstra)	Altísimo	Sí (Siempre)	Encontrar rutas baratas en grafos con pesos/costos reales.

Hacia la Búsqueda Informada

- **El problema de UCS:** Es como buscar una aguja en un pajar usando un detector de metales perfecto pero de corto alcance. Empiezas a caminar en círculos concéntricos desde donde estás, sin importar si la aguja está al norte o al sur.
- Preguntas:
 - ¿Qué pasaría si le diéramos a UCS un "GPS" o una "brújula"?
 - ¿Qué pasaría si el algoritmo supiera que la meta está hacia el Norte?
- Próximos contenidos: Búsqueda Informada, Heurísticas y el Algoritmo A* (A-Star).

*¿Qué diferencia tienen con
los algoritmos MST?*



1.- El Objetivo

- "Llegar a un destino" vs. "Conectar todo"
- **Algoritmos de Búsqueda (BFS, DFS, UCS/Dijkstra):** Son algoritmos de Enrutamiento (Pathfinding). Su objetivo es encontrar un camino específico desde un Punto A (Inicio) hasta un Punto B (Meta). Una vez que llegan a la Meta, el algoritmo se apaga. Les da exactamente igual si dejaron el 90% del mapa sin visitar.
- **Kruskal y Prim:** Son algoritmos de Árbol de Expansión Mínima. No hay un "Punto B" o Meta específica. Su objetivo es asegurar que absolutamente todos los nodos del grafo queden conectados entre sí, utilizando la menor cantidad de "cable" o costo total posible.

2.- La Analogía del Ingeniero

- **Usas UCS (Búsqueda):** Cuando eres un conductor de Uber y quieres ir desde tu ubicación actual hasta la casa del cliente gastando la menor cantidad de gasolina posible. Solo te importa esa ruta específica.
- **Usas Kruskal / Prim (MST):** Cuando eres el Ministro de Obras Públicas y tienes que pavimentar caminos para que todos los pueblos de una región queden conectados entre sí. No te importa una ruta en particular, te importa que nadie quede aislado gastando el mínimo de asfalto total.

3.- Diferencia en el Resultado (El Output)

- Salida de UCS: Una lista lineal de nodos (Ej: Inicio -> C -> F -> Meta).
- Salida de Kruskal/Prim: Un subgrafo (un árbol) que contiene todos los vértices originales, pero solo algunas aristas.

Ejercicios



Ejercicio 1: Topología de Red (No Ponderado)

- Tienes una topología de red simple de 6 routers (A, B, C, D, E, F). Todas las conexiones tienen el mismo costo (1 salto). Queremos encontrar una ruta desde el router A hasta el router F.
- Regla de desempate: En caso de múltiples vecinos, se encolan/apilan en orden alfabético.
- 1. Representación del Grafo en Memoria
- El grafo tiene las siguientes conexiones bidireccionales: A-B, A-C, B-D, B-E, C-E, E-F.

Ejercicio 1: Topología de Red (No Ponderado)

- Opción A: Lista de Adyacencia (Ideal para BFS/DFS en Python)
- Opción B: Matriz de Adyacencia (1 = hay conexión, 0 = no hay conexión)

```
A: [B, C]
B: [A, D, E]
C: [A, E]
D: [B]
E: [B, C, F]
F: [E]
```

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	0	1	1	0
C	1	0	0	0	1	0
D	0	1	0	0	0	0
E	0	1	1	0	0	1
F	0	0	0	0	1	0

Ejercicio 1: Topología de Red (No Ponderado)

1. Resolver usando BFS (Fila FIFO): Muestre el estado de la Cola paso a paso y la ruta final.
2. Resolver usando DFS (Pila LIFO): Muestre el estado de la Pila paso a paso y la ruta final.
3. Pregunta de análisis: ¿BFS y DFS encontraron la misma ruta? ¿Cuál es mejor y por qué?

Ejercicio 2: Trazado de Fibra Óptica (Ponderado / Costos)

- Eres el ingeniero a cargo. Debes conectar el Nodo Inicio (S) con el Nodo Meta (G).
- Los números representan el costo de instalación en miles de dólares.
- Regla: El algoritmo UCS usa una Cola de Prioridad basada en el costo total acumulado $g(n)$.
- 1. Representación del Grafo en Memoria: Conexiones dirigidas (para simplificar el ejercicio) y sus costos: $S \rightarrow A$ (100), $S \rightarrow B$ (10), $B \rightarrow C$ (200), $A \rightarrow G$ (50), $C \rightarrow G$ (20).

Ejercicio 2: Trazado de Fibra Óptica (Ponderado / Costos)

- Opción A: Lista de Adyacencia con Pesos (Tupla: Vecino, Costo)
- Opción B: Matriz de Adyacencia Ponderada (∞ = sin conexión directa)

```
S: [(A, 100), (B, 10)]
A: [(G, 50)]
B: [(C, 200)]
C: [(G, 20)]
G: []
```

	S	A	B	C	G
S	0	100	10	∞	∞
A	∞	0	∞	∞	50
B	∞	∞	0	200	∞
C	∞	∞	∞	0	20
G	∞	∞	∞	∞	0

Ejercicio 2: Trazado de Fibra Óptica (Ponderado / Costos)

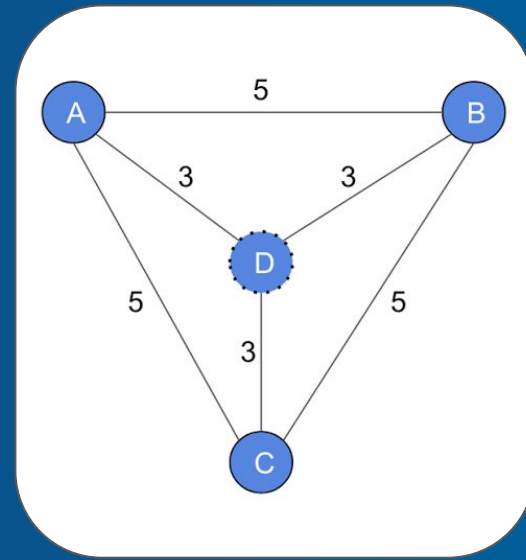
1. Resolver usando UCS (Cola de Prioridad): Dibuje la tabla con la Cola de Prioridad en cada iteración. Debe quedar claro en qué momento el algoritmo hace un "salto" de una rama a otra por culpa del costo.
2. Pregunta de análisis: Si hubiéramos usado BFS en este mismo mapa (ignorando los costos), ¿qué ruta habría elegido y cuánto dinero habríamos perdido?

Problema de los nodos de Steiner



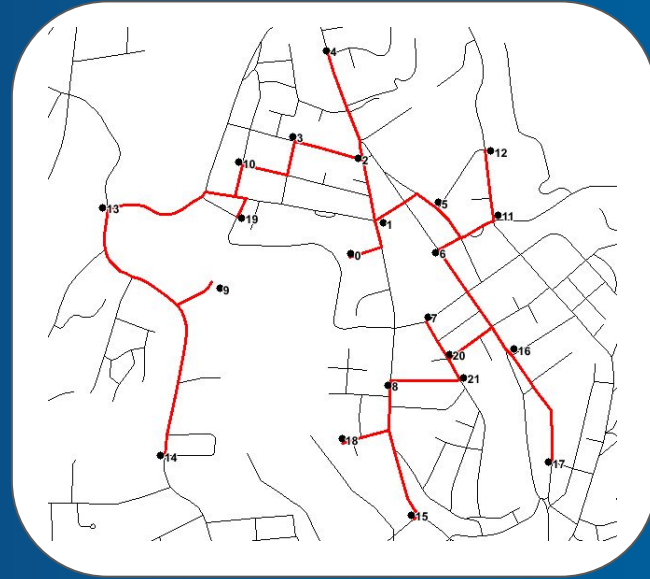
Problema de los nodos de Steiner

- En el diseño de redes de infraestructura (como telecomunicaciones, transporte o distribución eléctrica), uno de los principales desafíos consiste en conectar un conjunto de puntos obligatorios minimizando el costo total de la red.
- En muchos casos, es posible incorporar nodos intermedios adicionales que no forman parte del conjunto original, pero que permiten reducir significativamente el costo de conexión.



Problema de los nodos de Steiner

- Consiste en encontrar un subgrafo de costo mínimo que conecte un conjunto de nodos obligatorios (terminales), permitiendo opcionalmente el uso de nodos adicionales (nodos de Steiner) para reducir el costo total del árbol.

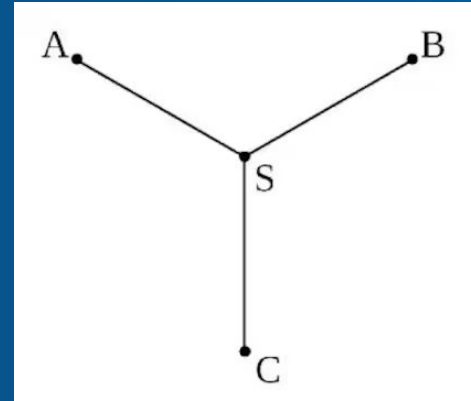


Contexto histórico

- El problema proviene de la geometría clásica.
- Relacionado con el matemático: Jakob Steiner (Siglo XIX).
- El problema original: ¿Cómo conectar un conjunto de puntos con la menor longitud total posible?
- Posteriormente el problema fue generalizado a:
 - grafos
 - optimización combinatoria
 - diseño de redes
- Hoy es un problema fundamental en optimización de redes.

Intuición del problema

- Supongamos que queremos conectar tres ciudades.
- Sin nodos adicionales: Conectamos las ciudades directamente.
- Pero si permitimos agregar un nodo intermedio: Se puede construir una red más corta.
- Este punto intermedio es un: Nodo de Steiner
- Resultado:
 - menor longitud total
 - red más eficiente
- El árbol de Steiner siempre es menor o igual al MST.



Complejidad del problema

- El problema del árbol de Steiner es: NP-hard
- Esto significa que: No se conoce un algoritmo eficiente que lo resuelva exactamente para grandes instancias.
- En consecuencia, se utilizan:
 - heurísticas
 - aproximaciones
 - metaheurísticas
 - algoritmos evolutivos
- Por eso es un problema muy estudiado en IA.

Aplicaciones reales

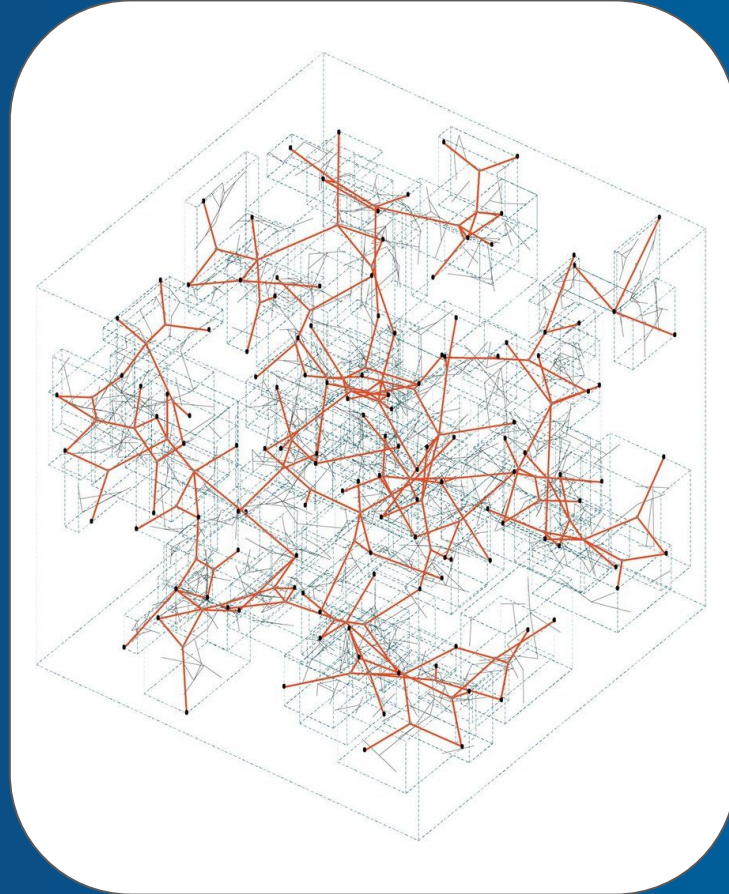
- El problema aparece en muchos sistemas reales:
 - Redes de telecomunicaciones
 - Diseñar cables con menor longitud.
 - Diseño de circuitos electrónicos
 - Optimización en chips VLSI.
 - Redes de transporte
 - Conectar ciudades con infraestructura mínima.
 - Redes de sensores
 - Minimizar energía y distancia de comunicación.

Relación con IA y optimización

- El Steiner Tree es un problema clásico de optimización combinatoria.
- Se usa frecuentemente para evaluar:
 - algoritmos genéticos
 - recocido simulado
 - colonia de hormigas
 - búsqueda local
 - algoritmos evolutivos
- Por eso aparece frecuentemente en papers de IA y optimización.

Problema de los nodos de Steiner

- En esta actividad, se trabajará con una instancia real del repositorio SteinLib, disponible en: <https://steinlib.zib.de/showset.php?E> , el cual es incluido junto a este enunciado además.
- La instancia proporcionada posee las siguientes características:
 - Número total de nodos: 2500
 - Nodos obligatorios (terminales): 417
 - Nodos opcionales (Steiner): 2083
 - Número de aristas: 62500



Requisito para llevar a cabo la actividad

- El objetivo es encontrar la mejor solución factible posible, es decir, un árbol que conecte todos los nodos terminales con el menor costo total posible, permitiendo el uso de nodos de Steiner.
- Requerimientos del sistema
 - El sistema debe leer y procesar la instancia entregada, modelando el problema como un grafo ponderado no dirigido, donde:
 - Los nodos representan puntos de conexión.
 - Las aristas representan posibles conexiones con un costo asociado
 - Existe un subconjunto de nodos que deben ser obligatoriamente conectados (terminales).

Proceso de desarrollo de la actividad (i)

1. **Construcción del grafo:** Leer correctamente el archivo de entrada (formato SteinLib) y representar el grafo.
2. **Solución base:** Construir una solución inicial utilizando un: Árbol de expansión mínima (MST) sobre el grafo completo o sobre una transformación del problema. Esta solución debe: Ser conexa, Incluir todos los nodos terminales y Servir como referencia base de comparación.
3. **Mejora de la solución:** El sistema deberá implementar al menos una técnica de mejora, a elección del grupo:
 - a. Heurística constructiva. Ej: inserción progresiva de nodos de Steiner
 - b. Búsqueda local: Adaptación de técnicas como:
 - c. intercambio de aristas
 - d. eliminación de caminos redundantes

Proceso de desarrollo de la actividad (ii)

4. Optimización estructural: Se debe implementar un proceso de poda de nodos innecesarios, especialmente:
 - a. nodos de grado 1 que no sean terminales
 - b. estructuras redundantes que aumenten el costo.
5. Validación de la solución: Se deben implementar algoritmos de verificación (DFS o BFS). Toda solución generada debe cumplir:
 - a. Inclusión de todos los terminales.
 - b. Estructura de árbol conexo.
6. Métricas de evaluación: El sistema debe calcular:
 - a. Costo total de la solución
 - b. Comparación con la solución base (MST)
 - c. Reducción porcentual de costo

Indicaciones del reporte

- El reporte debe incluir
 - Descripción del problema y modelación
 - Representación del grafo
 - Algoritmo base implementado (MST)
 - Estrategia de generación de solución factible
 - Método(s) de mejora utilizado(s)
 - Resultados obtenidos
 - Comparación entre soluciones
 - Dificultades encontradas
- Día subida viernes 16 de Octubre (día de la prueba).
- Equipos de 3 o 4 integrantes.

Ponderación

- Ejercicio total tendrá 15 puntos de la evaluación 2.
 - **1-3 puntos:** solución parcial (no conexo, elimina terminales, etc).
 - **4 puntos:** solución base MST (costo 2515).
 - **5 puntos:** solución base MST podada correctamente (costo 892).
 - **6-7 puntos:** solución factible de mala calidad ([601,891]).
 - **8-10 puntos:** solución factible de moderada calidad ([581,600]).
 - **11-14 puntos:** solución factible de buena calidad ([565,580]).
 - **15 puntos:** solución óptima (564).



Ingeniería Civil Informática

Inteligencia artificial

Fin de la clase

Dr(c). Francisco Philip Vásquez Iglesias

